RENDERING VANA'DIEL

Inside Final Fantasy XI's Graphics Technology and Building a Resource Explorer

English edition • Working manuscript • Edition 0.1

How to read this book

This textbook explains Final Fantasy XI's rendering techniques and graphics architecture at the depth required to reconstruct them in an independent resource explorer and visualizer. Technical explanation, evidence evaluation, C++ implementation, and desktop-application integration form one cumulative course.

Editorial boundary. This work is limited to the game's graphics technology and its technical context.

Implementation track. Every chapter advances one cumulative C++ desktop application. The completed program discovers an installation, catalogs resources, inspects binary structures, decodes supported DAT content, displays models, zones, characters, and effects, browses text and audio, exports documented results, restores investigations, and runs structural and visual regressions. Listings connect to the module contract established by earlier chapters, and every checkpoint implements the feature it names.

Contents

17. Alpha Test, Alpha Blending, and Draw Order
Build outcome: Decode alpha-related fields and build opaque, cutout, and blended queues with deliberate depth and ordering rules.

18. Texture Stages and Fixed-Function Combiners
Build outcome: Parse material-stage records into a loggable fixed-function texture-stage and combiner description system.

19. Vertex Shaders and Programmable Techniques
Build outcome: Load validated shader records, declarations, and constants with capability checks and an explicit fallback path.

Part III — Scene Construction and Interactive Workspaces

20. The World Scene: Zones, Chunks, and Draw Groups
Build outcome: Parse a zone resource graph and construct the world scene from validated chunks, entities, references, and draw groups.

21. Zone Loading and Scene Assembly
Build outcome: Implement dependency-aware zone loading and publish an immutable scene snapshot suitable for rendering and inspection.

22. Object Explorer, Selection, and Property Editing
Build outcome: Build synchronized tree, viewport, and property inspection with stable IDs, highlighting, reversible overrides, and source navigation.

23. Collision Geometry and Navigation Diagnostics
Build outcome: Implement collision inspection, acceleration, ray queries, optional grounding, and visual discrepancy overlays with explicit confidence labels.

24. Visibility: Frustum Culling, Distance, and Occlusion
Build outcome: Decode supported visibility fields and implement frustum, distance, and optional occlusion tests while preserving unknown flags.

25. Terrain, Ground, and Large-Scale Environment Geometry
Build outcome: Parse terrain sectors into reusable ground geometry, materials, bounds, and large-environment submissions.

26. Static Objects, Props, and Architectural Detail
Build outcome: Parse model and placement records, then instantiate static props and architecture with shared geometry and per-instance state.

27. Characters: Assembly, Skinning, and Equipment
Build outcome: Decode validated character components and assemble equipment selections, skeleton bindings, and skinned geometry.

28. Character Assembly Workspace
Build outcome: Build a character assembly workspace with slot resolution, compatibility checks, asynchronous replacement, provenance, and presets.

29. Animation, Pose Evaluation, and Motion Data
Build outcome: Maintain explicit SkeletonTime, SchedulerTime, GeneratorTime, and ParticleAge values. Do not reuse one frame counter for all four systems.

30. Animation Browser and Playback Controller
Build outcome: Build an animation catalog and transport controller that drives the chapter’s skeletal evaluator and exposes every fallback.

31. Lighting Models, Vertex Color, and Material Response
Build outcome: Translate decoded material, light, vertex-color, and texture-stage inputs into an inspectable shading configuration.

Part IV — Atmosphere, Effects, and Content Tools

32. Sky, Celestial Objects, and Time of Day
Build outcome: Keep environment interpolation, sky-dome drawing, and generator execution as separate modules joined by decoded resource identities.

33. Fog, Distance Haze, and Color Grading
Build outcome: Parse supported fog parameters and expose distance haze and scene-color controls for runtime inspection.

34. Water, Reflections, and Surface Motion
Build outcome: Complete the chapter's integrated application checkpoint.

35. Weather: Rain, Snow, Wind, and Ambient Particles
Build outcome: Implement WeatherState, EnvironmentState, and PrecipitationRuntime as separate objects. Connect them through weather tags and decoded generator references, not shared global guesses.

36. Environment Control and Comparison Workspace
Build outcome: Build an environment workspace that edits, compares, captures, and reports decoded state separately from reconstruction policy.

37. Particle Systems, Trails, and Spell Effects
Build outcome: Finish this chapter with a viewer that can pause at any simulation frame and display generator source offsets, decoded commands, unknown payloads, live particles, curve samples, child events, and the exact render descriptor for each draw.

38. Effect Browser and Simulation Workspace
Build outcome: Build a deterministic effect workspace with command, simulation, rendering, and evidence views driven by one timeline.

39. Transparency, Sorting, and Multi-Pass Effects
Build outcome: Translate supported pass records into transparent queues, target changes, sorting policies, and multi-pass composition.

40. Shadows, Decals, and Projected Visuals
Build outcome: Decode supported projected-visual records behind interchangeable shadow, decal, and projector interfaces.

41. Interface Rendering and 2D Composition
Build outcome: Parse supported interface records and build an orthographic renderer that composes evidence-scoped 2D layers.

42. Text, Icons, Fonts, and Window Layers
Build outcome: Decode supported font, glyph, icon, and window records, then implement text layout, atlases, clipping, and layering.

43. Text, Dialog, and Message Resource Explorer
Build outcome: Build a layout-pluggable text explorer with token-aware decoding, search, provenance, and lossless diagnostic export.

44. Audio Catalog, Decode, and Playback
Build outcome: Build a searchable audio workspace with safe header parsing, asynchronous decode, playback transport, looping, and unsupported-codec reporting.

Part V — Desktop Application, Presentation, and Performance

45. Resolution, Aspect Ratio, and Display Scaling
Build outcome: Separate decoded presentation settings, internal rendering dimensions, output resolution, viewport, aspect ratio, and UI scaling.

46. Desktop Shell, Docking, and Workspace Lifecycle
Build outcome: Build the multi-document shell and workspace factory/lifecycle contract used by every explorer and viewer.

47. Commands, Menus, Settings, and Tool Windows
Build outcome: Implement typed commands and versioned settings that drive menus, shortcuts, dialogs, and tool windows through one policy.

48. Input, Cameras, Picking, and Navigation Modes
Build outcome: Build configurable input, four navigation modes, reliable pointer capture, and selection rays consistent with the rendered viewport.

49. State Changes, Batching, and Resource Reuse
Build outcome: Add state caching, draw sorting, batching, and resource reuse without erasing source identity or required order.

50. CPU–GPU Work Distribution
Build outcome: Measure the full asset-to-frame path with parse/decode/upload timings and CPU-GPU frame telemetry.

51. Background Jobs, Cancellation, and UI Handoffs
Build outcome: Build the reusable job system used by cataloguing, decode, streaming, audio, screenshots, and export.

52. Memory Budgets, Streaming, and Loading Behavior
Build outcome: Build dependency-aware streaming, caches, eviction, and recoverable handling for missing, corrupt, or unsupported records.

53. Resource Caches and Explicit Lifetime
Build outcome: Implement layered CPU/GPU caches with stable handles, coalesced loads, budgets, diagnostics, and device-loss recovery.

54. Performance Tradeoffs and Legacy Hardware Design
Build outcome: Enforce configurable budgets and benchmark file decoding, resource preparation, rendering, and presentation independently.

55. Screenshots, Reports, and Data Export
Build outcome: Build transactional screenshot, table, texture, geometry, and report exports with manifests and explicit conversion policy.

56. Sessions, Presets, and Reproducible Workspaces
Build outcome: Implement versioned sessions and focused presets that reproduce an inspection without serializing runtime-only state.

Part VI — Case Studies, Verification, and Delivery

57. Reading an Opaque Environment Draw
Build outcome: Build an opaque environment case study that verifies source bytes, parsed records, resources, state, depth behavior, and draw order.

58. Reading a Character Draw
Build outcome: Build a character case study exposing each decoded component, binding, animation, material, equipment, and draw decision.

59. Reading a Transparent Effect
Build outcome: Export the effect-frame record as machine-readable JSON and a human-readable report so parser changes can be diffed across revisions.

60. Reconstructing a Frame from Evidence
Build outcome: Create a frame-inspection mode that links file evidence to parsed records, resources, state transitions, and intermediate targets.

61. Confirmed Architecture, Open Questions, and Competing Models
Build outcome: Maintain architecture, header, and flag registries with provenance, confidence, contradictions, and reproducible next tests.

62. Lessons from Vana’diel’s Visual Technology
Build outcome: Integrate the verified DAT-reading and rendering systems into a testable visual client with regression scenes and documented extension points.

63. Testing, Fuzzing, and Regression Hardening
Build outcome: Build a layered test runner and fuzz entry point that exercise every decoder and application boundary without requiring interactive operation.

64. Packaging, Documentation, and Safe Extension
Build outcome: Produce a toolchain-agnostic release layout, manifest, first-run flow, diagnostic bundle, extension registry, and final acceptance checklist.

Appendices A–J: API, render states, combiners, mathematics, evidence register, glossary, chronology, sources, and index.

Part I — Context, Method, and Application Plan

7. The Device, Resources, and Frame Lifecycle
Build outcome: Implement safe DAT lookup and binary input, an evidence-scoped decrypt/deobfuscate/decompress dispatcher, graphics-device setup, resource creation, presentation, and recovery.

8. Installation Discovery and Resource Maps
Build outcome: Implement safe installation discovery and return a complete validation report rather than a single Boolean result.

9. The Resource Catalog and Browser
Build outcome: Build an incremental searchable catalog and a browser model whose selection survives filtering and rescanning.

10. Binary Inspector, Chunk Tree, and Field Provenance
Build outcome: Implement a raw/transformed binary inspector, validated chunk tree, and source-range navigation shared by every later decoder.

11. Transforms, Coordinate Spaces, and the Camera
Build outcome: Create bounds-checked endian-aware readers, typed numeric decoders, vectors, matrices, transforms, projection, and an inspectable camera.

12. Vertex Data, Declarations, and Index Buffers
Build outcome: Parse validated geometry records into typed headers and flag sets, then create vertex-layout, vertex-buffer, and index-buffer wrappers.

13. Textures, Surfaces, Palettes, and Mipmaps
Build outcome: Decode validated texture records into surfaces, palettes, and mip levels, then upload and cache them with preserved source metadata.

14. The Texture Workspace
Build outcome: Build an interactive texture workspace with lazy mip decoding, channel inspection, provenance display, and loss-safe GPU upload.

15. The Model Workspace
Build outcome: Build a model workspace with orbit navigation, draw-group selection, diagnostic overlays, material inspection, and source navigation.

16. Render States: Depth, Culling, Fog, and Fill
Build outcome: Represent render state in explicit descriptors populated from evidence-scoped flag decoders that never discard unknown bits.

17. Alpha Test, Alpha Blending, and Draw Order
Build outcome: Decode alpha-related fields and build opaque, cutout, and blended queues with deliberate depth and ordering rules.

18. Texture Stages and Fixed-Function Combiners
Build outcome: Parse material-stage records into a loggable fixed-function texture-stage and combiner description system.

19. Vertex Shaders and Programmable Techniques
Build outcome: Load validated shader records, declarations, and constants with capability checks and an explicit fallback path.

Part II — Binary Resources and Rendering Foundations

20. The World Scene: Zones, Chunks, and Draw Groups
Build outcome: Parse a zone resource graph and construct the world scene from validated chunks, entities, references, and draw groups.

21. Zone Loading and Scene Assembly
Build outcome: Implement dependency-aware zone loading and publish an immutable scene snapshot suitable for rendering and inspection.

22. Object Explorer, Selection, and Property Editing
Build outcome: Build synchronized tree, viewport, and property inspection with stable IDs, highlighting, reversible overrides, and source navigation.

23. Collision Geometry and Navigation Diagnostics
Build outcome: Implement collision inspection, acceleration, ray queries, optional grounding, and visual discrepancy overlays with explicit confidence labels.

24. Visibility: Frustum Culling, Distance, and Occlusion
Build outcome: Decode supported visibility fields and implement frustum, distance, and optional occlusion tests while preserving unknown flags.

25. Terrain, Ground, and Large-Scale Environment Geometry
Build outcome: Parse terrain sectors into reusable ground geometry, materials, bounds, and large-environment submissions.

26. Static Objects, Props, and Architectural Detail
Build outcome: Parse model and placement records, then instantiate static props and architecture with shared geometry and per-instance state.

27. Characters: Assembly, Skinning, and Equipment
Build outcome: Decode validated character components and assemble equipment selections, skeleton bindings, and skinned geometry.

28. Character Assembly Workspace
Build outcome: Build a character assembly workspace with slot resolution, compatibility checks, asynchronous replacement, provenance, and presets.

29. Animation, Pose Evaluation, and Motion Data
Build outcome: Maintain explicit SkeletonTime, SchedulerTime, GeneratorTime, and ParticleAge values. Do not reuse one frame counter for all four systems.

30. Animation Browser and Playback Controller
Build outcome: Build an animation catalog and transport controller that drives the chapter’s skeletal evaluator and exposes every fallback.

31. Lighting Models, Vertex Color, and Material Response
Build outcome: Translate decoded material, light, vertex-color, and texture-stage inputs into an inspectable shading configuration.

Part III — Scene Construction and Interactive Workspaces

32. Sky, Celestial Objects, and Time of Day
Build outcome: Keep environment interpolation, sky-dome drawing, and generator execution as separate modules joined by decoded resource identities.

33. Fog, Distance Haze, and Color Grading
Build outcome: Parse supported fog parameters and expose distance haze and scene-color controls for runtime inspection.

34. Water, Reflections, and Surface Motion
Build outcome: Complete the chapter's integrated application checkpoint.

35. Weather: Rain, Snow, Wind, and Ambient Particles
Build outcome: Implement WeatherState, EnvironmentState, and PrecipitationRuntime as separate objects. Connect them through weather tags and decoded generator references, not shared global guesses.

36. Environment Control and Comparison Workspace
Build outcome: Build an environment workspace that edits, compares, captures, and reports decoded state separately from reconstruction policy.

37. Particle Systems, Trails, and Spell Effects
Build outcome: Finish this chapter with a viewer that can pause at any simulation frame and display generator source offsets, decoded commands, unknown payloads, live particles, curve samples, child events, and the exact render descriptor for each draw.

38. Effect Browser and Simulation Workspace
Build outcome: Build a deterministic effect workspace with command, simulation, rendering, and evidence views driven by one timeline.

39. Transparency, Sorting, and Multi-Pass Effects
Build outcome: Translate supported pass records into transparent queues, target changes, sorting policies, and multi-pass composition.

40. Shadows, Decals, and Projected Visuals
Build outcome: Decode supported projected-visual records behind interchangeable shadow, decal, and projector interfaces.

41. Interface Rendering and 2D Composition
Build outcome: Parse supported interface records and build an orthographic renderer that composes evidence-scoped 2D layers.

42. Text, Icons, Fonts, and Window Layers
Build outcome: Decode supported font, glyph, icon, and window records, then implement text layout, atlases, clipping, and layering.

43. Text, Dialog, and Message Resource Explorer
Build outcome: Build a layout-pluggable text explorer with token-aware decoding, search, provenance, and lossless diagnostic export.

44. Audio Catalog, Decode, and Playback
Build outcome: Build a searchable audio workspace with safe header parsing, asynchronous decode, playback transport, looping, and unsupported-codec reporting.

Part IV — Atmosphere, Effects, and Content Tools

45. Resolution, Aspect Ratio, and Display Scaling
Build outcome: Separate decoded presentation settings, internal rendering dimensions, output resolution, viewport, aspect ratio, and UI scaling.

46. Desktop Shell, Docking, and Workspace Lifecycle
Build outcome: Build the multi-document shell and workspace factory/lifecycle contract used by every explorer and viewer.

47. Commands, Menus, Settings, and Tool Windows
Build outcome: Implement typed commands and versioned settings that drive menus, shortcuts, dialogs, and tool windows through one policy.

48. Input, Cameras, Picking, and Navigation Modes
Build outcome: Build configurable input, four navigation modes, reliable pointer capture, and selection rays consistent with the rendered viewport.

49. State Changes, Batching, and Resource Reuse
Build outcome: Add state caching, draw sorting, batching, and resource reuse without erasing source identity or required order.

50. CPU–GPU Work Distribution
Build outcome: Measure the full asset-to-frame path with parse/decode/upload timings and CPU-GPU frame telemetry.

51. Background Jobs, Cancellation, and UI Handoffs
Build outcome: Build the reusable job system used by cataloguing, decode, streaming, audio, screenshots, and export.

52. Memory Budgets, Streaming, and Loading Behavior
Build outcome: Build dependency-aware streaming, caches, eviction, and recoverable handling for missing, corrupt, or unsupported records.

53. Resource Caches and Explicit Lifetime
Build outcome: Implement layered CPU/GPU caches with stable handles, coalesced loads, budgets, diagnostics, and device-loss recovery.

54. Performance Tradeoffs and Legacy Hardware Design
Build outcome: Enforce configurable budgets and benchmark file decoding, resource preparation, rendering, and presentation independently.

55. Screenshots, Reports, and Data Export
Build outcome: Build transactional screenshot, table, texture, geometry, and report exports with manifests and explicit conversion policy.

56. Sessions, Presets, and Reproducible Workspaces
Build outcome: Implement versioned sessions and focused presets that reproduce an inspection without serializing runtime-only state.

Part V — Desktop Application, Presentation, and Performance

57. Reading an Opaque Environment Draw
Build outcome: Build an opaque environment case study that verifies source bytes, parsed records, resources, state, depth behavior, and draw order.

58. Reading a Character Draw
Build outcome: Build a character case study exposing each decoded component, binding, animation, material, equipment, and draw decision.

59. Reading a Transparent Effect
Build outcome: Export the effect-frame record as machine-readable JSON and a human-readable report so parser changes can be diffed across revisions.

60. Reconstructing a Frame from Evidence
Build outcome: Create a frame-inspection mode that links file evidence to parsed records, resources, state transitions, and intermediate targets.

61. Confirmed Architecture, Open Questions, and Competing Models
Build outcome: Maintain architecture, header, and flag registries with provenance, confidence, contradictions, and reproducible next tests.

62. Lessons from Vana’diel’s Visual Technology
Build outcome: Integrate the verified DAT-reading and rendering systems into a testable visual client with regression scenes and documented extension points.

63. Testing, Fuzzing, and Regression Hardening
Build outcome: Build a layered test runner and fuzz entry point that exercise every decoder and application boundary without requiring interactive operation.

64. Packaging, Documentation, and Safe Extension
Build outcome: Produce a toolchain-agnostic release layout, manifest, first-run flow, diagnostic bundle, extension registry, and final acceptance checklist.

Part VI — Case Studies, Verification, and Delivery

Appendices A–J: API, render states, combiners, mathematics, evidence register, glossary, chronology, sources, and index.

Part I — Context, Method, and Application Plan

Appendices A–J: API, render states, combiners, mathematics, evidence register, glossary, chronology, sources, and index.

Part I — Context, Method, and Application Plan

The purpose of this part is not to claim a complete implementation map. It is to give the reader a reliable lens for interpreting later evidence.

Part I — Context, Method, and Application Plan

1. Final Fantasy XI in Its Rendering Era

Final Fantasy XI belongs to a transitional period in real-time graphics. In that period, a renderer commonly combined a fixed-function pipeline with carefully chosen programmable techniques, while still working within tight budgets for memory, fill rate, CPU time, and state changes. The result was not simply a lesser version of a later renderer. It was a different design problem: produce a coherent, persistent world with the resources and interfaces available at the time.

> Scope of this chapter. The discussion below describes the broader rendering era. It does not, by itself, establish that Final Fantasy XI used every technique mentioned.

> **Implementation checkpoint.** Create the project skeleton, source-tree conventions, asset-to-pixel architecture, and clean-room implementation boundary.

```
// Cumulative implementation excerpt: one clean asset-to-pixel boundary.
struct ClientApp {
    AssetRepository assets;
    Renderer renderer;
    EvidenceLog evidence;

    int run() {
        while (renderer.pumpWindow()) {
            Scene scene = assets.buildScene(evidence);
            renderer.draw(scene, evidence);
        }
        return 0;
    }
};

int main() { return ClientApp{}.run(); }
```

1.1 A world designed for real-time continuity

An online role-playing world asks its renderer to do several things at once. It must establish a readable landscape, animate characters and equipment, retain the atmosphere of changing weather and time, and present an interface that remains legible during motion and combat. These goals compete for the same frame budget. A distant horizon may need fog; a spell effect may need transparency; a character may need several visible materials; and all of them must be composed in a stable order.

The visual character of a scene is therefore shaped as much by prioritization as by individual assets. Geometry can be economical while color, fog, animation, texture work, and layering create a sense of scale. A technical analysis should ask which parts of the image appear to carry that work, then seek evidence for the mechanism rather than assuming one.

1.2 Fixed-function thinking and programmable opportunity

The contemporary graphics vocabulary includes transforms, vertex data, textures, render states, depth rules, blending, and texture combiners. A fixed-function path can combine these ingredients in a prescribed pipeline: transform vertices, apply lighting, sample textures, combine colors, test depth, and write a result. Programmable vertex processing expanded the range of possible vertex work, but it did not eliminate the usefulness of the fixed-function path.

For this reason, a visual feature should not be classified by appearance alone. A color gradient might arise from vertex color, a texture stage, fog, or a combination of them. A moving surface might use transformed texture coordinates, an animated texture, geometry motion, or multiple passes. The later API chapters introduce ways to distinguish these alternatives.

1.3 Constraints that shape the image

Several constraints were especially influential in this era. Texture memory encouraged compact reuse and careful resolution choices. Limited fill rate made large blended regions and repeated full-screen work expensive. CPU submission costs rewarded batches of similar draws. Depth and draw ordering mattered because opaque surfaces, cutout textures, and translucent effects cannot all be treated identically.

These are design pressures, not proof of a particular implementation. They help explain why an engine may separate opaque and translucent work, reuse material state, simplify distant scenery, or favor texture-based detail over additional geometry. Each such conclusion must be marked as confirmed only when the game-specific evidence supports it.

1.4 Why the platform context matters

A platform is more than a list of specifications. It defines the graphics API surface, display behavior, memory organization, and debugging assumptions available to the developers. When comparing scenes or captures, it is important to record the platform and build under examination. A behavior visible in one environment may reflect configuration, compatibility, or a platform-specific path rather than the renderer’s universal design.

This book will therefore identify the observation context whenever it makes a technical claim: the platform, the scene, the camera conditions, the evidence type, and the alternative explanations considered. That practice makes later corrections possible without turning uncertainty into false certainty.

1.5 Questions that guide the investigation

The first questions are intentionally concrete. Which features are visible at different distances? Which changes occur when weather, time, or camera angle changes? Which draws are opaque, cut out, or blended? Which textures and state groups recur? Which results depend on a single pass, and which require composition? The answers form a bridge from historical context to reproducible technical analysis.

> Evidence standard. A familiar technique is a hypothesis, not a finding. The evidence register should state what was observed, what competing explanations remain, and what observation would resolve them.

1.6 The implementation contract

The implementation track uses C++20 because later chapters depend on std::span, std::endian, and other facilities standardized at that level. This is a language requirement, not a choice of compiler, editor, IDE, or build system. The rendering checkpoint targets Windows because the graphics API examined by this book is exposed through Windows types and libraries.

Code appears in two forms. A complete checkpoint listing contains its required declarations, entry point, and error path and can be compiled as a small executable. A cumulative implementation excerpt shows the function being added to the client at that chapter; it depends only on interfaces introduced by earlier checkpoints or named in the module contract. An excerpt is not presented as a standalone program.

For any conforming toolchain, compile the named C++ translation units in C++20 mode and link them into one executable with the platform import library that supplies the Direct3D 8 symbols. The exact command syntax is intentionally outside this book. A successful toolchain choice must provide the standard C++ library, Windows declarations, the Direct3D 8 declarations, and the corresponding import library.

1.7 The cumulative module contract

The client is divided by responsibility, not by a prescribed directory layout. core owns errors, identifiers, byte ranges, checked arithmetic, vectors, and matrices. evidence owns provenance and confidence records. dat owns bounded file input, chunk walking, transformation dispatch, and typed record views. assets owns decoded CPU resources and caches. scene owns cameras, instances, animation, visibility, and draw descriptions. platform owns the window and event pump. graphics owns the device and GPU resources. client owns scheduling and integration.

A reader may rename files or combine translation units. The required invariant is dependency direction: evidence and DAT parsing must not depend on graphics; decoded assets must retain source identity; graphics must consume validated descriptions rather than reinterpret file bytes; and unsupported records must remain recoverable diagnostic results.

1.8 Checkpoints and the definition of done

Each checkpoint has four gates: it compiles without undeclared textbook-only symbols; malformed input produces a bounded error rather than an out-of-range access; a documented fixture produces the expected structural result; and the evidence log can identify the source byte range behind each supported decoded field. Visual similarity is an additional test, never a substitute for those gates.

The first executable opens a window, creates the graphics device, clears and presents frames, and survives temporary device loss. The second reads one explicitly selected DAT file and walks its retail-confirmed chunk boundaries without interpreting unsupported payloads. Later checkpoints add one verified asset family at a time. This produces a useful custom visual client before it produces broad format coverage.

1.9 Phase gates

Gate A — foundation (Chapters 1–5). Prerequisite: a C++20 implementation of core and evidence responsibilities. Pass condition: the client shell runs, byte comparisons are deterministic, and every experiment records its inputs.

Gate B — bytes to GPU (Chapters 7–19). Prerequisite: Gate A and both complete Chapter 7 checkpoints. Pass condition: one verified record family crosses bounded input, chunk walking, supported transformation, typed parsing, upload, state application, and a visible diagnostic draw; unsupported records remain named failures.

Gate C — scene reconstruction (Chapters 20–35). Prerequisite: Gate B. Pass condition: one small fixture scene resolves references into static geometry, camera visibility, environment state, and at least one character or animated object without reading raw file bytes in the renderer.

Gate D — ordered composition (Chapters 37–45). Prerequisite: Gate C. Pass condition: deterministic effect simulation, transparent ordering, render-target transitions, and 2D composition can be paused and inspected with source offsets and render descriptors.

Gate E — resilient client (Chapters 49–62). Prerequisite: Gates A–D. Pass condition: caches, streaming, budgets, case-study reports, and regression scenes run through one executable; a parser revision can be compared by evidence, structure, and image without erasing the previous result.

2. Platforms, Display Targets, and Technical Constraints

Final Fantasy XI's history includes PlayStation 2, Windows, and Xbox 360 versions. Publisher records also establish an important end point for comparative work: service for the PlayStation 2 and Xbox 360 versions ended on March 31, 2016, while the Windows version continued. This chapter uses that platform history as a factual baseline; it does not assume that every version used an identical renderer, asset path, or display configuration.

> Primary-source note. Platform history in this chapter is based on Square Enix's official We Are Vana'diel history and official transition notices. Detailed version-specific behavior requires its own evidence record.

> **Implementation checkpoint.** Define platform, installation, resource-map, display, and client-configuration abstractions without assuming identical paths.

```
// Cumulative implementation excerpt: platform and installation are data, not globals.
struct ClientConfig {
    std::filesystem::path installRoot;
    std::vector<std::filesystem::path> romVolumes;
    std::uint32_t backBufferWidth = 1280;
    std::uint32_t backBufferHeight = 720;
    bool windowed = true;
};

std::filesystem::path resolveResource(
    const ClientConfig& c, std::size_t volume, std::filesystem::path relative) {
    if (volume >= c.romVolumes.size()) throw std::out_of_range("ROM volume");
    return c.installRoot / c.romVolumes[volume] / relative;
}
```

2.1 One world, several technical contexts

A multi-platform title can share a world, game rules, and broad visual direction while differing in the route by which it reaches the screen. Differences may arise in graphics interfaces, texture formats, resource limits, output timing, display options, input assumptions, or later compatibility work. The presence of the same scene on two platforms therefore demonstrates continuity of content, not identity of implementation.

For a graphics investigation, the practical unit is not simply the game title. It is a specific client, build, platform, configuration, and scene. A capture or observation without those details is difficult to compare, because it may silently mix platform behavior with general engine behavior.

2.2 Display targets are part of the design

A renderer produces an image for a display environment, not for an abstract screenshot. Display size, aspect ratio, safe areas, color conversion, filtering, and scaling all influence what the player can read. An interface panel, a thin outline, or a fog boundary may be chosen with the expected display in mind. When a modern capture looks unusually sharp, soft, wide, or cropped, the first question should be whether the presentation path changed before assuming that the underlying scene changed.

This does not mean that every visual difference is a display artifact. Rather, it establishes a discipline: document the output conditions before treating pixels as proof. The evidence register should include resolution, aspect ratio, window or full-screen mode, any scaling layer, capture method, and the location of the camera.

2.3 Constraints are budgets, not defects

Real-time rendering is governed by several budgets at once. Memory limits how much geometry, texture data, animation data, and temporary working space can remain available. Bandwidth limits how quickly resources move. CPU time limits scene preparation and draw submission. Graphics processing limits geometry work, texture sampling, blending, and pixel output. A design succeeds by deciding where detail has the greatest visual value.

This perspective is more useful than treating old hardware as a single bottleneck. A scene can be light in geometry yet expensive in overdraw; it can use small textures yet incur many state changes; it can fit in memory yet take too long to prepare. The later performance chapters will examine these categories separately rather than using a vague claim that a feature was present merely because it was efficient.

2.4 Implications for world rendering

In a large online world, distance management becomes a visual language. Fog can join a horizon to the sky. Reduced detail can preserve the frame budget for nearby characters. Repeated materials can make a zone coherent while conserving resources. Layered effects can communicate weather or magic without requiring every pixel of the screen to be equally complex. These are era-wide possibilities, not confirmed descriptions of a particular Final Fantasy XI subsystem.

The same caution applies to platform comparisons. If an effect is absent, altered, or less visible in one environment, several explanations remain possible: a configuration difference, an asset variant, a rendering path, a capture limitation, or a scene condition. A useful comparison changes one variable at a time and records the result.

2.5 A capture protocol for later chapters

Each future case study should begin with a compact record: platform and client build; zone and coordinates or landmark; time and weather; camera position and field of view when known; display mode; and the exact visual question being tested. The record should also distinguish direct capture, observed gameplay, static image comparison, API trace, data inspection, and implementation analysis.

This protocol makes negative results useful. If a claimed effect cannot be reproduced under documented conditions, that does not immediately disprove it; it narrows the conditions under which the claim may be true. Reproducibility is the bridge between an attractive explanation and a dependable technical account.

> Working rule. State the platform first, then describe the observation, then evaluate the explanation. Never let a platform-wide generalization substitute for evidence of a specific rendering behavior.

3. How a Real-Time Frame Is Built

A frame is not a single operation. It is a sequence in which the engine decides what the scene contains, prepares the data needed to draw it, submits work to the graphics system, and combines the resulting images into the picture the player sees. The exact sequence differs among engines and can change with platform or scene. This chapter provides a working model for analysis, not a confirmed internal diagram of Final Fantasy XI.

> Working model. The stages in this chapter are analytical categories. A confirmed frame order requires evidence from the client and the observed rendering behavior.

Implementation checkpoint. Implement the application loop, ordered frame scheduler, and an asynchronous asset-request pipeline with traceable stages.

```
// Cumulative implementation excerpt: ordered frame work plus asynchronous loading.
class AssetPipeline {
    std::mutex mutex_;
    std::queue<DecodedAsset> ready_;
public:
    void request(ResourceId id) {
        std::thread([this, id] {
            auto decoded = decodeAsset(readResource(id));
            std::lock_guard lock(mutex_);
            ready_.push(std::move(decoded));
        }).detach();
    }
    void publishReady(Scene& scene) {
        std::lock_guard lock(mutex_);
        while (!ready_.empty()) { scene.install(std::move(ready_.front())); ready_.pop(); }
    }
};

void frame(ClientApp& app) {
    app.assets.publishReady(app.scene);
    app.scene.update();
    app.renderer.beginFrame(); app.renderer.draw(app.scene); app.renderer.present();
}
```

3.1 Begin with a scene state

Every frame begins from a scene state: the player and camera positions, animation time, zone data, visible entities, weather, time-related parameters, and interface state. Some of this data changes every frame; some changes only when a resource is loaded or an event occurs. The engine must turn this mixed state into a smaller set of decisions suitable for drawing.

The camera is especially important because it defines a view of the world rather than the world itself. A building behind the camera, a character beyond the viewing volume, or a distant object hidden by a visibility rule may not require a draw in that frame. Later chapters will distinguish camera transforms from visibility tests and from artistic distance effects such as fog.

3.2 Prepare and select work

Before a draw can occur, the renderer needs usable resources: vertex data, index data, textures, transforms, material parameters, and the state required to interpret them. The engine may reuse work prepared earlier, update a small part of it, or prepare a new set of resources. It may also choose which candidate objects are worth submitting at all.

This selection is often called visibility management, but it is broader than a single culling test. It can include distance thresholds, object categories, zone partitions, character priority, and special rules for effects or interface elements. The visible result alone rarely identifies which rule was used. An analyst should name the observed change first, then test candidate explanations.

3.3 Submit geometry and material state

A draw submission describes what to render and how to render it. At minimum, it connects geometry with a transform, a material, and a set of graphics states. Geometry supplies positions and other per-vertex values. A material supplies texture bindings and combination rules. States govern conditions such as depth testing, culling, fog, alpha testing, and blending.

Efficient renderers try to avoid needless changes between similar draws, but efficiency is not the only goal. A correct image may require a particular order. For example, a surface that writes depth is different from a translucent effect that must be blended with an image already behind it. The relevant question is not simply how many draw calls exist, but which state changes are necessary to make the intended layer appear.

3.4 Opaque work, cutouts, and transparency

A useful conceptual split is between opaque work, cutout work, and blended work. Opaque pixels normally provide reliable depth coverage. Cutout work uses an alpha threshold so that a pixel is either accepted or discarded, which is useful for shapes such as foliage or patterned edges. Blended work combines a source color with the destination already in the frame buffer, creating translucency, glow, or other composited results.

These categories describe rendering behavior, not the names of asset files. An object can require more than one category, and a scene can interleave them for a specialized effect. The book will use captures and state evidence to determine the actual grouping rather than assuming that all transparent-looking content follows the same path.

3.5 Compose the presentation

After world and effect work, the renderer may add interface layers, text, cursors, fades, or other presentation elements. It then presents the completed image to the display path. This final step can involve buffering, synchronization, scaling, or color conversion outside the visible world renderer. A screenshot is therefore an observation of the complete presentation chain, not necessarily of one isolated scene pass.

For later case studies, the frame should be reconstructed as a timeline of evidence: what is present before a draw, what changes after it, which states and resources are active, and what remains visible in the finished image. The aim is not merely to label passes. It is to explain why their order produces the observed pixels.

3.6 A frame map for this book

The chapters that follow use six recurring questions: What scene data exists? What work is selected? What geometry is submitted? Which material and state rules are active? How are layers composed? How is the result presented? This map gives every later claim a place while leaving room for evidence that the game’s actual order differs from the general model.

> Evidence standard. A proposed pass order is speculation until it can be tied to reproducible behavior, direct API activity, implementation analysis, or several independent forms of corroboration.

3.7 Executable checkpoints

Do not attempt to integrate every subsystem before running the client. Checkpoint 0 presents an empty frame. Checkpoint 1 prints validated chunk boundaries. Checkpoint 2 renders diagnostic geometry whose source record and state can be inspected. Checkpoint 3 replaces that geometry with one verified texture or mesh family. Checkpoint 4 constructs a small scene from verified references. Animation, weather, effects, interface composition, streaming, and optimization follow only after the preceding image and evidence records are repeatable.

At every checkpoint, keep the last passing executable and its fixture hashes. When a new parser changes an older result, compare evidence records before changing the reference image. This turns the book into an incremental implementation path rather than a collection of unrelated samples.

4. Research Methods and the Evidence Register

A technical textbook earns trust by making its path to a conclusion visible. For this subject, that means recording not only a claim about the renderer, but also the observation that supports it, the environment in which it was made, the alternatives that remain possible, and the confidence appropriate to the evidence. The evidence register is the book's shared memory for that work.

> Editorial rule. An attractive explanation is not enough. Every game-specific technical claim must be traceable to a record that states its evidence and confidence.

> **Implementation checkpoint.** Build diagnostics and an evidence-register logger that links every parsed field and reconstructed behavior to its source bytes.

```
// Cumulative implementation excerpt: every decoded field retains its byte provenance.
struct EvidenceEntry {
    ResourceId resource;
    std::uint64_t offset;
    std::uint32_t size;
    std::string field;
    std::string interpretation;
    Confidence confidence;
};

class EvidenceLog {
    std::vector<EvidenceEntry> entries_;
public:
    template<class T>
    void field(ResourceId r, std::uint64_t off, std::string name,
               const T& value, Confidence c) {
        entries_.push_back({r, off, sizeof(T), std::move(name),
                            toDiagnosticString(value), c});
    }
    std::span<const EvidenceEntry> entries() const { return entries_; }
};
```

4.1 The three confidence labels

Confirmed means that a claim is directly supported by reproducible observation, primary documentation, implementation analysis, or another source that unambiguously establishes the point. Strong inference means that several observations and established graphics practice support one explanation better than its alternatives, while a decisive trace is still absent. Speculation means that the explanation is plausible or useful to test, but remains unproven.

These labels describe the claim, not the author's enthusiasm. A useful speculative model may guide an experiment. A confirmed detail may be narrow and unglamorous. The point is to let the reader see exactly how far the evidence reaches.

4.2 Evidence types

The strongest analysis usually combines several evidence types. Direct visual observation shows what reaches the screen. Controlled comparison shows what changes when one documented condition changes. API activity can show resources, state changes, and draw submission. Data inspection can reveal structure and relationships. Implementation analysis can connect observed behavior to code paths. Period documentation can establish the capabilities and terminology of the relevant platform or API.

No one type is universally sufficient. A capture can show a result but not always its mechanism. A code path can exist without proving that it runs in the scene under study. A file structure can suggest intent without proving a rendering state. The register should record both the strength and the limits of each source.

4.3 The minimum claim record

Each record should contain: a short claim; a stable identifier; the platform, client build, scene, and capture conditions; the evidence type; a compact description of the observation; the alternative explanations considered; the confidence label; and the next action that could raise or lower confidence. Source files, captures, or traces should be named precisely enough that another researcher can locate them.

For example, a record should not say only that a surface is 'water.' It should say what was observed: whether the surface moves, whether its appearance changes with the camera, whether it writes depth, whether it blends, and which conditions were held constant. The conclusion may remain uncertain, but the observation becomes reusable.

4.4 Controlled comparisons

A controlled comparison changes one relevant variable while keeping the others as stable as practical. Useful variables include distance, camera angle, time, weather, model state, display mode, and platform. The record should say which variable changed and which did not. When several variables move together, the result may still be interesting, but its explanatory power is lower.

Negative evidence belongs in the register as well. If a proposed condition produces no visible change, that result may rule out a simple explanation or reveal that an unrecorded variable matters. A failed reproduction is often more valuable than an undocumented success.

4.5 Separating observation from interpretation

Observation uses language close to the evidence: 'the edge disappears beyond a threshold,' 'the color changes when the camera rotates,' or 'two draws use different blend states.' Interpretation proposes a cause: distance culling, view-dependent texture coordinates, or separate material passes. Keeping these sentences separate makes later correction straightforward.

This distinction is especially important for familiar effects. A shimmering surface may resemble a known technique, but resemblance does not identify its implementation. The evidence register should preserve the competing models until an experiment or trace can distinguish them.

4.6 Publication standard

The final prose of a chapter should state only the confidence that the register supports. Confirmed findings can be written directly. Strong inferences should name the basis for the inference. Speculation should be visibly labeled and paired with a testable question. When evidence conflicts, the disagreement is part of the finding rather than a defect to conceal.

> Register template. Claim; evidence; context; alternatives; confidence; next test. If any field is missing, the statement is not ready to become a technical conclusion.

5. Reading Visual Evidence Without Overclaiming

The finished image is indispensable evidence, but it is not a transparent view of the renderer. A screenshot contains the results of geometry, textures, state rules, camera conditions, display processing, and timing. It can reveal relationships worth testing; it cannot normally identify a single implementation by appearance alone.

> Reading rule. Describe the visible change before naming a rendering technique.

> **Implementation checkpoint.** Add byte-diff, round-trip, capture-comparison, and experiment-control tools to the client test harness.

```
// Cumulative implementation excerpt: byte diff and round-trip verification.
struct ByteDifference { std::size_t offset; std::byte expected, actual; };

std::vector<ByteDifference> diffBytes(
    std::span<const std::byte> a, std::span<const std::byte> b) {
    std::vector<ByteDifference> out;
    const auto n = std::min(a.size(), b.size());
    for (std::size_t i = 0; i < n; ++i)
        if (a[i] != b[i]) out.push_back({i, a[i], b[i]});
    for (std::size_t i = n; i < std::max(a.size(), b.size()); ++i)
        out.push_back({i, i < a.size() ? a[i] : std::byte{},
                          i < b.size() ? b[i] : std::byte{}});
    return out;
}

void requireRoundTrip(const DecodedAsset& asset) {
    const auto rebuilt = encodeAsset(asset);
    const auto differences = diffBytes(asset.sourceBytes, rebuilt);
    if (!differences.empty()) throw ValidationError("round-trip mismatch");
}
```

5.1 Start with what is observable

Useful observations are concrete and comparative: an edge becomes softer with distance; a surface disappears behind another surface; a color shifts when the camera rotates; particles remain visible through geometry; an effect changes between weather conditions. These statements describe pixels and conditions without presuming the mechanism that produced them.

The next step is to list candidate mechanisms. A distant fade may involve fog, an alpha gradient, a texture change, distance selection, or several of these together. A moving pattern may involve animated texture coordinates, texture replacement, geometry motion, or compositing. The list prevents the first familiar explanation from becoming an unearned conclusion.

5.2 Use comparisons that isolate a variable

A comparison is strongest when it changes one relevant condition at a time. Hold location and display mode stable while changing camera distance; hold camera position stable while changing time or weather; hold the scene stable while comparing two documented client environments. Record the conditions that could not be held constant as possible limitations.

A series matters more than a single image. If an object changes over several measured distances, the pattern may reveal a threshold, a gradual interpolation, or a camera-dependent response. The conclusion should remain proportional: the series can establish the pattern before it establishes its internal cause.

5.3 Watch for presentation artifacts

Capture and display steps can create misleading evidence. Scaling can soften edges. Compression can create bands or noise. Different color-management paths can change apparent brightness. Frame timing can catch a transient effect at a misleading moment. Cropping can hide the context that explains a visual boundary. A record should preserve original captures where possible and identify later edits or conversions.

These cautions do not make visual evidence weak. They make it usable. A repeatable observation under documented conditions is valuable even when the underlying implementation remains unknown.

5.4 Treat absence carefully

The absence of a visible effect is not automatically evidence that a system is absent. The effect may be conditional on distance, direction, time, weather, state, platform, configuration, resource availability, or a timing window. A negative result becomes informative when the tested conditions are explicit and when plausible conditions have been varied systematically.

5.5 From image to technical claim

A responsible technical claim has three layers: observation, alternatives, and conclusion. First state what the images show. Then identify the explanations that remain compatible with them. Finally state the confidence-supported conclusion, including what additional observation would decide between alternatives. This structure lets later chapters become precise without becoming overconfident.

> Publication standard. Screenshots illustrate a claim; they do not replace the evidence record behind it.

6. Application Architecture and the Executable Roadmap

This chapter turns the research program into a product plan. The cumulative result is a desktop resource explorer and visualizer: it discovers an installation, catalogs resources, opens bounded binary views, decodes supported records, renders models and zones, and keeps unsupported data inspectable.

> Build outcome.

Build the application context, ordered startup and shutdown, typed service status, and a shell that remains functional while individual subsystems fail.

6.1 Responsibilities and ownership

The application owns six long-lived services: installation discovery, resource catalog, decoder registry, workspace manager, graphics runtime, and diagnostics store. Views borrow stable identifiers; they do not own source files or GPU objects directly.

6.2 Implementation sequence

Startup loads settings, validates the installation, builds or restores the catalog, creates the device, registers workspace factories, and enters the event loop. Shutdown cancels jobs, closes workspaces, releases GPU resources, saves settings, and only then destroys the windowing layer.

6.3 Failure and recovery

A failed service reports a typed error and leaves earlier services destructible. The shell must remain usable when no installation is configured, a catalog is stale, a decoder rejects a file, or the graphics device is temporarily unavailable.

6.4 Verification gate

The acceptance fixture starts with an empty settings directory, reaches an idle shell, attaches a fixture installation, opens one resource workspace, then exits with no live jobs or resources.

6.5 Cumulative C++ implementation

Implementation checkpoint. Build the application context, ordered startup and shutdown, typed service status, and a shell that remains functional while individual subsystems fail.

```
// Cumulative implementation listing
enum class ServiceState { Missing, Ready, Degraded, Failed };
struct ServiceStatus { ServiceState state; std::string message; };

class ExplorerApplication {
    Settings settings_;
    Diagnostics diagnostics_;
    InstallationService installations_;
    ResourceCatalog catalog_;
    DecoderRegistry decoders_;
    GraphicsRuntime graphics_;
    WorkspaceManager workspaces_;
public:
    int run() {
        settings_.load();
        installations_.configure(settings_, diagnostics_);
        catalog_.open(installations_, diagnostics_);
        graphics_.create(settings_.display(), diagnostics_);
        workspaces_.registerBuiltins(decoders_, graphics_);
        const int result = workspaces_.eventLoop();
        workspaces_.cancelAndCloseAll();
        graphics_.shutdown();
        settings_.save();
        return result;
    }
};
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

Part II — Binary Resources and Rendering Foundations

7. The Device, Resources, and Frame Lifecycle

This chapter connects two ends of the resource lifecycle: trustworthy bytes acquired from storage and graphics resources submitted through a rendering device. The clean-room client must preserve evidence while moving from a DAT byte range through validated transformations and parsing to device-owned textures, buffers, surfaces, and draws.

At the API end, Direct3D 8 presents rendering through a device-oriented interface. An application obtains an IDirect3DDevice8 through IDirect3D8::CreateDevice, then uses that device to create resources, establish state, issue draw calls, and present completed back buffers. This is API reference context. It does not prove that a particular Final Fantasy XI client uses any given call sequence.

> API reference note. Microsoft's Direct3D 8 documentation identifies IDirect3DDevice8 as the interface used for DrawPrimitive-based rendering, resource creation, device state, and presentation.

> **Implementation checkpoint.** Implement safe DAT lookup and binary input, an evidence-scoped decrypt/deobfuscate/decompress dispatcher, graphics-device setup, resource creation, presentation, and recovery.

```
// Cumulative implementation excerpt: bounded input, scoped transforms, device lifecycle.
DecodedAsset loadDat(ResourceId id, GraphicsDevice& graphics, EvidenceLog& log) {
    const auto path = resourceTable().resolve(id);
    const std::vector<std::byte> file = readWholeFileBounded(path, 64_MiB);
    ChunkHeader header = parseChunkHeader(file, log);
    auto payload = checkedSubspan(file, 16, header.size - 16);
    std::vector<std::byte> clear = transformDispatcher().decode(
        header.type, header.version, payload, log);
    DecodedAsset asset = parseValidatedPayload(header, clear, log);
    asset.gpuResource = graphics.createResource(asset);
    return asset;
}

void renderFrame(GraphicsDevice& g, const Scene& scene) {
    if (!g.beginFrame()) { g.recover(); return; }
    g.draw(scene); g.endFrame(); g.present();
}
```

7.1 A trustworthy byte view comes before decryption

Before proposing any transformation, the client needs a byte view that can be trusted. The first program in the data path therefore does not decrypt, decompress, or interpret a DAT resource. It opens a selected file in binary mode, reads a declared range, and reports each byte with its absolute source offset. This deliberately modest tool becomes the reference against which every later parser and transformation is checked.

A trustworthy byte view answers four preliminary questions. Did the program open the intended file? Which exact byte range did it read? How are offsets represented? Does an independent byte-for-byte view show the same values at the same positions? Until these questions have reproducible answers, a failed parser cannot be distinguished from an incorrect file selection, truncated read, display error, or offset mismatch.

> Evidence boundary. A matching hex dump confirms byte acquisition and presentation. It does not, by itself, identify a header, prove encryption, reveal a key, or establish the meaning of a readable tag.

7.2 A bounded C++20 reader

Listing 7.1 is a compact first implementation. It uses an unsigned byte buffer, checks the requested range against the file size, distinguishes a short read from success, and announces the 64 KiB preview limit. The formatter preserves one output position per input byte and uses a conservative ASCII gutter; character-set decoding belongs in a separate view because multibyte text can obscure byte alignment.

Listing 7.1. Bounded raw-byte acquisition and a hexadecimal dump.

```
#include <algorithm>
#include <cctype>
#include <cstdint>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <vector>

using ByteBuffer = std::vector<std::uint8_t>;

ByteBuffer read_range(const std::filesystem::path& path,
                      std::uint64_t offset,
                      std::size_t requested) {
    std::ifstream input(path, std::ios::binary);
    if (!input) {
        throw std::runtime_error("cannot open input file");
    }

    input.seekg(0, std::ios::end);
    const auto end = input.tellg();
    if (end < 0) {
        throw std::runtime_error("cannot determine file size");
    }

    const auto file_size = static_cast<std::uint64_t>(end);
    if (offset > file_size) {
        throw std::runtime_error("offset lies beyond end of file");
    }

    const auto available = file_size - offset;
    const auto count = static_cast<std::size_t>(
        std::min<std::uint64_t>(available, requested));
    ByteBuffer bytes(count);

    input.seekg(static_cast<std::streamoff>(offset), std::ios::beg);
    input.read(reinterpret_cast<char*>(bytes.data()),
               static_cast<std::streamsize>(bytes.size()));
    if (input.gcount() != static_cast<std::streamsize>(bytes.size())) {
        throw std::runtime_error("short or failed read");
    }
    return bytes;
}

void dump_hex(const ByteBuffer& bytes, std::uint64_t base_offset) {
    constexpr std::size_t width = 16;
    std::cout << "Offset(h)  00 01 02 03 04 05 06 07  "
                 "08 09 0A 0B 0C 0D 0E 0F  ASCII\n";

    for (std::size_t row = 0; row < bytes.size(); row += width) {
        std::cout << std::hex << std::uppercase << std::setfill('0')
                  << std::setw(8) << (base_offset + row) << "  ";

        for (std::size_t column = 0; column < width; ++column) {
            if (row + column < bytes.size()) {
                std::cout << std::setw(2)
                          << static_cast<unsigned>(bytes[row + column])
                          << ' ';
            } else {
```

```
                std::cout << "   ";
            }
            if (column == 7) std::cout << ' ';
        }

        std::cout << ' ';
        for (std::size_t column = 0;
             column < width && row + column < bytes.size(); ++column) {
            const unsigned char value = bytes[row + column];
            std::cout << (std::isprint(value)
                              ? static_cast<char>(value) : '.');
        }
        std::cout << '\n';
    }
}

int main(int argc, char** argv) {
    if (argc != 2) {
        std::cerr << "usage: dat_bytes <file>\n";
        return 2;
    }

    try {
        constexpr std::size_t preview_size = 64 * 1024;
        const auto bytes = read_range(argv[1], 0, preview_size);
        std::cout << "Read " << std::dec << bytes.size()
                  << " bytes (preview limit " << preview_size << ")\n";
        dump_hex(bytes, 0);
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return 1;
    }
}
```

The reader accepts only a file path in this first draft. A later revision should accept an explicit starting offset and byte count, record the complete file size, and calculate a hash for the complete file or selected range. Those additions make two observations comparable even when filenames, installations, or client builds differ.

7.3 Compare bytes, not appearances

Validation should compare values at absolute offsets. A useful check selects several nonadjacent ranges, including the beginning of the file, a region containing many zeroes, a region containing varied values, and the final partial row. The custom reader and an independent reference must agree byte for byte. Copying only the visible text column is insufficient because character decoding can vary while the underlying bytes remain identical.

Offset notation must be explicit. In a decimal display, consecutive rows are 0, 16, 32, and 48. In a hexadecimal display, the same rows are 0x00, 0x10, 0x20, and 0x30. Both are valid, but a heading that says hexadecimal while the rows are decimal creates false disagreements. This book uses the 0x prefix in prose and Offset(h) in dumps whenever offsets are hexadecimal.

7.4 What the first observation establishes

In one examined DAT file, the beginning includes printable byte sequences that appear as syst and coll in an ASCII gutter, followed by zero-filled and strongly repeating regions. The confirmed observation is limited to those byte values and their locations in the examined file. The sequences are candidate tags, not confirmed type names; the zeroes are candidate padding or fields, not proof of either; and repeating patterns may be structured numeric data, packed values, transformed data, or something else.

The coexistence of readable sequences, long zero runs, and repeated values makes uniform encryption of the entire visible range a poor first model. That is an inference, not proof that no encrypted region exists. A DAT container may mix plain headers with transformed payloads, or different resource families may use different

treatments. Analysis must classify bounded regions rather than assign one property to every file carrying the DAT extension.

7.5 Classify before transforming

The word decryption should be reserved for reversing encryption with a defined algorithm and required key material. Before using it, test alternatives: plain structured data, character encoding, compression, byte or bit permutation, XOR-like obfuscation, platform-specific numeric representation, texture tiling, and corruption. These categories can be combined, and the order matters. For example, a payload might be deobfuscated and then decompressed, while its surrounding record header remains directly readable.

A candidate transformation should be represented as an explicit function from an input byte range to an output byte range. The evidence record must name the source file, source offset, input length, algorithm version, parameters or key material, output length, and validation result. The original buffer remains immutable. This makes it possible to reproduce the result, compare competing transformations, and reverse an incorrect interpretation without losing the evidence.

7.6 Validation gates for a proposed decode

A transformation is not accepted merely because its output looks less random. Its result should satisfy independent invariants: record sizes remain within the available range; offsets point to valid boundaries; counts do not overflow their containing records; alignment is consistent across several samples; known fields take plausible values; references resolve where expected; and re-encoding or reapplying a reversible operation reproduces the original bytes exactly.

The strongest early evidence comes from repetition across files. If the same proposed header layout, size rule, flag behavior, and transformation order parse many independently selected records without special cases, confidence rises. If the rule works only for one hand-picked sample, it remains a hypothesis. Unknown fields and unused flag bits must be preserved verbatim rather than cleared to make the parser appear successful.

> Working conclusion. The raw reader is a foundation for decryption research because it supplies reproducible input, absolute offsets, and failure checks. It is not itself a decryptor. The next implementation layer may transform bytes only after the transformation type and its validation contract are stated explicitly.

7.6.1 Retail-confirmed chunk stepping

The retail client reads the 32-bit information word at chunk offset +4, shifts it right by seven bits, masks the result with 0x7FFFF, and multiplies it by sixteen to reach the next chunk. The size field is therefore 19 bits. The remaining bits preserve a 7-bit type, is_shadow, is_extracted, a 3-bit version, and is_virtual. This layout consumes all 32 bits; widening the size to 20 bits would overlap is_shadow.

Listing 7.2 is a complete checkpoint program. It acquires one named file, decodes only the confirmed information word, rejects zero or out-of-range advances, and prints every boundary. It deliberately does not call the stepped range a payload or assume a universal payload-header size.

Listing 7.2. Complete C++20 checkpoint for bounded DAT chunk walking.

```
#include <bit>
#include <cstddef>
#include <cstdint>
#include <cstring>
#include <filesystem>
#include <fstream>
#include <iomanip>
```

```
#include <iostream>
#include <span>
#include <stdexcept>
#include <vector>

using Bytes = std::vector<std::byte>;

std::uint32_t read_u32_le(std::span<const std::byte> bytes,
                          std::size_t offset) {
    if (offset > bytes.size() || bytes.size() - offset < 4)
        throw std::runtime_error("truncated 32-bit value");
    std::uint32_t value{};
    std::memcpy(&value, bytes.data() + offset, sizeof value);
    if constexpr (std::endian::native == std::endian::big) {
        value = ((value & 0x000000ffu) << 24) |
                ((value & 0x0000ff00u) << 8)  |
                ((value & 0x00ff0000u) >> 8)  |
                ((value & 0xff000000u) >> 24);
    }
    return value;
}

struct ChunkInfo {
    std::uint8_t type;
    std::uint32_t next_blocks;
    bool is_shadow;
    bool is_extracted;
    std::uint8_t version;
    bool is_virtual;
};

ChunkInfo decode_info(std::uint32_t word) {
    return {
        static_cast<std::uint8_t>(word & 0x7fu),
        (word >> 7) & 0x7ffffu,
        ((word >> 26) & 1u) != 0,
        ((word >> 27) & 1u) != 0,
        static_cast<std::uint8_t>((word >> 28) & 0x7u),
        ((word >> 31) & 1u) != 0
    };
}

Bytes read_file(const std::filesystem::path& path,
                std::uintmax_t maximum = 256u * 1024u * 1024u) {
    const auto size = std::filesystem::file_size(path);
    if (size > maximum) throw std::runtime_error("file exceeds safety limit");
    Bytes bytes(static_cast<std::size_t>(size));
    std::ifstream input(path, std::ios::binary);
    if (!input) throw std::runtime_error("cannot open input file");
    input.read(reinterpret_cast<char*>(bytes.data()),
               static_cast<std::streamsize>(bytes.size()));
    if (!input) throw std::runtime_error("short read");
    return bytes;
}

void walk_chunks(std::span<const std::byte> bytes) {
    std::size_t offset = 0;
    while (offset < bytes.size()) {
        if (bytes.size() - offset < 8)
            throw std::runtime_error("truncated chunk prefix");

        const std::uint32_t word = read_u32_le(bytes, offset + 4);
        const ChunkInfo info = decode_info(word);
        const std::size_t next = static_cast<std::size_t>(info.next_blocks) << 4;
        if (next == 0) throw std::runtime_error("zero-length chunk");
        if (next > bytes.size() - offset)
            throw std::runtime_error("chunk extends beyond file");

        std::cout << "offset=0x" << std::hex << offset
                  << " type=0x" << unsigned(info.type)
                  << " bytes=0x" << next
                  << " shadow=" << info.is_shadow
```

```
                  << " extracted=" << info.is_extracted
                  << " version=" << std::dec << unsigned(info.version)
                  << " virtual=" << info.is_virtual << '\n';
        offset += next;
    }
}

int main(int argc, char** argv) {
    try {
        if (argc != 2) {
            std::cerr << "usage: dat_chunks <input.dat>\n";
            return 2;
        }
        const Bytes bytes = read_file(argv[1]);
        walk_chunks(bytes);
        return 0;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return 1;
    }
}
```

Checkpoint test. Compare several printed offsets against an independent byte view, including the final chunk. A file passes only when every step is aligned to 16 bytes, remains within the file, and terminates exactly at the selected range boundary. A failure is evidence that the selected resource does not use this walk from offset zero, is truncated, or requires a separately established outer container rule.

7.6.2 A transformation registry that cannot guess

The cumulative client routes transformations through a registry. Initially the registry supports only the identity operation. A decryption, deobfuscation, decompression, or layout transform is registered only after its input range, parameters, output invariants, and round-trip or cross-file validation are documented. Unknown identifiers produce UnsupportedTransform and remain visible in the evidence log.

Listing 7.3. Transformation dispatch with an explicit unsupported path.

```
enum class TransformId : std::uint32_t { Plain = 0 };

class UnsupportedTransform : public std::runtime_error {
public:
    explicit UnsupportedTransform(std::uint32_t id)
        : std::runtime_error("unsupported payload transform " +
                             std::to_string(id)) {}
};

using TransformFunction = std::vector<std::byte> (*)(
    std::span<const std::byte> source,
    std::span<const std::byte> parameters);

class TransformRegistry {
    std::unordered_map<std::uint32_t, TransformFunction> functions_;
public:
    TransformRegistry() {
        functions_.emplace(static_cast<std::uint32_t>(TransformId::Plain),
            [](std::span<const std::byte> source,
               std::span<const std::byte>) {
                return std::vector<std::byte>(source.begin(), source.end());
            });
    }

    void register_verified(std::uint32_t id, TransformFunction function) {
        if (!function || functions_.contains(id))
            throw std::logic_error("invalid transform registration");
        functions_.emplace(id, function);
    }

    std::vector<std::byte> decode(
        std::uint32_t id, std::span<const std::byte> source,
        std::span<const std::byte> parameters = {}) const {
```

```
        const auto found = functions_.find(id);
        if (found == functions_.end()) throw UnsupportedTransform(id);
        return found->second(source, parameters);
    }
};
```

7.7 The device as a rendering context

The device is the point at which the application describes work to the graphics system. It owns or exposes the render targets, depth-stencil surface, swap-chain behavior, capability information, and the active state used by draws. A device is not the scene itself: scene data becomes visible only after the application binds resources and submits geometry through the device.

For investigation, device activity is useful because it supplies an ordered record. Resource creation, state changes, texture bindings, stream selection, and draw calls can be placed on a timeline. That timeline still needs scene context; a call name alone does not identify whether it belongs to terrain, a character, an effect, or the interface.

7.8 Resources and ownership

Direct3D 8 resources include textures, vertex buffers, index buffers, surfaces, render targets, and depth-stencil surfaces. The device supplies creation methods such as CreateTexture, CreateVertexBuffer, CreateIndexBuffer, CreateRenderTarget, and CreateDepthStencilSurface. Resource descriptions record properties such as dimensions, format, usage, and memory pool.

A resource is data with a graphics role, not necessarily a single asset. One texture can be reused by many materials; one vertex buffer can contain more than one draw range; a surface can serve as an intermediate destination rather than a visible image. When reading a trace, record the resource identifier and its binding history before assigning a visual name to it.

7.9 A conventional Direct3D 8 frame

A conventional API-level frame often prepares or updates resources, clears targets when needed, calls BeginScene, establishes state and bindings, issues DrawPrimitive or DrawIndexedPrimitive calls, calls EndScene, and finally calls Present. BeginScene and EndScene define a scene boundary in the API, while Present transfers the next device-owned back buffer through the presentation path.

This sequence is a useful reference model, not a statement about every internal pass. An engine can perform resource work before the scene boundary, use several targets, reuse state, or defer work in ways that are not apparent from one visible image. Only an observed trace can establish the actual order for a client and build.

7.9.1 Complete graphics checkpoint

Listing 7.4 is the first complete rendering executable. It creates a native window, obtains the Direct3D 8 interface and device, clears the back buffer, brackets the frame with BeginScene and EndScene, presents it, and handles the cooperative-level reset path. It contains no asset assumptions.

Compile this listing as C++20 and link it with the platform import library that implements the Direct3D 8 entry points. The toolchain-specific spelling of those operations is intentionally not prescribed. Success is a responsive 960 by 540 window with a dark blue client area; closing the window must release the device and API interfaces.

Listing 7.4. Complete window, device, frame, presentation, and recovery checkpoint.

```
#define WIN32_LEAN_AND_MEAN
#include <windows.h>
```

```
#include <d3d8.h>
#include <stdexcept>

LRESULT CALLBACK window_proc(HWND window, UINT message,
                             WPARAM wparam, LPARAM lparam) {
    if (message == WM_DESTROY) {
        PostQuitMessage(0);
        return 0;
    }
    return DefWindowProcW(window, message, wparam, lparam);
}

struct GraphicsCheckpoint {
    HWND window{};
    IDirect3D8* api{};
    IDirect3DDevice8* device{};
    D3DPRESENT_PARAMETERS presentation{};

    ~GraphicsCheckpoint() {
        if (device) device->Release();
        if (api) api->Release();
        if (window) DestroyWindow(window);
    }

    void create(HINSTANCE instance) {
        WNDCLASSW wc{};
        wc.hInstance = instance;
        wc.lpfnWndProc = window_proc;
        wc.lpszClassName = L"RenderingVanadielCheckpoint";
        wc.hCursor = LoadCursorW(nullptr, IDC_ARROW);
        if (!RegisterClassW(&wc) && GetLastError() != ERROR_CLASS_ALREADY_EXISTS)
            throw std::runtime_error("RegisterClassW failed");

        window = CreateWindowExW(0, wc.lpszClassName,
            L"Rendering Vana'diel - graphics checkpoint",
            WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT,
            960, 540, nullptr, nullptr, instance, nullptr);
        if (!window) throw std::runtime_error("CreateWindowExW failed");

        api = Direct3DCreate8(D3D_SDK_VERSION);
        if (!api) throw std::runtime_error("Direct3DCreate8 failed");

        presentation.Windowed = TRUE;
        presentation.SwapEffect = D3DSWAPEFFECT_DISCARD;
        presentation.BackBufferFormat = D3DFMT_UNKNOWN;
        presentation.EnableAutoDepthStencil = TRUE;
        presentation.AutoDepthStencilFormat = D3DFMT_D16;
        presentation.hDeviceWindow = window;

        HRESULT result = api->CreateDevice(D3DADAPTER_DEFAULT,
            D3DDEVTYPE_HAL, window, D3DCREATE_HARDWARE_VERTEXPROCESSING,
            &presentation, &device);
        if (FAILED(result)) {
            result = api->CreateDevice(D3DADAPTER_DEFAULT, D3DDEVTYPE_HAL,
                window, D3DCREATE_SOFTWARE_VERTEXPROCESSING,
                &presentation, &device);
        }
        if (FAILED(result)) throw std::runtime_error("CreateDevice failed");
        ShowWindow(window, SW_SHOW);
    }

    bool recover_if_needed() {
        const HRESULT state = device->TestCooperativeLevel();
        if (state == D3DERR_DEVICELOST) return false;
        if (state == D3DERR_DEVICENOTRESET)
            return SUCCEEDED(device->Reset(&presentation));
        return SUCCEEDED(state);
    }

    void render() {
        if (!recover_if_needed()) return;
        device->Clear(0, nullptr, D3DCLEAR_TARGET | D3DCLEAR_ZBUFFER,
```

```
                    D3DCOLOR_XRGB(24, 40, 64), 1.0f, 0);
        if (SUCCEEDED(device->BeginScene())) device->EndScene();
        device->Present(nullptr, nullptr, nullptr, nullptr);
    }
};

int main() {
    try {
        GraphicsCheckpoint app;
        app.create(GetModuleHandleW(nullptr));
        MSG message{};
        while (message.message != WM_QUIT) {
            if (PeekMessageW(&message, nullptr, 0, 0, PM_REMOVE)) {
                TranslateMessage(&message);
                DispatchMessageW(&message);
            } else {
                app.render();
            }
        }
        return static_cast<int>(message.wParam);
    } catch (...) {
        return 1;
    }
}
```

7.10 Lifetime, updates, and reuse

Resources have a lifetime that can span many frames. Static geometry or a frequently reused texture may be created once and bound repeatedly. Dynamic data may be locked, updated, and unlocked as animation or effects change. The distinction matters because an expensive-looking visual result may depend on data prepared earlier rather than heavy work performed in the current frame.

The evidence register should separate creation, update, binding, and draw use. Doing so prevents a common error: treating a texture's creation as proof that it was sampled in the scene under study, or treating a buffer update as proof that all of its contents were rendered.

7.11 What to record in a device trace

At minimum, record device creation parameters when known; target and depth surfaces; resource descriptors; state-setting calls; texture and stream bindings; draw calls and their counts; scene boundaries; and presentation calls. Link each event to the capture conditions described in Chapter 2. A later chapter can then reconstruct a pass from evidence instead of narrating a plausible but unsupported sequence.

> Game-specific boundary. The Direct3D 8 API defines what can be expressed through the device; it does not reveal which choices Final Fantasy XI made without client-specific evidence.

7.6.3 Resolving a file identifier through VTABLE and FTABLE

A client-facing resource identifier is not a filesystem path. The installation tables first select a ROM volume and then encode the directory and filename as one 16-bit value: directory = packed / 0x80 and file = packed % 0x80. Listing 7.5 performs that lookup across the installed volumes and treats a missing table entry, truncated table, and missing target file as distinct outcomes.

Listing 7.5. Toolchain-agnostic resource-ID resolution.

```
struct ResolvedResource {
    ResourceId id;
    std::uint8_t volume;
    std::uint16_t packedLocation;
    std::filesystem::path path;
};
```

```
std::optional<ResolvedResource> resolveResourceId(
    const std::filesystem::path& root, ResourceId id) {
    for (std::uint8_t volume = 1; volume < 20; ++volume) {
        const std::string suffix = volume == 1 ? "" : std::to_string(volume);
        const auto tableRoot = volume == 1 ? root : root / ("ROM" + suffix);
        const auto vtable = tableRoot / ("VTABLE" + suffix + ".DAT");
        const auto ftable = tableRoot / ("FTABLE" + suffix + ".DAT");
        if (!exists(vtable) || !exists(ftable)) continue;

        const auto selectedVolume = readU8At(vtable, id.value);
        if (!selectedVolume || *selectedVolume != volume) continue;
        const auto packed = readU16LEAt(ftable, checkedMul(id.value, 2));
        if (!packed) throw ParseError("truncated FTABLE entry");

        const auto directory = *packed / 0x80u;
        const auto file = *packed % 0x80u;
        const auto rom = volume == 1 ? "ROM" : "ROM" + suffix;
        const auto path = root / rom / std::to_string(directory) /
                          (std::to_string(file) + ".DAT");
        if (!exists(path)) throw ParseError("table entry names a missing DAT");
        return ResolvedResource{id, volume, *packed, path};
    }
    return std::nullopt;
}
```

7.6.4 Recognize before decoding

The common walker must not send every type value into a decoder. A handler first recognizes whether the bounded record is supported, structurally invalid, or merely unknown to this client. Unknown records remain recoverable; invalid records stop the affected resource; supported records may be decoded. This distinction is part of the file format contract, not only error handling.

Listing 7.6. A registry with separate recognition and decoding phases.

```
enum class Recognition { Invalid, Unsupported, Supported };

struct ChunkView {
    std::size_t offset;
    ChunkInfo info;
    std::span<const std::byte> raw;
    std::span<const std::byte> data;
};

struct ChunkHandler {
    std::uint8_t type;
    Recognition (*recognize)(const ChunkView&, EvidenceLog&);
    AssetResult (*decode)(const ChunkView&, EvidenceLog&);
};

class ChunkRegistry {
    std::unordered_map<std::uint8_t, ChunkHandler> handlers_;
public:
    void add(ChunkHandler handler) {
        if (!handlers_.emplace(handler.type, handler).second)
            throw std::logic_error("duplicate chunk handler");
    }

    Recognition inspect(const ChunkView& chunk, EvidenceLog& log) const {
        const auto found = handlers_.find(chunk.info.type);
        if (found == handlers_.end()) return Recognition::Unsupported;
        return found->second.recognize(chunk, log);
    }

    AssetResult decode(const ChunkView& chunk, EvidenceLog& log) const {
        const auto found = handlers_.find(chunk.info.type);
        if (found == handlers_.end()) return UnsupportedChunk{chunk.info.type};
        if (found->second.recognize(chunk, log) != Recognition::Supported)
            return FailedChunk{chunk.offset, "invalid supported record"};
        return found->second.decode(chunk, log);
```

```
    }
};
```

7.6.5 One verified transformation and its boundary

The object-map transformation shown below operates on chunk data after the 16-byte outer header. Versions below 0x1B are left unchanged. Later versions use a rolling key to conditionally invert bounded runs and then XOR each 16-byte object identifier. The 256-byte key table is treated as separately versioned evidence data; it must be supplied from a verified retail build and stored with its hash rather than hidden in the algorithm.

> Confidence boundary. The object-map algorithm and its version gate are confirmed for the examined map family. Applying the same algorithm or key material to textures, skeletons, geometry, or animation merely because their records share a container would be speculation.

Listing 7.7. Bounded object-map transformation with explicit key provenance.

```
// The key table is evidence data, not an algorithmic constant invented here.
using KeyTable = std::span<const std::uint8_t, 256>;

void decodeObjectMap(std::span<std::byte> data, KeyTable keyTable) {
    if (data.size() < 8 || u8(data[3]) < 0x1b) return;
    const std::size_t declared = readU24LE(data, 0);
    const std::size_t limit = std::min(declared, data.size());
    std::uint32_t key = keyTable[u8(data[7]) ^ 0xffu];
    std::uint32_t counter = 0;

    for (std::size_t pos = 8; pos < limit;) {
        const std::size_t run = ((key >> 4) & 7u) + 16u;
        if ((key & 1u) && run < limit - pos)
            for (std::size_t i = 0; i < run; ++i) data[pos + i] ^= std::byte{0xff};
        key += ++counter;
        pos = checkedAdd(pos, run);
    }

    constexpr std::size_t header = 32, nodeStride = 92, nameBytes = 16;
    const std::size_t declaredNodes = readU24LE(data, 4);
    const std::size_t availableNodes = data.size() > header
        ? (data.size() - header) / nodeStride : 0;
    for (std::size_t n = 0; n < std::min(declaredNodes, availableNodes); ++n)
        for (std::size_t i = 0; i < nameBytes; ++i)
            data[header + n * nodeStride + i] ^= std::byte{0x55};
}
```

8. Installation Discovery and Resource Maps

A resource tool must discover data without assuming one hard-coded directory. This chapter defines explicit installation roots, ROM-volume discovery, version identity, and read-only validation before any catalog or viewer opens a file.

> Build outcome.

Implement safe installation discovery and return a complete validation report rather than a single Boolean result.

8.1 Responsibilities and ownership

An InstallationRecord stores the chosen root, discovered volumes, table paths, identity evidence, and validation issues. Paths are data, never process-global constants.

8.2 Implementation sequence

The user selects or confirms a root. Discovery enumerates only expected volume names, resolves canonical paths, checks that every result remains below the root, opens required tables read-only, and records optional volumes separately.

8.3 Failure and recovery

Missing optional volumes degrade coverage; missing core tables prevent identifier lookup but must not crash the shell. Symlinks, traversal components, duplicate volumes, and unreadable files are reported explicitly.

8.4 Verification gate

Tests cover a minimal installation, sparse optional volumes, duplicate case variants, a path escaping the root, and a table whose length is incompatible with its record width.

8.5 Cumulative C++ implementation

> Implementation checkpoint. Implement safe installation discovery and return a complete validation report rather than a single Boolean result.

```
// Cumulative implementation listing
InstallationRecord discoverInstallation(const std::filesystem::path& root) {
    InstallationRecord out{canonicalChecked(root), {}, {}, {}};
    for (unsigned i = 0; i <= 19; ++i) {
        const std::string name = i == 0 ? "ROM" : "ROM" + std::to_string(i);
        const auto candidate = out.root / name;
        if (!std::filesystem::exists(candidate)) continue;
        const auto volume = canonicalChecked(candidate);
        if (!isDescendant(out.root, volume)) {
            out.issues.push_back({Severity::Error, candidate, "volume escapes root"});
            continue;
        }
        out.volumes.push_back({i, volume});
    }
    out.vtable = validateTable(out.root / "VTABLE.DAT", 2, out.issues);
    out.ftable = validateTable(out.root / "FTABLE.DAT", 2, out.issues);
    return out;
}
```

> Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

9. The Resource Catalog and Browser

The catalog converts millions of anonymous paths and numeric identifiers into a searchable, reproducible inventory. It does not pretend that every resource type is known; classification results carry confidence and decoder support separately.

> Build outcome.

Build an incremental searchable catalog and a browser model whose selection survives filtering and rescanning.

9.1 Responsibilities and ownership

Each CatalogEntry stores resource ID, resolved volume and path, byte length, hash, recognition candidates, support state, and scan diagnostics. UI rows reference entry IDs so rescanning cannot invalidate selection through raw pointers.

9.2 Implementation sequence

A scan resolves identifiers, reads only the bounded prefix needed by recognizers, and schedules deeper metadata extraction only for visible or explicitly requested entries. Filters combine identifier range, volume, size, recognition, support, and text search.

9.3 Failure and recovery

Unreadable files remain catalog entries with errors. A cancelled scan commits complete batches and marks the catalog partial; it never publishes half-initialized entries.

9.4 Verification gate

A deterministic fixture catalog must produce the same sorted identifiers, hashes, recognizer candidates, and diagnostic counts regardless of worker count.

9.5 Cumulative C++ implementation

Implementation checkpoint. Build an incremental searchable catalog and a browser model whose selection survives filtering and rescanning.

```
// Cumulative implementation listing
void ResourceCatalog::scan(const InstallationRecord& install,
                           StopToken stop, CatalogSink& sink) {
    std::vector<CatalogEntry> batch;
    for (ResourceId id : enumerateResourceIds(install)) {
        if (stop.stopRequested()) break;
        CatalogEntry e;
        e.id = id;
        e.path = resolveResourceId(install, id);
        try {
            PrefixFile file(e.path, 4096);
            e.byteLength = file.length();
            e.prefixHash = hash(file.prefix());
            e.candidates = recognizers_.classify(file.prefix());
        } catch (const std::exception& ex) {
            e.issues.push_back({Severity::Error, ex.what()});
        }
        batch.push_back(std::move(e));
        if (batch.size() == 256) { sink.commit(batch); batch.clear(); }
    }
    if (!batch.empty()) sink.commit(batch);
    sink.finish(stop.stopRequested() ? ScanState::Partial : ScanState::Complete);
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

10. Binary Inspector, Chunk Tree, and Field Provenance

The binary inspector is the bridge between a hex dump and a typed decoder. It displays raw and transformed views, chunk boundaries, interpreted fields, flags, unknown bits, and the exact source range supporting every value.

> Build outcome.

Implement a raw/transformed binary inspector, validated chunk tree, and source-range navigation shared by every later decoder.

10.1 Responsibilities and ownership

InspectorNode contains a stable path, label, source range, optional transformed range, typed value, confidence, and children. The raw-byte pane and tree selection communicate through ranges rather than UI object addresses.

10.2 Implementation sequence

Opening a resource maps or reads bounded pages, runs recognition, walks only validated chunks, then asks supported decoders to contribute field nodes. Selecting a node highlights its bytes; selecting bytes finds the narrowest containing node.

10.3 Failure and recovery

Invalid sizes produce terminal error nodes and stop the affected branch. Unknown records remain opaque child nodes. A transformation view must never overwrite or disguise the original bytes.

10.4 Verification gate

Tests assert bidirectional range selection, truncation behavior, unknown-bit preservation, and stable node paths after an unrelated sibling is added.

10.5 Cumulative C++ implementation

> Implementation checkpoint. Implement a raw/transformed binary inspector, validated chunk tree, and source-range navigation shared by every later decoder.

```
// Cumulative implementation listing
InspectorNode inspectChunks(ByteView raw, const DecoderRegistry& decoders) {
    InspectorNode root{"/", "resource", {0, raw.size()}, {}, {}, Confidence::Observed};
    std::size_t offset = 0;
    while (offset < raw.size()) {
        const ChunkHeader h = readChunkHeader(raw, offset);
        if (!h.valid || h.byteSize == 0 || h.byteSize > raw.size() - offset) {
            root.children.push_back(errorNode(offset, raw.size() - offset,
                                              "invalid chunk boundary"));
            break;
        }
        InspectorNode node = chunkNode(h, offset);
        if (const Decoder* d = decoders.find(h.type, raw.subview(offset, h.byteSize)))
            d->describe(raw.subview(offset, h.byteSize), node);
        else
            node.children.push_back(opaqueNode(offset + h.headerSize,
                                               h.byteSize - h.headerSize));
        root.children.push_back(std::move(node));
        offset += h.byteSize;
    }
```

```
    return root;
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

11. Transforms, Coordinate Spaces, and the Camera

A renderer does not draw an object in one universal coordinate system. It moves data through named spaces so that a mesh can be authored locally, placed in a world, viewed from a camera, projected onto a screen, and finally mapped to pixels. These spaces provide a precise language for questions that otherwise look like simple position errors.

Scope. This chapter explains the standard transformation vocabulary. It does not establish the exact matrix conventions or camera implementation of Final Fantasy XI.

Implementation checkpoint. Create bounds-checked endian-aware readers, typed numeric decoders, vectors, matrices, transforms, projection, and an inspectable camera.

```
// Cumulative implementation excerpt: bounded little-endian reader and inspectable camera.
class ByteReader {
    std::span<const std::byte> bytes_; std::size_t pos_ = 0;
public:
    explicit ByteReader(std::span<const std::byte> b) : bytes_(b) {}
    template<class T> T little() {
        static_assert(std::is_trivially_copyable_v<T>);
        if (pos_ + sizeof(T) > bytes_.size()) throw ParseError("read past end");
        T v; std::memcpy(&v, bytes_.data() + pos_, sizeof(T)); pos_ += sizeof(T);
        if constexpr (std::endian::native == std::endian::big) v = byteSwap(v);
        return v;
    }
};

struct Camera {
    Vec3 eye, target, up{0, 1, 0}; float fovY, aspect, nearZ, farZ;
    Mat4 view() const { return lookAtLH(eye, target, up); }
    Mat4 projection() const { return perspectiveFovLH(fovY, aspect, nearZ, farZ); }
};
```

11.1 Local, world, view, and projection space

Local space describes geometry relative to its own origin. A world transform places that geometry in the scene. A view transform expresses the scene relative to the camera. A projection transform converts that camera-relative view into a form suitable for perspective division and rasterization. The product is often called a world-view-projection transform, although engines may organize the calculation in several ways.

Keeping these spaces distinct is essential. A rotating character may change a local-to-world transform while the camera remains fixed. A camera pan changes the view transform while object positions remain unchanged. A change in field of view affects projection and can make distant objects appear to move or scale even though their world coordinates do not.

11.2 The camera is a transform, not only a viewpoint

The camera defines orientation and position, but it also determines which scene relationships are visible. Near and far clipping planes limit the depth range considered for drawing. The projection establishes perspective behavior and an aspect relationship. These settings affect apparent scale, screen-space motion, and the point at which objects leave the view volume.

When analyzing captures, record the camera position, direction, distance to the subject, and field-of-view conditions when known. Do not treat a change in apparent size as proof of geometry LOD or texture replacement until camera and projection effects have been excluded.

11.3 Hierarchies and animation

Characters and articulated objects usually require more than one transform. A bone or node inherits the transform of a parent, then contributes its own local transform. Evaluating this hierarchy yields the final pose used to place rigid parts or deform skinned vertices. Equipment can follow the same hierarchy without sharing the same geometry.

Visible motion therefore has several possible sources: object placement, node animation, skeletal deformation, vertex processing, texture-coordinate motion, or camera motion. A credible explanation identifies which space changes and what evidence distinguishes it from the alternatives.

11.4 Texture coordinates and non-geometric motion

Texture coordinates are values associated with vertices that tell the renderer how to sample an image. They can be transformed independently of object position. Scrolling, rotating, or otherwise changing texture coordinates can create motion on a surface while its geometry remains static. This is a common analytical possibility for water, energy, and decorative materials, but it must be proved per effect.

11.5 A transform checklist

For each investigated draw, ask: What is the local origin? What places it in the world? Which camera view applies? Which projection applies? Is a hierarchy involved? Do texture coordinates have their own transform? The answers turn a vague claim that something 'moves' into a testable account of what changes.

> Evidence standard. Matrix names, values, or API calls are direct evidence; an apparent rotation or scroll in a screenshot is an observation that may have several transform-based explanations.

12. Vertex Data, Declarations, and Index Buffers

Geometry reaches the rendering device as structured vertex data, optionally paired with an index buffer. A vertex is more than a position. It can carry normals, diffuse color, texture coordinates, blend information, and other values required by the active rendering path. The declaration or flexible vertex format tells the device how to read that layout.

> Scope. This chapter describes Direct3D-era geometry concepts. It does not identify the layouts used by any Final Fantasy XI model without direct evidence.

> **Implementation checkpoint.** Parse validated geometry records into typed headers and flag sets, then create vertex-layout, vertex-buffer, and index-buffer wrappers.

```
// Cumulative implementation excerpt: validated geometry and owned D3D8 buffers.
Geometry parseGeometry(ByteReader& r, std::size_t recordEnd) {
    Geometry g;
    g.vertexCount = r.little<std::uint32_t>();
    g.indexCount = r.little<std::uint32_t>();
    g.layout = decodeVertexLayout(r.little<std::uint32_t>());
    const auto vertexBytes = checkedMultiply(g.vertexCount, g.layout.stride);
    g.vertices = r.bytesWithin(vertexBytes, recordEnd);
    g.indices = r.arrayWithin<std::uint16_t>(g.indexCount, recordEnd);
    validateIndices(g.indices, g.vertexCount);
```

```
    return g;
}

GpuGeometry upload(IDirect3DDevice8& d, const Geometry& g) {
    return {createVertexBuffer(d, g.vertices, g.layout.fvf),
            createIndexBuffer(d, g.indices), g.vertexCount, g.indexCount};
}
```

12.1 Vertex streams and layouts

A vertex buffer stores a sequence of vertex records. The stride states how many bytes separate one record from the next. A layout assigns meaning to fields at known offsets: position, normal, color, one or more texture-coordinate sets, or data used by a programmable vertex path. In Direct3D 8, flexible vertex formats describe common fixed-function layouts, while vertex shaders provide a more explicit declaration mechanism.

The same mesh position can therefore produce different results under different layouts or states. A normal may enable lighting; a vertex color may tint a material; a second texture-coordinate set may support an additional texture stage. A trace that shows only a buffer address is incomplete until its stride, declaration, and draw range are known.

12.2 Index buffers and reuse

An index buffer refers to vertices by number, allowing triangles to reuse shared vertices instead of duplicating them in the stream. This improves storage efficiency and can express a connected mesh clearly. The index count, primitive type, and base or start values help define exactly which subset of geometry a draw uses.

Indexing does not itself reveal an object boundary. One buffer can contain several objects or material groups, and one object can be drawn through several index ranges. Evidence should therefore associate a draw with its specific ranges rather than treating a buffer as a single visible model.

12.3 Primitive topology

The device interprets vertices and indices as primitives such as points, lines, triangle lists, or triangle strips. Topology affects the order and number of vertices consumed by a draw, but does not by itself determine visual complexity. A short draw can cover a large screen area; a large draw can be mostly hidden or culled.

12.4 Data for animation and materials

Character rendering may require position, normal, texture coordinates, blend weights, blend indices, and other per-vertex attributes. Environment rendering may use a simpler layout or may add vertex color and multiple coordinate sets. These are common possibilities, not a classification of this game’s scene classes. The data layout must be observed or recovered before it is named.

12.5 Reading geometry evidence

A useful geometry record includes buffer size and pool, stride, layout or FVF, index format, primitive type, draw offsets, counts, and the transforms and textures active at the draw. Pair it with a capture that identifies the visible result. This lets an analyst distinguish one material pass over shared geometry from a genuinely separate mesh.

> Evidence standard. Vertex semantics and draw ranges are confirmed by declarations, data inspection, or trace evidence—not by the apparent shape of an object alone.

12.6 A first supported geometry slice

A useful first implementation is deliberately narrower than the known format. Listing 12.1 accepts only unskinned geometry, validates the header's offsets and sizes in 16-bit units, reads positions and optional normals, and handles material-selection and triangle-list commands. Weighted vertices, mirrored passes, strips, draw-state records, and unknown opcodes return an explicit unsupported result instead of being guessed.

Listing 12.1. Evidence-scoped decoding of unskinned triangle-list geometry.

```
struct SupportedGeo {
    SourceRef source;
    std::string material;
    std::vector<VertexPNUV> vertices;
    std::vector<std::uint32_t> indices;
};

SupportedGeo decodeUnskinnedGeo(SourceRef source, std::span<const std::byte> data) {
    if (data.size() < 80) throw ParseError("short geometry header");
    const std::uint16_t flags = u16le(data, 2);
    const auto drawOffset = checkedMul<std::size_t>(u32le(data, 8), 2);
    const auto drawBytes = checkedMul<std::size_t>(u16le(data, 12), 2);
    const auto weightedCounts = u32le(data, 24);
    const auto weightOffset = u32le(data, 32);
    const auto vertexOffset = checkedMul<std::size_t>(u32le(data, 40), 2);
    const auto vertexBytes = checkedMul<std::size_t>(u16le(data, 44), 2);
    if (weightedCounts || weightOffset)
        throw UnsupportedRecord("first geometry slice is unskinned only");
    requireRange(data, drawOffset, drawBytes);
    requireRange(data, vertexOffset, vertexBytes);

    const bool withoutNormals = (flags & 0x007fu) != 0;
    const std::size_t stride = withoutNormals ? 12 : 24;
    if (vertexBytes % stride) throw ParseError("non-integral vertex stream");

    SupportedGeo out{source};
    for (std::size_t p = 0; p < vertexBytes; p += stride) {
        VertexPNUV v{};
        v.position = readVec3(data, vertexOffset + p);
        v.normal = withoutNormals ? Vec3{0, 1, 0}
                                  : readVec3(data, vertexOffset + p + 12);
        out.vertices.push_back(v);
    }

    CommandReader commands(data.subspan(drawOffset, drawBytes));
    while (!commands.empty()) {
        switch (const std::uint16_t op = commands.u16()) {
        case 0x8000: out.material = commands.fixedString(16); break;
        case 0x0054: {
            const auto triangleCount = commands.u16();
            for (std::uint16_t t = 0; t < triangleCount; ++t) {
                const auto corner = commands.triangleRecord(); // 3 indices + 3 UVs
                for (int i = 0; i < 3; ++i) {
                    if (corner.index[i] >= out.vertices.size())
                        throw ParseError("triangle index outside vertex stream");
                    out.vertices[corner.index[i]].uv = corner.uv[i];
                    out.indices.push_back(corner.index[i]);
                }
            }
            break;
        }
        case 0xffff: return out;
        default: throw UnsupportedRecord("geometry draw opcode", op);
        }
    }
    throw ParseError("geometry stream lacks terminator");
}
```

Acceptance test. The maximum referenced index must be smaller than the decoded vertex count; every command must remain inside the declared draw stream; the end opcode must be present; and re-reading the recorded source ranges must reproduce the same positions, normals, indices, UVs, and material name.

13. Textures, Surfaces, Palettes, and Mipmaps

Textures supply sampled image data to a material. Surfaces provide two-dimensional storage that can serve as texture levels, render targets, depth buffers, or other device resources. Their visible role depends on how they are bound and used, not on their file name or dimensions alone.

> Scope. Direct3D 8 supports textures, surfaces, formats, pools, and mip levels. This chapter does not claim which formats or palette paths a particular Final Fantasy XI resource uses.

> **Implementation checkpoint.** Decode validated texture records into surfaces, palettes, and mip levels, then upload and cache them with preserved source metadata.

```
// Cumulative implementation excerpt: decoded mip chain with source-aware caching.
struct TextureSurface { std::uint32_t width, height; PixelFormat format;
                        std::vector<std::byte> pixels; };
struct DecodedTexture { ResourceId source; std::uint64_t sourceOffset;
                        std::vector<TextureSurface> mips; };

DecodedTexture decodeTexture(ResourceId id, ByteReader& r, EvidenceLog& log) {
    TextureHeader h = readTextureHeader(r, log);
    DecodedTexture out{id, h.sourceOffset, {}};
    for (std::uint32_t level = 0; level < h.levelCount; ++level)
        out.mips.push_back(decodeMip(r, h, level, log));
    validateMipDimensions(out.mips);
    return out;
}

IDirect3DTexture8* cachedUpload(TextureCache& cache, GraphicsDevice& g,
                                const DecodedTexture& t) {
    return cache.getOrCreate({t.source, hashMips(t.mips)}, [&] { return g.upload(t); });
}
```

13.1 Texture identity and binding

A texture resource has dimensions, format, level count, usage, and a memory pool. The application binds it to a texture stage, where sampling and combination state determine its contribution. One texture can serve many materials; the same material can bind different textures over time. A trace should therefore record both the resource descriptor and the stage binding at each draw.

The visual result also depends on sampler-related choices such as addressing and filtering. Repeating, clamping, point sampling, and filtered sampling can make the same image appear dramatically different. A visible seam or blur is an observation; its explanation requires the relevant state.

13.2 Mipmaps and distance

A mipmapped texture stores successively smaller versions of an image. During sampling, an appropriate level can reduce aliasing and control texture detail as a surface becomes smaller on screen. Direct3D 8 texture creation can specify a level count, including automatic generation of supported sublevels when requested.

Mipmaps are a common explanation for distance changes, but not the only one. A material can also change through LOD selection, altered texture bindings, fog, geometry changes, or display filtering. Confirming a mip chain requires resource or sampling evidence rather than a distant screenshot alone.

13.3 Formats and palettes

A format determines how texels encode color and alpha. Palettized representations store indices into a palette rather than complete colors per texel, trading representational flexibility for compactness. Palette changes can alter the appearance of indexed data without replacing the texture image itself. Direct3D 8 exposes palette-related device operations, but an API capability is not proof that it is used by a given asset.

13.4 Texture surfaces and render surfaces

A surface can be sampled as part of a texture or used as a rendering destination, depending on its role and device support. Off-screen render targets allow one pass to produce an image consumed by another pass. That capability is relevant when investigating reflections, distortions, or composited interface elements, but it must be demonstrated by target changes and bindings.

13.5 Texture evidence checklist

Record dimensions, format, level count, usage, pool, palette association when present, stage binding, addressing, filtering, and the draw that sampled the resource. Preserve a copy of the resource only as evidence of data; pair it with binding and state records to explain the final pixel result.

> Evidence standard. A texture image proves available data. It does not, by itself, prove the material, stage, sampling rule, or pass in which that data appeared.

13.6 A first supported texture slice

The texture family contains several representations. The first decoder should not pretend to support all of them. Listing 13.1 accepts the observed version-40 header and two paletted type values only when the palette entries are 32-bit BGRA. It validates dimensions, reads 256 palette entries, expands one index per pixel, and reverses the stored row order. DXT and mixed palette/DXT records remain unsupported until their separate prefixes and payload lengths are validated.

Listing 13.1. A bounded 32-bit-palette texture decoder.

```
struct SupportedTexture {
    std::string name;
    std::uint32_t width, height;
    std::vector<Rgba8> pixels;
};

SupportedTexture decodePalette32(std::span<const std::byte> data) {
    constexpr std::size_t headerBytes = 60;
    if (data.size() < headerBytes) throw ParseError("short texture header");
    const std::uint8_t type = u8(data[0]);
    const std::uint32_t version = u32le(data, 20);
    const std::uint32_t width = u32le(data, 24);
    const std::uint32_t height = u32le(data, 28);
    const std::uint32_t paletteBits = u32le(data, 56);
    if ((type != 0x91 && type != 0x01) || version != 40 || paletteBits != 32)
        throw UnsupportedRecord("texture is outside the first supported slice");
    if (!width || !height || width > 4096 || height > 4096)
        throw ParseError("invalid texture dimensions");

    constexpr std::size_t paletteEntries = 256;
    constexpr std::size_t paletteBytes = paletteEntries * 4;
    const std::size_t pixelCount = checkedMul<std::size_t>(width, height);
    requireRange(data, headerBytes, checkedAdd(paletteBytes, pixelCount));
    const auto palette = data.subspan(headerBytes, paletteBytes);
    const auto indices = data.subspan(headerBytes + paletteBytes, pixelCount);
```

```
    SupportedTexture out{fixedString(data.subspan(1, 16)), width, height,
                         std::vector<Rgba8>(pixelCount)};
    for (std::uint32_t y = 0; y < height; ++y) {
        for (std::uint32_t x = 0; x < width; ++x) {
            const auto index = u8(indices[(height - 1 - y) * width + x]);
            const auto p = palette.subspan(index * 4, 4); // stored BGRA
            out.pixels[y * width + x] = {u8(p[2]), u8(p[1]), u8(p[0]), u8(p[3])};
        }
    }
    return out;
}
```

Acceptance test. Record the texture name, type, dimensions, palette width, payload range, and decoded hash. Compare corner pixels and several nonadjacent rows against an independent decode. A plausible image is insufficient if row order or channel order differs.

14. The Texture Workspace

A decoder becomes useful when readers can inspect its result. The texture workspace combines source metadata, palette and mip controls, channel views, zoom, checkerboard alpha, and export while retaining the byte provenance of the selected image.

> Build outcome.

Build an interactive texture workspace with lazy mip decoding, channel inspection, provenance display, and loss-safe GPU upload.

14.1 Responsibilities and ownership

TextureDocument owns decoded CPU surfaces and source records. TextureView owns only display choices. GPU texture handles live in the graphics cache and are recreated after device loss.

14.2 Implementation sequence

Open first produces metadata and thumbnail; full mip decoding occurs on demand. Changing palette, mip, channel, or background invalidates only the presentation layer, not the source document.

14.3 Failure and recovery

Unsupported formats show dimensions and header fields when trustworthy, plus an explicit unavailable-preview panel. Export is disabled until a decoded surface passes size and stride validation.

14.4 Verification gate

Fixtures cover opaque, binary alpha, graduated alpha, multiple mips, palette extremes, odd widths, truncation, and device recreation.

14.5 Cumulative C++ implementation

> Implementation checkpoint. Build an interactive texture workspace with lazy mip decoding, channel inspection, provenance display, and loss-safe GPU upload.

```
// Cumulative implementation listing
void TextureWorkspace::open(ResourceId id) {
    close();
    document_ = textures_.readMetadata(id, diagnostics_);
```

```
    requestMip(0);
}

void TextureWorkspace::requestMip(unsigned level) {
    if (!document_ || level >= document_->mipCount) return;
    selectedMip_ = level;
    jobs_.replace(JobKey{document_->id, level}, [this, level](StopToken stop) {
        DecodedSurface s = textures_.decode(*document_, level, stop);
        ui_.post([this, level, s = std::move(s)]() mutable {
            if (!document_ || level != selectedMip_) return;
            cpuSurface_ = std::move(s);
            gpuTexture_ = graphics_.textures().upload(cpuSurface_);
            invalidate();
        });
    });
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

15. The Model Workspace

The model workspace is the first complete visual inspection tool. It loads supported geometry and materials, frames the camera, exposes draw groups, overlays bounds and normals, and lets the reader connect a selected draw back to its records and bytes.

Build outcome.

Build a model workspace with orbit navigation, draw-group selection, diagnostic overlays, material inspection, and source navigation.

15.1 Responsibilities and ownership

ModelDocument owns decoded meshes, materials, dependencies, bounds, and provenance. ModelViewport stores camera and visualization flags. Selection uses stable mesh and draw-group IDs.

15.2 Implementation sequence

Loading resolves dependencies, decodes CPU data, computes trustworthy bounds, uploads immutable buffers, and publishes the document atomically. The viewport then derives near/far planes and orbit target from those bounds.

15.3 Failure and recovery

A missing texture uses a conspicuous diagnostic material; malformed geometry never reaches buffer creation. Empty or extreme bounds trigger a documented fallback camera rather than undefined matrices.

15.4 Verification gate

Tests verify draw counts, index bounds, dependency errors, camera framing, selection stability, state restoration between overlays, and deterministic screenshots.

15.5 Cumulative C++ implementation

Implementation checkpoint. Build a model workspace with orbit navigation, draw-group selection, diagnostic overlays, material inspection, and source navigation.

```
// Cumulative implementation listing
void ModelWorkspace::render(const ViewportSize size) {
    if (!document_) return;
    RenderFrame frame = renderer_.begin(size, camera_.matrices(size));
    for (const DrawGroup& group : document_->drawGroups) {
        DrawPacket packet = makeDrawPacket(*document_, group);
        packet.highlight = group.id == selection_.drawGroup;
        frame.submit(packet);
    }
    frame.flushOpaque();
    frame.flushTransparentBackToFront();
    if (view_.showBounds) overlays_.drawBounds(frame, document_->bounds);
    if (view_.showNormals) overlays_.drawNormals(frame, *document_);
    frame.present();
}

void ModelWorkspace::select(DrawGroupId id) {
    selection_.drawGroup = id;
    inspector_.navigate(document_->drawGroup(id).source);
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

16. Render States: Depth, Culling, Fog, and Fill

Render states are device-level settings that govern how a draw is interpreted. They can be changed independently of geometry and textures, so two draws using the same mesh can produce very different images. In Direct3D-era analysis, state history is often the missing link between an asset and its visible result.

Scope. This chapter explains state categories and evidence practice. It does not claim which state values Final Fantasy XI uses for any material class.

Implementation checkpoint. Represent render state in explicit descriptors populated from evidence-scoped flag decoders that never discard unknown bits.

```
// Cumulative implementation excerpt: known bits are decoded; unknown bits survive.
struct RenderStateDesc {
    bool depthTest = true, depthWrite = true, fog = false;
    CullMode cull = CullMode::CounterClockwise;
    std::uint32_t rawFlags = 0, unknownFlags = 0;
};

RenderStateDesc decodeRenderFlags(std::uint32_t raw) {
    constexpr std::uint32_t Known = 0x0000000Fu;
    RenderStateDesc s;
    s.rawFlags = raw; s.unknownFlags = raw & ~Known;
    s.depthTest  = (raw & 0x1) != 0;
    s.depthWrite = (raw & 0x2) != 0;
    s.fog        = (raw & 0x4) != 0;
    s.cull       = (raw & 0x8) ? CullMode::None : CullMode::CounterClockwise;
    return s;
}
```

16.1 Depth testing and depth writes

Depth testing compares a candidate pixel with depth already stored for that screen location. Depth writing updates that stored value when a pixel is accepted. Together, these settings determine much of the ordinary occlusion in opaque scenes. They also matter for later transparent work, which may test against existing depth without writing new depth.

An object disappearing behind another object is consistent with depth testing, but screenshots alone do not establish the state values, target setup, or draw order. A trace can show whether a draw used depth testing and depth writes; a controlled visual test can show the resulting behavior.

16.2 Face culling

Triangle winding gives a renderer a basis for deciding which face is front-facing. Culling can reject front-facing or back-facing triangles, reducing work and preventing the inside of a closed model from appearing. Culling is not equivalent to visibility culling: one removes individual triangles by orientation, while the other may omit an object or group before it is drawn.

16.3 Fog

Fog blends or otherwise modifies a fragment according to depth-related or distance-related rules. It can connect distant geometry to a sky color, reduce contrast, and limit the visual cost of far scenery. Its appearance can resemble alpha fading, lighting, texture change, or post-processing; the relevant fog state, parameters, and ordering are needed to identify it.

16.4 Fill and rasterization choices

Fill mode controls whether primitives are rasterized as points, wireframe, or filled triangles. Other rasterization-related settings can influence clipping, depth bias, shading, and how polygons meet. These settings are often invisible in a normal image, yet they can explain artifacts such as gaps, coplanar flicker, or a diagnostic rendering mode.

16.5 State groups and state blocks

States are most informative when read as groups. A change from opaque environment work to an effect may involve depth writes, culling, fog, alpha rules, texture stages, and blend factors together. Direct3D 8 also provides state-block operations that can capture and apply sets of device state. The presence of grouped changes is evidence of a rendering transition, not automatically of a named effect.

16.6 Recording state evidence

For each draw, record the active values relevant to depth, culling, fog, fill, clipping, and rasterization; then compare them with nearby draws and the resulting image. The most useful conclusion explains the observed difference with the smallest state-supported claim.

> Evidence standard. State values in a trace are confirmed facts about that trace. Their semantic label—such as 'water pass' or 'shadow pass'—requires scene correlation and may remain an inference.

16.7 From decoded material to a reproducible state packet

A reconstruction renderer needs a complete state baseline so one draw cannot inherit culling, alpha, depth, or texture-transform state from the previous draw. Listing 16.1 applies an explicit packet using Direct3D 8. The packet is populated only from supported material fields; texture-scroll rates in this example are a viewer policy and therefore remain separate from decoded evidence.

Listing 16.1. Explicit fixed-function material state and an isolated reconstruction policy.

```
struct MaterialState {
    bool twoSided = false;
```

```
    bool alphaTest = false;
    std::uint8_t alphaReference = 0;
    bool alphaBlend = false;
    bool depthWrite = true;
    bool textureScroll = false;
};

void applyMaterial(IDirect3DDevice8& d, const MaterialState& m, float seconds) {
    d.SetRenderState(D3DRS_CULLMODE, m.twoSided ? D3DCULL_NONE : D3DCULL_CW);
    d.SetRenderState(D3DRS_ALPHATESTENABLE, m.alphaTest);
    d.SetRenderState(D3DRS_ALPHAREF, m.alphaReference);
    d.SetRenderState(D3DRS_ALPHAFUNC, D3DCMP_GREATER);
    d.SetRenderState(D3DRS_ALPHABLENDENABLE, m.alphaBlend);
    d.SetRenderState(D3DRS_SRCBLEND, D3DBLEND_SRCALPHA);
    d.SetRenderState(D3DRS_DESTBLEND, D3DBLEND_INVSRCALPHA);
    d.SetRenderState(D3DRS_ZWRITEENABLE, m.depthWrite);
    d.SetTextureStageState(0, D3DTSS_COLOROP, D3DTOP_MODULATE2X);
    d.SetTextureStageState(0, D3DTSS_COLORARG1, D3DTA_TEXTURE);
    d.SetTextureStageState(0, D3DTSS_COLORARG2, D3DTA_DIFFUSE);

    D3DMATRIX uv = identityD3DMatrix();
    uv._31 = m.textureScroll ? std::fmod(seconds * 0.012f, 1.0f) : 0.0f;
    uv._32 = m.textureScroll ? std::fmod(seconds * 0.006f, 1.0f) : 0.0f;
    d.SetTransform(D3DTS_TEXTURE0, &uv);
    d.SetTextureStageState(0, D3DTSS_TEXTURETRANSFORMFLAGS,
        m.textureScroll ? D3DTTFF_COUNT2 : D3DTTFF_DISABLE);
}
```

Reconstruction boundary. MODULATE2X, the alpha comparison, conventional source-alpha blending, and the example scroll rates must each be promoted or replaced independently when a retail trace or decoded state field establishes the actual value.

17. Alpha Test, Alpha Blending, and Draw Order

Alpha is often described as transparency, but it supports several different rendering decisions. An alpha test can discard a fragment completely. Alpha blending can combine a new fragment with the color already stored in the target. Draw order determines which destination color is available when that combination occurs. These mechanisms must be analyzed separately.

Scope. This chapter describes standard rendering behavior. A transparent-looking result does not establish which alpha mechanism a Final Fantasy XI draw used.

Implementation checkpoint. Decode alpha-related fields and build opaque, cutout, and blended queues with deliberate depth and ordering rules.

```
// Cumulative implementation excerpt: explicit opaque, cutout, and blended queues.
struct QueuedDraw { Draw draw; float cameraDepth; std::uint64_t sourceOrder; };
struct DrawQueues { std::vector<QueuedDraw> opaque, cutout, blended; };

void enqueue(DrawQueues& q, QueuedDraw item, const AlphaDesc& a) {
    if (a.blendEnabled) q.blended.push_back(std::move(item));
    else if (a.testEnabled) q.cutout.push_back(std::move(item));
    else q.opaque.push_back(std::move(item));
}

void submit(DrawQueues& q, Renderer& r) {
    stableStateSort(q.opaque); stableStateSort(q.cutout);
    std::stable_sort(q.blended.begin(), q.blended.end(),
        [](auto& a, auto& b) { return a.cameraDepth > b.cameraDepth; });
    r.draw(q.opaque,  DepthWrite::On);
    r.draw(q.cutout,  DepthWrite::On);
    r.draw(q.blended, DepthWrite::Off);
}
```

17.1 Alpha testing: accepted or discarded

With alpha testing, a sampled alpha value is compared with a reference according to a test function. A fragment that fails is discarded; a fragment that passes proceeds as an ordinary candidate for later operations. The result is a hard cutout rather than partial transparency. This makes alpha testing useful for patterned edges, foliage-like shapes, and masked details where depth behavior should remain similar to opaque geometry.

A jagged or binary-looking edge is compatible with alpha testing, but filtering, resolution, compression, and capture scaling can imitate the same appearance. Confirmation requires the relevant alpha-test state or a controlled test that isolates the threshold behavior.

17.2 Alpha blending: source and destination

Alpha blending combines a source color from the current draw with a destination color already in the render target. Blend factors determine how much of each contributes. Conventional source-alpha blending can create translucency; additive combinations can create light-like accumulation; other factors support specialized results. The important point is that the destination already reflects previous work.

Because blending depends on destination color, blended geometry is sensitive to order. Two overlapping translucent surfaces may look different when drawn in the opposite order. Depth testing may prevent a surface from appearing through nearer opaque geometry, while depth writing is often treated differently to avoid one transparent surface incorrectly hiding another.

17.3 Draw order is part of the material

A material description is incomplete if it names only texture and blend factors. Its place in the frame matters too. Opaque work is commonly arranged so depth can reject hidden pixels efficiently. Cutouts can often participate in that depth structure. Blended work may require a later phase, sorting, grouping, or a deliberate approximation. An engine can make different choices for particles, character accessories, weather, interface layers, and screen-space effects.

17.4 Common visual clues and their limits

A glow that accumulates over a dark background is consistent with additive blending. A pane that reveals the background is consistent with source-destination blending. A leaf-like mask that retains a firm depth edge is consistent with alpha testing. Each statement is only a hypothesis until state evidence and draw context support it. Similar imagery can arise through multiple passes, precomposited textures, or target-based effects.

17.5 A practical alpha record

For an investigated draw, record whether alpha testing is enabled, its function and reference; whether blending is enabled, its source and destination factors; the depth-test and depth-write values; culling; target; draw order; and the visible overlap conditions. Compare the same draw against opaque and blended neighbors. This record is usually more informative than a label such as 'transparent texture.'

> Evidence standard. Blend factors and alpha-test values are confirmed trace facts. Explaining an artistic intent or a pass name requires correlation with the scene and may remain a strong inference.

18. Texture Stages and Fixed-Function Combiners

Before programmable pixel shaders became the standard material language, a fixed-function pipeline could assemble a material through texture stages. Each stage samples a bound texture and combines its values with a current color according to configured operations and arguments. The result of one stage can become an input to the next.

> Scope. This chapter explains what the fixed-function stage model can express. A visible multi-texture effect is not proof that Final Fantasy XI used a particular stage sequence.

> **Implementation checkpoint.** Parse material-stage records into a loggable fixed-function texture-stage and combiner description system.

```
// Cumulative implementation excerpt: parse, log, and apply fixed-function stages.
struct TextureStageDesc {
    std::uint32_t colorOp, colorArg1, colorArg2;
    std::uint32_t alphaOp, alphaArg1, alphaArg2;
    std::uint32_t texCoordIndex;
};

TextureStageDesc readStage(ByteReader& r, EvidenceLog& log) {
    const auto start = r.position();
    TextureStageDesc s{r.u32(), r.u32(), r.u32(), r.u32(), r.u32(), r.u32(), r.u32()};
    log.field(currentResource(), start, "textureStage", s, Confidence::Observed);
    return s;
}

void applyStage(IDirect3DDevice8& d, DWORD n, const TextureStageDesc& s) {
    d.SetTextureStageState(n, D3DTSS_COLOROP, s.colorOp);
    d.SetTextureStageState(n, D3DTSS_COLORARG1, s.colorArg1);
    d.SetTextureStageState(n, D3DTSS_COLORARG2, s.colorArg2);
    d.SetTextureStageState(n, D3DTSS_ALPHAOP, s.alphaOp);
    d.SetTextureStageState(n, D3DTSS_TEXCOORDINDEX, s.texCoordIndex);
}
```

18.1 The current color

A stage works with a current color and, where configured, a current alpha. At the beginning of a material evaluation, those values can originate from diffuse vertex color or other defined inputs. A stage then chooses arguments such as a texture sample, diffuse color, current result, or a complement, and applies a color or alpha operation. Typical operations include selection, modulation, addition, and interpolation.

The separation between color and alpha is important. A material may multiply color while selecting alpha from a different input, or use alpha to control a later blend. An image that looks like one texture can therefore be the product of several independently configured operations.

18.2 Multiple coordinate sets and stages

Each texture stage can use texture coordinates chosen or transformed for that stage. A mesh with more than one coordinate set can support a base image and a differently mapped detail, light-like, or mask-like image. The same coordinates can also be transformed to scroll or project a pattern. These mechanisms are capabilities; determining their use requires vertex-layout, stage-state, and visual evidence together.

18.3 Common material patterns

Modulation can tint a base texture by vertex color or another texture. Addition can create a brighter contribution. Interpolation can select between sources using an alpha-like factor. A later stage can replace an earlier result

instead of extending it. These patterns provide hypotheses for investigating detail textures, masks, light-like overlays, and animated surfaces, but a pattern name must never substitute for the recorded operations.

18.4 Stage order and disabled stages

Stage order matters because each stage can consume the current result from the previous one. A disabled stage terminates the fixed-function chain in the relevant direction. Therefore, an identical set of textures can yield different images when stages are reordered or when their operations change. State diffs should record both texture bindings and the color/alpha operation of every active stage.

18.5 Reading a combiner trace

For each stage, record the bound texture, coordinate source, transform, color operation, color arguments, alpha operation, alpha arguments, and any stage-specific state. Then write a small symbolic expression for the resulting color and alpha. The expression may be provisional, but it makes assumptions visible and lets a later capture test a precise prediction.

> Evidence standard. A recorded stage operation is confirmed API evidence. Identifying its artistic role—detail map, light map, mask, or another term—requires correlation with resources and the scene.

19. Vertex Shaders and Programmable Techniques

Programmable vertex processing expands the work performed on vertex data before rasterization. Instead of relying only on the fixed-function transform-and-lighting path, an application can select a vertex shader that interprets declared inputs and writes transformed positions and other outputs. In the Direct3D 8 era, this flexibility coexisted with fixed-function materials and texture-stage combiners.

> Scope. This chapter is an API and analysis primer. It does not claim that any Final Fantasy XI draw uses a vertex shader until a shader binding, declaration, or equivalent client-specific trace establishes it.

> **Implementation checkpoint.** Load validated shader records, declarations, and constants with capability checks and an explicit fallback path.

```
// Cumulative implementation excerpt: capability-gated shader with fixed-function fallback.
struct ShaderProgram { DWORD handle = 0; bool programmable = false; };

ShaderProgram loadVertexProgram(IDirect3DDevice8& d,
                                const ShaderRecord& record,
                                const D3DCAPS8& caps) {
    if (caps.VertexShaderVersion >= D3DVS_VERSION(1, 1) && record.validated) {
        DWORD handle = 0;
        if (SUCCEEDED(d.CreateVertexShader(record.declaration.data(),
                                           record.bytecode.data(), &handle, 0)))
            return {handle, true};
    }
    d.SetVertexShader(record.fallbackFvf);
    return {record.fallbackFvf, false};
}
```

19.1 What a vertex shader can change

A vertex shader can transform positions, calculate or modify per-vertex lighting values, generate or transform texture coordinates, apply deformation-related calculations, and prepare values for later interpolation across a triangle. It operates per vertex, not per pixel. Therefore, a shader can alter silhouette motion, lighting gradients, coordinate behavior, and other vertex-derived aspects of an image without requiring a programmable pixel stage.

19.2 Inputs, declarations, and constants

A programmable path needs an agreement about input layout and constants. The vertex declaration describes how stream data maps into shader inputs. Constants provide values that can vary by draw or by frame, such as matrices, colors, time-like parameters, or material values. The shader program defines how those inputs become outputs.

For evidence work, shader identity alone is not enough. Record the declaration, streams, stride, constants when recoverable, textures and states, primitive range, and visible result. A shared shader can serve several material classes through different constants and bindings.

19.3 Fixed-function and programmable paths can coexist

An engine may use fixed-function processing for many ordinary draws and reserve programmable vertex processing for selected transformations or effects. It may also switch paths within a frame. This possibility warns against treating a game as either wholly fixed-function or wholly programmable. The relevant unit of analysis is the draw and its active state.

19.4 Visual clues are hypotheses

A deformation, unusual coordinate motion, or complex lighting gradient can motivate a shader hypothesis. None of these appearances uniquely identifies a vertex shader. Comparable results may arise from CPU-prepared vertices, skeletal matrices, fixed-function texture transforms, multiple draws, or precomputed color. A trace that shows a shader selection is stronger evidence than the effect’s appearance; recovered code or constants can strengthen the explanation further.

19.5 A programmable-technique record

When a programmable path is suspected, record the device’s shader selection, declaration, stream bindings, constants, draw call, nearby state changes, and the condition under which the result appears. Preserve unknown values as unknown rather than assigning names by guesswork. A later comparison can then ask whether changing a constant, matrix, or input stream changes the observed effect.

> Evidence standard. A bound vertex shader is confirmed for that draw. Its semantic purpose—skinning, wave motion, coordinate generation, or another role—requires inputs, outputs, and scene behavior to agree.

Part III — Scene Construction and Interactive Workspaces

20. The World Scene: Zones, Chunks, and Draw Groups

A large game world cannot normally be treated as one undifferentiated mesh every frame. The renderer needs a way to select, prepare, and submit manageable portions of the scene. Terms such as zone, chunk, sector, cell, batch, and draw group are useful analytical labels, but they are not interchangeable and should not be imposed on an engine without evidence.

> Scope. This chapter provides a scene-organization vocabulary. It does not confirm the internal partition names, boundaries, or submission groups of Final Fantasy XI.

> **Implementation checkpoint.** Parse a zone resource graph and construct the world scene from validated chunks, entities, references, and draw groups.

```
// Cumulative implementation excerpt: build a scene graph from bounded chunk references.
Scene buildZone(const DatFile& dat, EvidenceLog& log) {
    Scene scene;
    std::unordered_map<ResourceName, Mesh> meshes;
    for (const ChunkView& c : dat.chunks()) {
        if (c.type == 0x2E) {
            Mesh m = parseZoneMesh(c, log);
            meshes.emplace(m.name, std::move(m));
        }
    }
    for (const ChunkView& c : dat.chunks()) if (c.type == 0x1C) {
        for (const Placement& p : parsePlacements(c, log)) {
            auto found = meshes.find(p.meshName);
            if (found == meshes.end()) { scene.missing.push_back(p.meshName); continue; }
            scene.instances.push_back({&found->second, placementMatrix(p), p.flags});
        }
    }
    scene.meshes = std::move(meshes);
    return scene;
}
```

20.1 Zone as an experience boundary

A zone is a useful player-facing unit: a named area with a particular landscape, object population, atmosphere, and loading context. It may correspond to a package of data, a world-space coordinate region, a server concept, a rendering scene, or several of these at once. The word alone does not reveal how geometry and materials are divided for drawing.

20.2 Spatial chunks and selection

A renderer can divide a zone into chunks or cells so that it can consider nearby or visible portions rather than every object. Such a division may follow a grid, a hierarchy, authored rooms, terrain tiles, object lists, or another structure. The observed behavior of objects appearing with camera motion may suggest selection boundaries, but only data or trace evidence can establish their actual form.

A chunk is also not necessarily a draw. One selected region can issue many draws for different materials, layers, or object classes. Conversely, one draw can combine compatible geometry from several nearby pieces. Keep spatial organization separate from submission organization.

20.3 Draw groups and material groups

A draw group is a practical set of geometry rendered under compatible state, material, and transform conditions. Grouping can reduce changes in textures, shaders, streams, and render state, but it may be constrained by transparency, animation, culling, or artistic ordering. A material group is similarly an analytical term until the active bindings and draw ranges demonstrate it.

20.4 Static and dynamic scene content

World scenes commonly contain both stable structural geometry and dynamic content such as characters, moving objects, weather, and interface markers. The distinction is useful because the two categories can differ in update frequency, visibility rules, data lifetime, and order of rendering. It is not a claim that every apparently static or dynamic item follows one shared code path.

20.5 How to investigate a world scene

Start with a repeatable location and camera route. Record which objects appear, disappear, change detail, or change state while the route varies one condition at a time. Pair the route with resource and draw evidence: which buffers, textures, states, and target changes recur? Then ask whether the observations support a spatial partition, a material grouping, a distance rule, or a combination.

A strong result names only what the evidence supports. For example, 'these draws recur together when the camera crosses this boundary' is stronger than prematurely naming the structure a chunk. Later evidence may show that the boundary is a zone portal, a material switch, or an unrelated event.

> Evidence standard. A world-scene classification becomes confirmed only when spatial data, client behavior, or trace evidence establishes it. Visual proximity alone is not an internal grouping rule.

20.6 The provenance spine of a world draw

A scene node should not erase how it was produced. The records below preserve the resource identifier, chunk and field offsets, placement, material, state, and draw range. An inspector can therefore walk backward from a selected pixel contribution to the exact file ranges that supplied it.

Listing 20.1. Provenance retained from decoded field to draw submission.

```
struct DecodedField {
    ResourceId resource;
    std::size_t chunkOffset;
    std::size_t fieldOffset;
    std::size_t byteCount;
    std::string name;
    std::string value;
    Confidence confidence;
};

struct DrawProvenance {
    ResourceId resource;
    std::size_t geometryChunk;
    std::vector<DecodedField> fields;
    std::string objectName;
    Mat4 placement;
    std::string materialName;
    MaterialState state;
    std::size_t firstIndex, indexCount;
};

DrawProvenance makeDrawProvenance(const Placement& placement,
```

```
                                const SupportedGeo& mesh,
                                const EvidenceLog& log) {
    return {placement.source.resource, mesh.source.chunkOffset,
            log.fieldsFor(mesh.source), placement.objectName,
            placement.world, mesh.material, placement.materialState,
            0, mesh.indices.size()};
}
```

The corresponding UI should display unsupported references and unresolved object names alongside successful instances. Missing geometry is evidence about the current decoder or resource set, not permission to silently omit the placement from the scene report.

21. Zone Loading and Scene Assembly

A zone is not one mesh. This chapter builds a loader that resolves a resource graph, decodes supported chunks, instantiates reusable models, applies placements, attaches environment state, and publishes a scene only after its required foundation is valid.

> Build outcome.

Implement dependency-aware zone loading and publish an immutable scene snapshot suitable for rendering and inspection.

21.1 Responsibilities and ownership

ZoneDocument separates immutable source records, decoded asset handles, instance transforms, spatial indices, environment controls, and diagnostics. Render objects never keep raw byte pointers.

21.2 Implementation sequence

The loader first discovers roots and dependencies, then validates and decodes in parallel, assembles instances on one deterministic thread, builds bounds and spatial structures, uploads visible priorities, and streams optional content afterward.

21.3 Failure and recovery

Cycles, missing dependencies, unsupported chunks, and cancelled optional work become diagnostics. Failure of required placement or coordinate data prevents scene publication; failure of one decoration does not.

21.4 Verification gate

The fixture scene verifies dependency closure, deterministic instance order, transform provenance, bounds, partial-content behavior, cancellation, and a reference overview image.

21.5 Cumulative C++ implementation

> Implementation checkpoint. Implement dependency-aware zone loading and publish an immutable scene snapshot suitable for rendering and inspection.

```
// Cumulative implementation listing
ZoneLoadResult ZoneLoader::load(ResourceId root, StopToken stop) {
    ZoneLoadResult result;
    ResourceGraph graph = discoverDependencies(root, stop, result.issues);
    auto decoded = decoderPool_.decode(graph.nodes(), stop, result.issues);
    if (stop.stopRequested()) return result;
```

```
    ZoneBuilder builder(root);
    for (ResourceId id : graph.topologicalOrder()) {
        const DecodedResource* r = decoded.find(id);
        if (!r) continue;
        builder.accept(*r, result.issues);
    }
    if (!builder.hasRequiredFoundation()) {
        result.issues.push_back({Severity::Error, "required zone records missing"});
        return result;
    }
    result.scene = builder.finishDeterministically();
    return result;
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

22. Object Explorer, Selection, and Property Editing

A scene viewer becomes an exploration tool when its hierarchy, viewport, and properties agree on identity. This chapter implements a virtualized object tree, picking and highlighting, source navigation, and reversible edits stored separately from decoded data.

Build outcome.

Build synchronized tree, viewport, and property inspection with stable IDs, highlighting, reversible overrides, and source navigation.

22.1 Responsibilities and ownership

SceneObjectId is stable for one scene snapshot. SelectionModel is shared by tree, list, viewport, and property panel. EditLayer stores overrides keyed by object ID and never mutates source records.

22.2 Implementation sequence

Filtering produces a visible row index rather than copying objects. Viewport picking returns candidate IDs and depth; the selection model applies modifier rules; every observer updates from one versioned selection event.

22.3 Failure and recovery

Deleted or filtered objects remain valid selections but may be hidden. Invalid edits are rejected before entering undo history. Rebuilding a scene attempts selection restoration through source identity and reports ambiguity.

22.4 Verification gate

Tests cover ten-thousand-row virtualization, pick ordering, multi-selection, filter changes, undo/redo, source navigation, and selection restoration after reload.

22.5 Cumulative C++ implementation

Implementation checkpoint. Build synchronized tree, viewport, and property inspection with stable IDs, highlighting, reversible overrides, and source navigation.

```
// Cumulative implementation listing
void SelectionModel::replace(std::span<const SceneObjectId> ids,
                             SelectionOrigin origin) {
    std::vector<SceneObjectId> next(ids.begin(), ids.end());
```

```
    std::sort(next.begin(), next.end());
    next.erase(std::unique(next.begin(), next.end()), next.end());
    if (next == selected_) return;
    selected_ = std::move(next);
    ++version_;
    changed_.publish({version_, origin, selected_});
}

bool EditLayer::setTransform(SceneObjectId id, const Transform& value) {
    if (!isFinite(value) || value.scale.minComponent() <= 0.0f) return false;
    history_.push(TransformCommand{id, overrides_[id].transform, value});
    overrides_[id].transform = value;
    return true;
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

23. Collision Geometry and Navigation Diagnostics

Collision data serves inspection even when it is not drawn by the original renderer. The explorer exposes collision meshes, walkability hypotheses, ray tests, grounding, and discrepancy overlays without confusing collision records with visible terrain.

Build outcome.

Implement collision inspection, acceleration, ray queries, optional grounding, and visual discrepancy overlays with explicit confidence labels.

23.1 Responsibilities and ownership

CollisionDocument stores decoded triangles, source ranges, acceleration structure, and classification labels. Navigation state references collision snapshot versions so results cannot silently outlive a reload.

23.2 Implementation sequence

Decoding validates indices and finite coordinates, removes no triangles silently, builds a bounding-volume hierarchy, and generates optional diagnostic classifications in a separate derived layer.

23.3 Failure and recovery

Degenerate or non-finite triangles are quarantined with source references. If no collision data is supported, camera navigation falls back to free flight and labels grounding unavailable.

23.4 Verification gate

Fixtures verify triangle preservation, ray-hit order, BVH equivalence with brute force, grounding tolerances, version invalidation, and overlay state isolation.

23.5 Cumulative C++ implementation

Implementation checkpoint. Implement collision inspection, acceleration, ray queries, optional grounding, and visual discrepancy overlays with explicit confidence labels.

```
// Cumulative implementation listing
std::optional<RayHit> CollisionScene::cast(const Ray& ray) const {
    std::optional<RayHit> nearest;
```

```
    bvh_.visitRay(ray, [&](TriangleId id) {
        const Triangle& t = triangles_[id.value];
        if (auto hit = intersect(ray, t.positions)) {
            if (!nearest || hit->distance < nearest->distance)
                nearest = RayHit{id, hit->distance, hit->barycentric, t.source};
        }
    });
    return nearest;
}

std::optional<float> CollisionScene::groundHeight(Vec3 p, float above) const {
    Ray ray{{p.x, p.y + above, p.z}, {0.0f, -1.0f, 0.0f}};
    if (auto hit = cast(ray)) return ray.origin.y - hit->distance;
    return std::nullopt;
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

24. Visibility: Frustum Culling, Distance, and Occlusion

Visibility is the process of deciding which candidate work merits rendering for a particular view. It should not be reduced to one test. A renderer may exclude objects outside the camera frustum, omit content beyond a distance rule, select simpler representations, reject geometry hidden by other geometry, or use authored visibility boundaries. The same object disappearing from a screenshot can be consistent with several of these mechanisms.

Scope. This chapter is a visibility-analysis framework. It does not establish which visibility systems Final Fantasy XI implements.

Implementation checkpoint. Decode supported visibility fields and implement frustum, distance, and optional occlusion tests while preserving unknown flags.

```
// Cumulative implementation excerpt: layered visibility without discarding unknown flags.
bool visible(const VisibilityRecord& v, const Camera& camera,
             const OcclusionBuffer* occlusion) {
    if (!camera.frustum().intersects(v.bounds)) return false;
    const float distance = length(v.bounds.center() - camera.eye);
    if (distance > v.drawDistance) return false;
    if (occlusion && v.occlusionTestSupported && occlusion->hidden(v.bounds)) return false;
    return true; // v.unknownFlags remain stored for diagnostics.
}

std::vector<const Instance*> gatherVisible(const Scene& s, const Camera& c) {
    std::vector<const Instance*> out;
    for (const Instance& i : s.instances)
        if (visible(i.visibility, c, s.occlusion.get())) out.push_back(&i);
    return out;
}
```

24.1 The viewing frustum

The view and projection transforms define a volume of space that can reach the screen. Frustum culling tests a candidate object or bounding volume against that volume and can reject work that lies entirely outside it. The result depends on camera position, orientation, projection, and the bounds chosen for the candidate. A large bound can remain visible after its detailed geometry would appear off-screen; a small bound can be more selective.

24.2 Distance and detail selection

Distance-based rules can remove, simplify, or substitute content as it becomes less important to the image. The transition may be abrupt or gradual and may depend on category, camera direction, quality settings, or scene

conditions. A distant object becoming less detailed is not automatically geometric LOD: fog, mipmapping, texture filtering, alpha behavior, and display scaling can produce related visual changes.

24.3 Occlusion and hidden work

Occlusion concerns geometry that is within the camera's broad view but hidden behind other geometry. Some systems test occlusion before submitting work; others rely mostly on depth testing after submission; still others use authored portals or room boundaries. A pixel-level depth result does not prove an earlier occlusion-culling stage. The question is whether the candidate was omitted before drawing, and that requires draw or selection evidence.

24.4 Visibility is often hierarchical

A renderer can first select a zone region, then test object groups, then let depth reject individual hidden pixels. Different content may use different levels of this hierarchy. Terrain, static structures, characters, particles, and interface elements need not share the same visibility policy. Do not infer a universal rule from a single scene class.

24.5 A controlled visibility experiment

Choose a stable object and vary one camera parameter at a time: rotate until it leaves the frustum, move away while preserving direction, move behind an occluder, or alter a documented quality setting. Record whether the object's draw calls, resource bindings, or only its final pixels change. Repeating the test at several positions helps distinguish a continuous visual effect from a discrete selection boundary.

A good result might state: 'the object's draws cease after this recorded distance under these conditions.' It should not jump to a particular culling algorithm unless the trace, bounds, or implementation evidence supports that name.

> Evidence standard. An absent draw is evidence of pre-raster selection for the captured conditions. An absent pixel with a present draw may instead be a depth, alpha, fog, or clipping result.

25. Terrain, Ground, and Large-Scale Environment Geometry

Ground rendering carries much of a world's scale, navigation readability, and atmosphere. It may combine terrain-like meshes, authored structural surfaces, repeated materials, vertex color, fog, decals, and distance rules. The word terrain is useful visually, but it should not be used as a claim about a particular data format or generation method without evidence.

> Scope. This chapter provides an analysis framework for large environment surfaces. It does not confirm the ground data structures or rendering paths of Final Fantasy XI.

Implementation checkpoint. Parse terrain sectors into reusable ground geometry, materials, bounds, and large-environment submissions.

```
// Cumulative implementation excerpt: validated terrain sectors become reusable draws.
TerrainSector readTerrainSector(ByteReader& r, std::size_t end) {
    TerrainSector s;
    s.bounds = readAabb(r);
    s.material = ResourceId{r.u32()};
    const auto vertexCount = r.u32(), indexCount = r.u32();
    s.vertices = readTerrainVertices(r, vertexCount, end);
    s.indices = r.arrayWithin<std::uint16_t>(indexCount, end);
    validateIndices(s.indices, s.vertices.size());
    return s;
```

```
}

void submitTerrain(const Terrain& terrain, const Frustum& f, DrawList& draws) {
    for (const TerrainSector& sector : terrain.sectors)
        if (f.intersects(sector.bounds))
            draws.add({sector.gpuGeometry, sector.material, sector.bounds});
}
```

25.1 Geometry and surface appearance are separate

A low-density mesh can appear rich through texture detail, color variation, lighting, fog, and layered materials. Conversely, a dense mesh can appear simple under a uniform material. Analysis should therefore separate geometric evidence—vertex positions, topology, draw ranges, transforms—from surface evidence—textures, stages, vertex color, state, and distance behavior.

25.2 Repetition and variation

Large surfaces often reuse a limited set of images. Variation can arise from tiling, coordinate transforms, vertex colors, masks, overlays, or separate objects. A repeated pattern in a screenshot can suggest a tiled material, but it does not prove coordinate scale, texture dimensions, or the number of stages. Resource and stage evidence are needed to make those claims.

25.3 Boundaries and seams

Visible boundaries may mark a change in geometry, material, texture coordinate behavior, fog, lighting, zone data, or an authored decal-like overlay. Seams are particularly useful because they can be tested from several angles and distances. If a seam moves with the camera, it may be presentation or projection related; if it follows geometry, it may reflect a material or spatial boundary.

25.4 Ground, collision, and rendering

A surface used for player movement or collision need not be the same surface rendered to the screen. Render geometry can be simplified, displaced, split into materials, or absent where collision remains; collision can also include invisible boundaries. Keep movement observations separate from graphics claims until both data sets can be connected.

25.5 Large-scale distance treatment

Fog, mipmaps, reduced geometric detail, visibility selection, and sky composition can all contribute to a convincing horizon. Their effects interact: a missing distant feature may have been culled, hidden by fog, merged into a lower-detail representation, or never submitted for the test condition. Use draw records and controlled camera routes to separate these possibilities.

25.6 Investigation checklist

For a selected ground region, record camera route, transforms, geometry buffers and ranges, material and texture-stage state, vertex colors when available, distance changes, neighboring draw groups, and visible seams. The goal is to reconstruct a surface from evidence rather than to infer its construction from a map-like image.

> Evidence standard. A terrain label is descriptive until geometry, resource, or client evidence defines the actual rendering unit.

26. Static Objects, Props, and Architectural Detail

Static objects give a zone its readable landmarks and material character: walls, doors, bridges, furniture, vegetation-like forms, and decorative structures. They may share geometry or textures while differing in placement, transform, material state, visibility rule, or animation state. Visual repetition is evidence of a relationship worth investigating, not proof of one resource or draw strategy.

> Scope. This chapter offers a method for analyzing static scene content. It does not identify Final Fantasy XI's object database, instancing system, or placement format.

> **Implementation checkpoint.** Parse model and placement records, then instantiate static props and architecture with shared geometry and per-instance state.

```
// Cumulative implementation excerpt: shared model geometry, per-placement state.
struct ModelInstance {
    std::shared_ptr<const Model> model;
    Mat4 world;
    InstanceFlags flags;
    ResourceId environment;
};

ModelInstance instantiate(const PlacementRecord& p, ModelCache& cache) {
    auto model = cache.getOrLoad(p.modelId);
    Mat4 world = translation(p.position) * rotationZ(p.rotation.z)
               * rotationY(p.rotation.y) * rotationX(p.rotation.x)
               * scale(p.scale);
    return {std::move(model), world, decodeInstanceFlags(p.rawFlags), p.environmentId};
}
```

26.1 Placement is separate from the model

A model describes reusable geometry and associated material possibilities. Placement gives it a location, orientation, scale, and sometimes additional flags or variants. Several placements can point to the same resource while producing distinct screen results. Conversely, visually similar props can be separate models with matching materials. Keep resource identity and world placement as separate evidence fields.

26.2 Reuse and variation

Reuse can save memory and reduce authoring cost, while variation avoids a visibly mechanical world. Transform differences, mirrored orientation, texture variation, vertex color, decals, nearby lighting, and partial occlusion can make shared assets appear different. A comparison should first establish what repeats on screen, then test whether buffer identities, textures, and draw ranges also repeat.

26.3 Architectural composition

Architecture often combines large structural surfaces with smaller details that may use distinct materials and draw order. A window, trim, sign, or transparent ornament can be a separate draw even when it visually belongs to one building. This is why object-level labels are often too coarse for trace analysis: one player-recognized object may consist of several geometry and material submissions.

26.4 Static does not mean unchanging

A static placement can still have animated textures, changing vertex colors, condition-dependent materials, or attached effects. It can also be selected or culled differently as the camera moves. Use static to mean stable placement unless evidence demonstrates a stronger claim about resource lifetime or render behavior.

26.5 Investigation checklist

For each prop class, record a visual reference, placement transform, geometry and index ranges, material and texture bindings, state group, nearby repeated instances, and changes across distance or time. Look for correlations across multiple placements before naming a shared resource. The most useful result distinguishes what is visually repeated, what is data-shared, and what is draw-shared.

> Evidence standard. Matching appearance is a hypothesis of reuse. Matching resource identifiers and compatible draw evidence can confirm reuse for the recorded client and scene.

27. Characters: Assembly, Skinning, and Equipment

A visible character is often an assembly rather than one mesh. Body parts, head, hair, equipment, weapons, accessories, and effects can have separate geometry, materials, transforms, and draw order. Character analysis must therefore move from the player-facing whole to a documented set of renderable components.

> Scope. This chapter explains common character-rendering concepts. It does not establish a particular Final Fantasy XI character format, skeleton layout, or equipment assembly rule.

> **Implementation checkpoint.** Decode validated character components and assemble equipment selections, skeleton bindings, and skinned geometry.

```
// Cumulative implementation excerpt: assemble parts and CPU-skin validated influences.
Character assembleCharacter(const CharacterSelection& choice, AssetRepository& assets) {
    Character c;
    c.skeleton = assets.loadSkeleton(choice.skeleton);
    for (ResourceId partId : choice.visibleParts) {
        CharacterPart part = assets.loadCharacterPart(partId);
        validateBonePalette(part, c.skeleton);
        c.parts.push_back(std::move(part));
    }
    return c;
}

Vec3 skinPosition(const SkinnedVertex& v, std::span<const Mat4> bones) {
    Vec3 p{};
    for (const Influence& i : v.influences)
        p += transformPoint(bones.at(i.bone), i.localPosition) * i.weight;
    return p;
}
```

27.1 Assembly and attachment

Assembly determines which components belong to a character instance and how they are placed relative to one another. Some parts may share a skeleton; others may follow an attachment node; still others may be independent objects updated in parallel. Matching screen position is not enough to identify an attachment relationship. Compare transforms, update behavior, and draw timing.

27.2 Skinning

Skinning deforms a mesh according to a skeleton pose. A vertex can be influenced by one or more bones or nodes, with weights determining their contribution. The final pose may be evaluated on the CPU, through fixed-function matrix blending, through programmable vertex processing, or by another path. Visible joint motion is evidence of deformation or articulated placement, not a unique implementation choice.

To investigate skinning, record vertex layout, blend-related inputs where present, active transforms or constants, shader or fixed-function state, and a controlled pose sequence. Separating rigid attachments from deforming surfaces prevents a common error: calling every moving equipment part skinned.

27.3 Equipment and material boundaries

Equipment can change silhouette, texture bindings, material states, and the number of draws. A visual slot does not necessarily map to one mesh or one texture. Conversely, several visual parts can be merged into one compatible submission. Use draw ranges and resource bindings to identify the render boundary rather than relying only on a menu or item name.

27.4 Layering and transparency

Characters can include cutout hair, translucent ornaments, additive effects, and opaque body surfaces. These may require different depth and blending treatment even when they originate from one conceptual outfit. Character captures should be analyzed with their neighboring draws so that attachment, transparency, and ordering artifacts are not mistaken for geometry faults.

27.5 A character evidence record

Record the character state, camera conditions, visible equipment set, component draws, buffers and ranges, textures, material states, transforms, animation time, and overlap behavior. Repeating the capture with one equipment or pose change can identify which draws are stable and which belong to the changed component.

> Evidence standard. A confirmed assembly relation requires shared identifiers, transforms, animation correlation, or trace evidence. Similar appearance or a familiar equipment category is only a hypothesis.

28. Character Assembly Workspace

Character rendering becomes tractable when assembly is a visible data model. This workspace exposes race and body choices, equipment slots, component resources, skeleton bindings, materials, draw groups, and unresolved substitutions.

> Build outcome.

Build a character assembly workspace with slot resolution, compatibility checks, asynchronous replacement, provenance, and presets.

28.1 Responsibilities and ownership

CharacterSelection is user intent; CharacterRecipe is resolved resource identity; CharacterDocument is decoded immutable content; CharacterInstance contains pose and runtime visibility. Keeping these layers separate makes presets and diagnostics reliable.

28.2 Implementation sequence

A selection change resolves only affected slots, reuses shared resources, validates skeleton compatibility, rebuilds draw groups, and atomically swaps the completed instance into the viewport.

28.3 Failure and recovery

Missing parts use a labeled empty slot or explicit diagnostic substitute. Incompatible palettes are rejected; they are never clamped into a superficially plausible pose.

28.4 Verification gate

Tests cover every slot independently, shared-resource reuse, incompatible skeletons, rapid selection cancellation, preset round-trip, and stable screenshot framing.

28.5 Cumulative C++ implementation

> Implementation checkpoint. Build a character assembly workspace with slot resolution, compatibility checks, asynchronous replacement, provenance, and presets.

```
// Cumulative implementation listing
CharacterBuildResult CharacterAssembler::build(const CharacterSelection& s,
                                               StopToken stop) {
    CharacterBuildResult out;
    out.recipe = catalog_.resolve(s, out.issues);
    out.skeleton = assets_.loadSkeleton(out.recipe.skeleton, stop);
    for (const ResolvedPart& part : out.recipe.parts) {
        if (stop.stopRequested()) return out;
        auto model = assets_.loadCharacterPart(part.resource, stop);
        if (!paletteFits(model, out.skeleton)) {
            out.issues.push_back({Severity::Error, part.slot,
                                  "bone palette is incompatible"});
            continue;
        }
        out.parts.push_back({part.slot, std::move(model), part.source});
    }
    out.complete = !stop.stopRequested() && out.skeleton.valid();
    return out;
}
```

> Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

29. Animation, Pose Evaluation, and Motion Data

Animation in this client family is broader than skeletal pose playback. Character clips, effect curves, texture-coordinate motion, generator clocks, and child-effect scheduling all change visible state over time. The first implementation rule is therefore to name the clock and the animated quantity before interpreting a field.

> Evidence scope. Skeletal formats and generator curves are separate resource families. The curve and generator observations below are supported by DAT structure, corpus comparison, and controlled parameter-editing experiments; unresolved fields remain labeled.

29.1 Clips, tracks, and time domains

A skeletal clip advances pose channels. A scheduler opens an effect-emission window. A generator has an emission cadence. Each particle has its own age and lifetime. A curve evaluates normalized age, usually from 0 to 1. These clocks can overlap but are not interchangeable: stopping emission does not immediately kill particles that were already created.

29.2 Pose evaluation

Pose evaluation combines the active motion data with the skeleton hierarchy and bind transforms. The resulting matrices place rigid components, deform skinned geometry, and provide attachment transforms for actor-bound effects. Effect attachment should consume the evaluated bone transform for the current frame, not a stale bind-pose position.

29.3 Blending and layered motion

Character motion, equipment placement, facial controls, and attached effects can share an actor while retaining different data sources and clocks. A correct client updates the pose first, resolves attachments second, and simulates or renders dependent effects afterward. This ordering is an implementation requirement; exact retail task scheduling requires separate evidence.

29.4 Normalized effect curves

Type-0x19 resources contain paired scalar samples whose first component progresses through normalized time and whose second component is the value. Generator commands can bind such curves to position, rotation, scale, RGBA, UV coordinates, and morph-related controls. Linear interpolation between adjacent pairs is a supported working model; the curve's use is confirmed only when the referencing command and visible result agree.

```
struct CurveKey { float t; float value; };

float sampleCurve(std::span<const CurveKey> keys, float normalizedAge) {
    if (keys.empty()) return 0.0f;
    const float t = std::clamp(normalizedAge, 0.0f, 1.0f);
    auto right = std::lower_bound(keys.begin(), keys.end(), t,
        [](const CurveKey& k, float x) { return k.t < x; });
    if (right == keys.begin()) return right->value;
    if (right == keys.end()) return keys.back().value;
    const CurveKey& b = *right;
    const CurveKey& a = *(right - 1);
    const float u = (b.t == a.t) ? 0.0f : (t - a.t) / (b.t - a.t);
    return std::lerp(a.value, b.value, u);
}
```

29.5 Constant rates versus curves

The format can express both a constant per-frame change and a referenced curve. Velocity, angular velocity, scale rate, and UV scrolling belong to the first category; normalized position, rotation, scale, color, opacity, UV, and morph channels belong to the second. Preserve both representations instead of baking every command into sampled frames.

29.6 Capturing an animation experiment

Record the resource, command section, opcode, payload bytes, start condition, emission window, particle lifetime, curve identity, and visible timestamps. Compare the decoded duration and loop behavior with capture. If modifying one field changes only emission frequency while existing particles continue normally, that is evidence for a generator clock rather than particle lifetime.

> Implementation checkpoint. Maintain explicit SkeletonTime, SchedulerTime, GeneratorTime, and ParticleAge values. Do not reuse one frame counter for all four systems.

```
// Cumulative implementation excerpt: four clocks with explicit ownership.
struct AnimationClocks {
    double skeletonSeconds = 0.0;
```

```
    double schedulerSeconds = 0.0;
    std::uint64_t generatorFrame = 0;
};
struct ParticleClock { std::uint32_t ageFrames = 0, lifetimeFrames = 1; };

void advance(AnimationClocks& c, std::span<ParticleClock> particles, double dt) {
    c.skeletonSeconds += dt;
    c.schedulerSeconds += dt;
    ++c.generatorFrame;
    for (ParticleClock& p : particles) ++p.ageFrames;
}

bool alive(const ParticleClock& p) { return p.ageFrames < p.lifetimeFrames; }
```

Evidence standard. A monotonic pair table and a referencing command establish a curve relationship. The artistic purpose, interpolation rule, and retail update order require either controlled behavioral tests or original-client execution evidence.

29.7 A minimal skeletal-motion evaluator

The supported motion slice stores a base value for each transform component and optional channel offsets into a shared frame-value array. Listing 29.1 samples those channels at 30 frames per second, normalizes the quaternion, composes local transforms, and evaluates parents before children. It does not repair suspicious samples or resample incompatible clips.

Listing 29.1. Base-plus-channel pose evaluation and hierarchical composition.

```
struct BoneTrack {
    Vec3 baseTranslation;
    Quaternion baseRotation;
    std::array<std::optional<std::size_t>, 3> translationChannel;
    std::array<std::optional<std::size_t>, 4> rotationChannel;
};

LocalPose sampleTrack(const BoneTrack& track, std::span<const float> samples,
                      std::size_t frame, std::size_t frameCount) {
    LocalPose pose{track.baseTranslation, track.baseRotation};
    for (int axis = 0; axis < 3; ++axis)
        if (track.translationChannel[axis])
            pose.translation[axis] = samples.at(*track.translationChannel[axis] + frame);
    for (int component = 0; component < 4; ++component)
        if (track.rotationChannel[component])
            pose.rotation[component] = samples.at(*track.rotationChannel[component] + frame);
    pose.rotation = normalize(pose.rotation);
    return pose;
}

std::vector<Mat4> evaluateSkeleton(const Skeleton& skeleton,
                                   const Motion& motion, float seconds) {
    if (motion.frameCount == 0 || motion.tracks.size() != skeleton.bones.size())
        throw ParseError("motion and skeleton are incompatible");
    const std::size_t frame = std::min<std::size_t>(
        static_cast<std::size_t>(seconds * 30.0f), motion.frameCount - 1);
    std::vector<Mat4> world(skeleton.bones.size());
    for (std::size_t i = 0; i < skeleton.bones.size(); ++i) {
        const LocalPose local = sampleTrack(motion.tracks[i], motion.samples,
                                            frame, motion.frameCount);
        const Mat4 localMatrix = compose(local.translation, local.rotation);
        const int parent = skeleton.bones[i].parent;
        if (parent >= static_cast<int>(i))
            throw ParseError("bone hierarchy is not parent-first");
        world[i] = parent < 0 ? localMatrix : world.at(parent) * localMatrix;
    }
    return world;
}
```

Reconstruction boundary. Quaternion hemisphere repair, gap interpolation, head/body compatibility fallbacks, root-motion removal, and clip resampling are useful viewer policies. They must be logged as policies and tested against the untouched decoded samples rather than described as retail-client behavior.

30. Animation Browser and Playback Controller

Animation support needs more than pose evaluation. The application must catalog clips, preview them deterministically, scrub time, select loops, compare tracks, and explain missing or repaired channels.

Build outcome.

Build an animation catalog and transport controller that drives the chapter’s skeletal evaluator and exposes every fallback.

30.1 Responsibilities and ownership

AnimationCatalog stores identities and metadata; PlaybackController stores transport state; PoseEvaluator is pure for a clip, skeleton, and time; AnimationReport records fallbacks and source ranges.

30.2 Implementation sequence

The browser scans metadata before channel payloads. Play advances by measured time, scrub evaluates an exact requested time, and comparison mode evaluates two clips against the same skeleton and camera.

30.3 Failure and recovery

Zero-length clips, incompatible tracks, invalid parent order, and non-finite keys stop evaluation with diagnostics. Loop and missing-channel behavior are policies shown in the interface, not hidden guesses.

30.4 Verification gate

Tests verify exact boundary times, loop wrap, pause stability, scrub determinism, hierarchy validation, comparison alignment, and pose hashes.

30.5 Cumulative C++ implementation

Implementation checkpoint. Build an animation catalog and transport controller that drives the chapter’s skeletal evaluator and exposes every fallback.

```
// Cumulative implementation listing
void PlaybackController::update(double elapsedSeconds) {
    if (!state_.playing || state_.duration <= 0.0) return;
    state_.time += elapsedSeconds * state_.rate;
    if (state_.looping)
        state_.time = positiveModulo(state_.time, state_.duration);
    else if (state_.time >= state_.duration) {
        state_.time = state_.duration;
        state_.playing = false;
    }
    changed_.publish(state_);
}

void PlaybackController::seek(double seconds) {
    state_.time = std::clamp(seconds, 0.0, state_.duration);
    changed_.publish(state_);
}
```

> Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

31. Lighting Models, Vertex Color, and Material Response

Lighting is the relationship between scene inputs and the color ultimately written for a surface. In a Direct3D-era renderer, that relationship can involve normals, lights, material properties, vertex color, textures, fog, and texture-stage operations. A bright or dark region is an observation; the lighting model behind it is a claim that needs inputs and state evidence.

> Scope. This chapter explains analytical categories for surface shading. It does not identify Final Fantasy XI's light setup, material equations, or use of vertex color for a particular draw.

Implementation checkpoint. Translate decoded material, light, vertex-color, and texture-stage inputs into an inspectable shading configuration.

```
// Cumulative implementation excerpt: one inspectable shading configuration.
struct ShadingConfig {
    Color4 ambient, diffuse;
    Vec3 lightDirection;
    Color4 vertexColor;
    std::array<TextureStageDesc, 2> stages;
    std::uint32_t stageCount;
};

ShadingConfig makeShading(const MaterialRecord& m, const LightState& l,
                          Color4 vertexColor) {
    ShadingConfig s{l.ambient, l.diffuse, normalize(l.direction), vertexColor, {}, 0};
    for (const auto& stage : m.validatedStages) {
        if (s.stageCount == s.stages.size()) break;
        s.stages[s.stageCount++] = stage;
    }
    return s;
}
```

31.1 Normals and directional response

A normal gives a surface orientation used by many lighting calculations. As the relationship between a normal and a light direction changes, diffuse response can change. Specular-like highlights, ambient terms, and emissive contributions may add other behavior. The same visual gradient can also be baked into a texture or vertex color, so a normal-bearing layout alone does not prove dynamic lighting.

31.2 Vertex color

Vertex color stores color values at vertices and is interpolated across a primitive. It can represent precomputed shading, local tint, ambient variation, material masks, lighting input, or other information. Because it is cheap to interpolate, it can add substantial variation without an additional texture. Its semantic role must be inferred from correlation with data, state, and changing scene conditions.

31.3 Material response

A material controls how available inputs become output color. It may select a texture, modulate it with diffuse or vertex color, add another contribution, or route alpha separately. The visible response can depend on lighting state, texture-stage operations, blend state, fog, and target contents. Therefore a material name is not an equation; record the active inputs and operations.

31.4 Baked, dynamic, and mixed lighting

A scene can combine precomputed color with dynamic lights or time-dependent changes. Baked information may reside in textures, vertex colors, or separate overlays. Dynamic information may be computed per vertex, supplied through changing constants, or represented by moving materials. A mixed result is common in real-time graphics, but the split must be shown rather than presumed from the image.

31.5 Investigation checklist

For an investigated surface, record normals and color attributes where available, material and texture-stage operations, light-related state or constants, fog, target and blend state, and captures under controlled changes in camera, time, or nearby conditions. Ask what changes when the hypothesized input changes. A stable texture pattern and a camera-independent vertex gradient support different conclusions from a response that follows a documented light parameter.

> Evidence standard. Vertex colors or normals in data are confirmed attributes. Calling them baked lighting, dynamic lighting, ambient occlusion, or another semantic role requires a tested correlation.

Part IV — Atmosphere, Effects, and Content Tools

32. Sky, Celestial Objects, and Time of Day

The sky combines environment records with geometry that does not necessarily appear in the ordinary zone placement table. The decoded environment supplies clear color, fog, draw distance, lighting colors, and sky-dome rings; celestial and cloud-like geometry can be activated through the effect system.

> Evidence scope. The environment-record layout and camera-attached dome model are strongly supported. The activation and exact transforms of every celestial or cloud asset are not yet completely decoded.

32.1 Environment record and sky dome

Type-0x2F records provide an indoor flag, model and terrain light configurations, clear color, draw distance, sky radius, eight ring colors, and eight elevation stops. Outdoor records form a radial gradient dome. Draw it after the clear, with depth writing disabled and view translation removed, then restore depth state before the world.

32.2 Hour and weather interpolation

The active record is selected by environment identity, weather tag, and time. Blend adjacent hour records across midnight. Treat fogFar equal to zero as a discrete fog-disabled sentinel instead of interpolating it through small positive values.

32.3 Generator-activated sky layers

Zone DATs contain sky-, cloud-, star-, sun-, and moon-like meshes that may be absent from the placement list. Historical parameter-editing experiments and later corpus work agree that generators can activate environmental assets, including sky and water-related layers. A renderer that draws only placement records will therefore omit authored content.

32.4 Celestial orientation and depth

A celestial card may be camera-facing, axis-constrained, or camera-attached. These modes must be decoded from effect orientation data rather than guessed from the mesh name. Reported celestial draws use a large depth bias and generally avoid writing depth, but exact per-asset state remains a command- and pass-level question.

32.5 Implementation order

Implement the dome first because its record and draw model are better established. Next resolve generator resource references and render supported celestial cards. Add cloud motion and weather-linked activation only after their commands and dependencies can be traced.

> Implementation checkpoint. Keep environment interpolation, sky-dome drawing, and generator execution as separate modules joined by decoded resource identities.

```
// Cumulative implementation excerpt: environment, dome, and generators stay separate.
struct ZoneAtmosphere {
    EnvironmentInterpolator environments;
    SkyDomeRenderer sky;
    EffectRuntime effects;

    void draw(RenderContext& rc, WeatherTag weather, GameMinute minute) {
```

```
        const EnvironmentState state = environments.sample(weather, minute);
        rc.clear(state.clearColor);
        sky.draw(rc, state.sky, removeTranslation(rc.camera().view()));
        rc.applyFog(state.fog);
        effects.activate(state.environmentEffects);
    }
};
```

Evidence standard. Absence from the placement table plus presence in a generator dependency graph supports generator activation. It does not by itself prove the final transform, orientation constraint, or draw pass.

32.6 Reconstruction laboratory: a camera-attached sky

When a decoded retail sky mesh is not yet supported, a vertex-colored dome can test environment colors, depth ordering, fog isolation, and camera attachment without claiming to reproduce original sky geometry. The dome follows the camera, remains inside the far plane, disables depth writes, and is drawn before ordinary world geometry.

Listing 32.1. Clearly labeled fallback sky used to test the environment pipeline.

```
void drawReconstructionSky(IDirect3DDevice8& d, const SkyMesh& dome,
                           const Camera& camera, float farPlane) {
    const Mat4 world = translation(camera.eye) * scale(farPlane * 0.9f);
    setWorld(d, world);
    d.SetRenderState(D3DRS_LIGHTING, FALSE);
    d.SetRenderState(D3DRS_FOGENABLE, FALSE);
    d.SetRenderState(D3DRS_ZWRITEENABLE, FALSE);
    d.SetRenderState(D3DRS_ALPHABLENDENABLE, FALSE);
    drawVertexColoredMesh(d, dome);
    d.SetRenderState(D3DRS_ZWRITEENABLE, TRUE);
}
```

Speculation label. The generated dome topology and radius are reconstruction choices. Only decoded environment colors and any independently observed ordering should be entered as game-specific evidence.

33. Fog, Distance Haze, and Color Grading

Distance effects shape how a renderer transitions from nearby detail to the horizon. Fog can alter fragments according to depth-related or distance-related values; haze can describe the visual result of fog plus atmosphere and display behavior; color grading is a broader term for systematic color changes applied through materials, targets, or presentation. These labels overlap visually but describe different technical questions.

Scope. This chapter is a method for separating distance and color phenomena. It does not confirm Final Fantasy XI’s fog equation, parameters, weather controls, or presentation pipeline.

Implementation checkpoint. Parse supported fog parameters and expose distance haze and scene-color controls for runtime inspection.

```
// Cumulative implementation excerpt: parse and expose linear fog state.
FogState readFog(ByteReader& r) {
    FogState f;
    f.color = decodePackedRgb(r.u32());
    f.farDistance = r.f32();
    f.nearDistance = r.f32();
    f.enabled = f.farDistance != 0.0f;
    return f;
}

void applyFog(IDirect3DDevice8& d, const FogState& f) {
    d.SetRenderState(D3DRS_FOGENABLE, f.enabled);
    if (!f.enabled) return;
    d.SetRenderState(D3DRS_FOGTABLEMODE, D3DFOG_LINEAR);
    d.SetRenderState(D3DRS_FOGCOLOR, toD3dColor(f.color));
```

```
    d.SetRenderState(D3DRS_FOGSTART, std::bit_cast<DWORD>(f.nearDistance));
    d.SetRenderState(D3DRS_FOGEND, std::bit_cast<DWORD>(f.farDistance));
}
```

33.1 Fog as a draw-state question

In a fixed-function context, fog settings can be active for a draw and combine a fog color with the draw's computed fragment result. The mode and parameters influence whether the transition appears linear, exponential-like, or otherwise shaped. Fog can be enabled for one group of draws and disabled for another, so a world image may contain both fogged and unfogged elements.

33.2 Haze is an observation

Haze describes reduced contrast, shifted color, and loss of detail at distance. It may be caused by API fog, blended overlays, texture choices, sky coordination, light-like changes, display scaling, or several mechanisms together. Calling a distant fade 'fog' is often a useful shorthand, but a technical claim should identify the state or composition evidence that supports it.

33.3 Color changes across a scene

A scene-wide color shift can arise from changing fog color, clear color, vertex colors, material parameters, texture selection, a render-target pass, or final display conversion. The scope of the change is a valuable clue. If interface elements change with the world, a late presentation effect becomes more plausible; if only selected world draws change, material or fog state becomes more plausible. Neither inference is conclusive without trace evidence.

33.4 Controlled distance studies

Choose landmarks at several measured distances and capture them under fixed camera direction, time, weather, and display conditions. Compare color, contrast, alpha-like fading, draw presence, and depth behavior. Then vary one condition at a time. A measured series can establish a distance relationship even before it reveals the equation or source of the effect.

33.5 Evidence record

Record fog enablement and mode when available, fog color and parameters, depth state, material and texture stages, target sequence, background color, affected draw classes, and a controlled image series. Note whether the effect changes with world distance, camera rotation, time, weather, or platform. This makes it possible to distinguish a confirmed state change from an inferred atmospheric role.

> Evidence standard. A documented fog state is confirmed for its draw. 'Haze,' 'grading,' and 'atmosphere' are interpretations that should be tied to measured visual and state evidence.

34. Water, Reflections, and Surface Motion

Water in FFXI zones is best modeled as authored content using ordinary rendering machinery, not as a single dedicated water subsystem. Some surfaces are placed zone geometry; others are activated by ambient generators. Their motion can come from UV changes, curves, layered geometry, and particle-like components.

> Evidence scope. Placed water geometry, ordinary transparent-pass state, and generator UV motion are supported. Screen-space reflection, refraction, and a dedicated water shader have not been demonstrated.

34.1 Two content paths

The first path is ordinary zone geometry referenced by placement data and drawn in a transparent material group. The second is generator-activated geometry or particles, including otherwise unplaced ocean, waterfall, fountain, spray, and ambient layers. A client needs both paths before judging a zone incomplete.

34.2 Surface motion

Constant U and V increments in the generator command stream can scroll texture coordinates every update. Separate curve references can animate U and V against normalized particle age. Distinguish these from vertex deformation: UV motion changes sampling while leaving the surface geometry fixed.

34.3 Transparency and depth

Transparent zone groups use source-alpha/inverse-source-alpha blending, disable depth writes, retain a less-or-equal depth comparison, and use depth bias to preserve coplanar overlays. Vertex alpha frequently supplies shoreline feathering. If a loader already maps the half-range authoring convention so 0x80 becomes 1.0, it must not apply the fixed-function alpha compensation a second time.

34.4 Reflection-like appearance

Sky-colored textures, vertex colors, layered alpha, and time-of-day lighting can look reflective without sampling a reflected scene. Call the result reflection-like until a render target or environment-map dependency is observed. The confirmed character cubemap path does not automatically imply that water uses it.

34.5 Shorelines and non-rendering water data

Collision terrain codes can identify shallow or deep water for footsteps and splash selection, but they do not draw the surface. Likewise, path-like data associated with rivers or coastlines can serve movement or positional-audio behavior without being renderable water geometry.

34.6 C++ integration

```
void drawWaterAndOverlays(RenderContext& rc, const Scene& scene) {
    rc.setDepth(Compare::Less, true);
    drawOpaqueAndCutout(rc, scene);

    rc.setBlend(Blend::SrcAlpha, Blend::InvSrcAlpha);
    rc.setDepth(Compare::LessEqual, false);
    rc.setDepthBias(scene.zoneOverlayBias);
    drawTransparentZoneGroups(rc, scene);

    rc.disableBlend();
    rc.setDepth(Compare::Less, true);
    rc.setDepthBias(0);
}
```

Evidence standard. Geometry placement, generator references, UV commands, and render state can be confirmed independently. Do not collapse them into an unsupported claim of a proprietary water renderer.

34.7 Reconstruction laboratory: water without a water subsystem

A practical client should first render placed translucent geometry through the ordinary zone pipeline. Texture-coordinate motion can then be enabled as a per-material reconstruction policy, never by treating every object whose name resembles water as proof of a distinct format. The state packet in Listing 16.1 supplies the

experiment: blending on, depth writes off, culling chosen explicitly, and scroll isolated from the decoded material record.

Acceptance test. Freeze the camera and compare four captures: opaque/no motion, blended/no motion, blended/motion, and blended/motion with reversed draw order. Record which change explains each visible difference. Reflection, refraction, cubemap response, and screen-space effects remain unsupported until independent evidence establishes them.

35. Weather: Rain, Snow, Wind, and Ambient Particles

Weather is a coordination of three layers: an active weather identity, environment appearance selected for that identity and time, and optional visual generators such as precipitation. Keeping the layers separate prevents a server field, a fog record, and a particle emitter from being mistaken for one structure.

> Evidence scope. Weather identity and environment records are better established than the complete precipitation runtime. The latter remains a mixture of decoded fields, controlled experiments, and reconstruction.

35.1 Weather identity and transition

The active identifier chooses a four-character weather tag used to resolve environment records. Transition timing belongs to weather state; it is not particle lifetime. A zone can accept a weather identity yet omit the generator resources required for visible rain or snow.

35.2 Environment appearance

For the active weather, interpolate the surrounding hourly type-0x2F records. Update clear color, dome rings, fog, draw distance, and lighting inputs. These changes are atmospheric even when no precipitation geometry is present.

35.3 Precipitation generators

Precipitation is associated with the type-0x05 generator family and a specialized batched path. Decoded fields include emission cadence, count, bounds or position variation, per-particle lifetime, movement, color, scale, and curve references. The exact retail batching and orientation path is not yet complete enough to call byte-faithful.

35.4 Wind across systems

Wind can affect precipitation direction and the second-position blend used by vegetation. Agreement in timing does not prove a single stored wind field. Keep particle velocity, global vegetation blend, texture motion, and sound cues as separate inputs until their data flow is traced.

35.5 Camera-local fallback

A clean-room client may first implement precipitation as a camera-local emitter keyed by the active weather identity. This is a clearly labeled reconstruction: it provides stable coverage around the camera without claiming to reproduce the retail batched transform path.

35.6 Verification

Use repeated captures in one zone while holding camera, time, and weather constant. Record generator identity, emission interval, live particle count, lifetime distribution, bounds, blend/depth state, and environment values. Then test a zone with the same weather but different resources to isolate activation rules.

Implementation checkpoint. Implement WeatherState, EnvironmentState, and PrecipitationRuntime as separate objects. Connect them through weather tags and decoded generator references, not shared global guesses.

```
// Cumulative implementation excerpt: three weather layers connected by explicit tags.
struct WeatherState { WeatherTag active; float transition = 1.0f; };
struct PrecipitationRuntime { void update(WeatherTag, float dt); void draw(RenderContext&); };

class WeatherSystem {
    WeatherState weather_;
    EnvironmentDatabase environments_;
    PrecipitationRuntime precipitation_;
public:
    EnvironmentState update(GameMinute minute, float dt) {
        precipitation_.update(weather_.active, dt);
        return environments_.sample(weather_.active, minute, weather_.transition);
    }
    void drawParticles(RenderContext& rc) { precipitation_.draw(rc); }
};
```

Evidence standard. Weather-correlated particles are observation; a generator identity plus decoded and behaviorally tested fields supports an emitter model. Exact batching, gating, and pass ownership remain open.

35.7 Reconstruction laboratory: deterministic camera-local precipitation

Camera-local precipitation is a useful fallback when the generator format is not yet decoded. It should be deterministic, bounded, and visibly labeled as a substitute. Listing 35.1 derives repeatable positions from a seed and wraps particles through a camera-centered volume; it does not claim that the retail client uses this distribution.

Listing 35.1. Deterministic fallback precipitation for pipeline testing.

```
struct ReconstructionPrecipitation {
    std::uint32_t seed;
    std::vector<Vec3> particles;

    void rebuild(std::size_t count, const Camera& camera, float extent) {
        DeterministicRandom random(seed);
        particles.resize(count);
        for (Vec3& p : particles)
            p = camera.eye + Vec3{random.signedUnit() * extent,
                                  -60.0f + random.unit() * 85.0f,
                                  random.signedUnit() * extent};
    }
    void update(float dt, const Camera& camera, float speed) {
        for (Vec3& p : particles) {
            p.y += speed * dt;
            if (p.y > camera.eye.y + 25.0f) p.y -= 85.0f;
        }
    }
};
```

Keep weather identity, environment color transitions, wind, generator activation, and fallback precipitation as separate records. Replacing all five with one weather enum destroys the evidence needed to improve the client later.

36. Environment Control and Comparison Workspace

Sky, fog, water, and weather become useful research subjects when controls are synchronized and reproducible. This workspace exposes decoded environment values, reconstruction policies, time and weather transitions, and side-by-side or difference comparisons.

Build outcome.

Build an environment workspace that edits, compares, captures, and reports decoded state separately from reconstruction policy.

36.1 Responsibilities and ownership

EnvironmentState contains decoded inputs; EnvironmentPolicy contains explicitly chosen reconstruction behavior; EnvironmentRuntime contains interpolated values. ComparisonPreset records camera, time, weather, policy, and display settings.

36.2 Implementation sequence

Controls edit a pending state, validation produces a runtime snapshot, and both comparison panes consume the same scene snapshot with separately versioned environment policies.

36.3 Failure and recovery

Unknown source values remain visible and unchanged. Unsupported weather or water behavior disables only that control. A manual approximation is labeled reconstruction in captures and reports.

36.4 Verification gate

Tests reproduce presets, compare interpolation endpoints, freeze deterministic precipitation, verify identical cameras, and ensure policy labels enter exported reports.

36.5 Cumulative C++ implementation

Implementation checkpoint. Build an environment workspace that edits, compares, captures, and reports decoded state separately from reconstruction policy.

```
// Cumulative implementation listing
EnvironmentSnapshot EnvironmentController::commit() const {
    EnvironmentSnapshot out;
    out.source = pending_.source;
    out.policy = pending_.policy;
    out.time = wrapHours(pending_.time);
    out.weather = catalog_.contains(pending_.weather)
                  ? pending_.weather : WeatherId::Clear;
    out.fog = interpolateFog(pending_.source, out.time, out.weather);
    out.sky = interpolateSky(pending_.source, out.time, out.weather);
    out.label = out.policy.isReconstruction
                ? "reconstruction" : "evidence-scoped";
    return out;
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

37. Particle Systems, Trails, and Spell Effects

FFXI effects are compact programs. A type-0x05 generator selects resources and applies commands for attachment, emission, motion, orientation, scale, color, curves, texture coordinates, rendering class, sound or light-like work, and child generators. Type-0x19 curves and several geometry families supply data consumed by those programs. Building a client therefore requires both a bounded parser and a small deterministic simulation runtime.

> Evidence scope. Command framing, many payload sizes, resource relationships, curve bindings, UV scrolling, and several behavioral meanings are supported by DAT inspection and controlled parameter edits. Opcode meaning depends on its section. Some commands, flag bits, attachment semantics, and operation order remain unresolved.

37.1 Resource family

The working effect graph includes type 0x05 generators, type 0x19 normalized curves, type 0x1F expanded triangle geometry, type 0x21 card or sprite geometry, type 0x25 morphing geometry, texture resources, optional sounds and lights, and references to child generators. A spell effect can schedule several generators rather than correspond to one mesh.

37.2 Generator framing

The common generator header is approximately 0x80 bytes. Four monotonically increasing section boundaries are stored beginning near +0x70. Commands use a four-byte prefix containing an opcode, payload length, and two reserved bytes, followed by the payload. Validate every section boundary and payload before interpretation; do not search forward for plausible opcodes after corruption.

```
struct EffectCommandView {
    std::uint8_t section;
    std::uint8_t opcode;
    std::span<const std::byte> payload;
    std::uint64_t sourceOffset;
};

std::vector<EffectCommandView> parseCommandSection(
    ByteReader& r, std::uint8_t section, std::size_t end) {
    std::vector<EffectCommandView> out;
    while (r.position() < end) {
        const auto commandOffset = r.position();
        const auto opcode = r.u8();
        const auto payloadSize = r.u8();
        const auto reserved = r.u16le();
        if (opcode == 0) break;
        if (reserved != 0) r.noteUnknown(commandOffset + 2, reserved);
        if (r.position() + payloadSize > end)
            throw ParseError("effect command crosses section boundary");
        out.push_back({section, opcode, r.bytes(payloadSize), commandOffset});
    }
    if (r.position() > end) throw ParseError("effect section overrun");
    return out;
}
```

37.3 Section-qualified opcodes

The numeric opcode alone is not a global instruction identity. For example, values used for scale-curve bindings in the principal parameter section can denote constant UV increments in another section. Dispatch on the pair {section, opcode}; log unsupported pairs with their raw payload and source offset.

```
struct EffectOpcode {
    std::uint8_t section;
    std::uint8_t code;
```

```
    friend bool operator==(EffectOpcode, EffectOpcode) = default;
};

void decode(const EffectCommandView& c, EffectProgram& out) {
    switch ((std::uint16_t(c.section) << 8) | c.opcode) {
    case 0x0202: decodeVelocity(c.payload, out); break;
    case 0x020B: decodeAngularVelocity(c.payload, out); break;
    case 0x022E: decodeUCurve(c.payload, out); break;
    case 0x0327: decodeConstantUScroll(c.payload, out); break;
    default: out.unknownCommands.push_back(copyRaw(c)); break;
    }
}
```

37.4 Initial resource and particle state

The initial-resource command carries a referenced asset, a local position, a resource-family selector, lifetime, and orientation-related values. Observed resource targets include effect triangles, cards, morph meshes, sound-like records, point-light-like records, and zone geometry. Decode the reference first, then require the referenced resource’s type to agree with the selector before constructing a drawable.

37.5 Attachment and orientation

Controlled edits associated attachment values with repeatable locations around an actor: waist, chest, head, hands, feet, wrists, and offsets in front of or behind the body. This strongly supports an actor-relative attachment table, plausibly connected to bones, but it does not yet establish a universal bone-index formula. Store the raw attachment value and map only tested entries.

Observed orientation behavior includes unconstrained geometry, full camera-facing billboards, axis-constrained billboards, movement-oriented cards, and screen-attached forms. Resolve orientation after attachment position and particle motion, then build the final local-to-world transform. Do not apply one billboard matrix to every effect card.

37.6 Emission cadence and lifetime

The generator’s interval field controls the delay between emissions. The supported working interpretation is frames-per-emission minus one, so stored zero emits every frame and stored one every two frames. An emission-count field controls how many particles are created per event. Particle lifetime is separate, and the scheduler’s emission window is separate again.

```
struct EmitterClock {
    std::uint32_t periodFrames = 1;
    std::uint32_t accumulator = 0;

    bool tick() {
        if (++accumulator < periodFrames) return false;
        accumulator = 0;
        return true;
    }
};

EmitterClock makeClock(std::uint16_t storedInterval) {
    return EmitterClock{std::uint32_t(storedInterval) + 1u, 0u};
}
```

37.7 Spawn shapes and motion

Documented commands include constant velocity, velocity variation, spherical emission, volume-like or target-directed emission, radial inward or outward speed, and speed variation. Initial rotation, rotation variation, angular

velocity, and angular variation form a parallel group. Treat randomization as a deterministic input by seeding the generator instance; reproducible seeds are essential for tests.

```
struct ParticleState {
    Vec3 position{};
    Vec3 velocity{};
    Vec3 euler{};
    Vec3 angularVelocity{};
    Vec3 scale{1, 1, 1};
    Vec3 scaleVelocity{};
    Color4 color{1, 1, 1, 1};
    Vec2 uvOffset{};
    float ageFrames = 0;
    float lifetimeFrames = 1;
};

void integrate(ParticleState& p) {
    p.position += p.velocity;
    p.euler += p.angularVelocity;
    p.scale += p.scaleVelocity;
    p.ageFrames += 1.0f;
}
```

37.8 Scale, color, and texture coordinates

The command set can provide initial scale, per-axis and uniform scale variation, scale velocity, initial color, color variation, and curve-bound scale or RGBA components. Constant U/V increments and U/V curves animate texture sampling independently of geometry. The historical record contains inconsistent red/blue labels, so packed color order must be established from raw bytes and a discriminating red-versus-blue test.

37.9 Curves and morph controls

Position, rotation, scale, red, green, blue, alpha, U, and V can each reference normalized curves. Additional paired curve commands correlate with morphing geometry, but their exact semantics and playback order remain incomplete. Preserve the references even when the runtime cannot yet apply them.

37.10 Rendering class, depth, and order

Observed effect classes include additive, ordinary alpha, reverse-subtract or darkening, zero/inverse-source-like, and opaque composition. Separate flags can affect depth testing, depth writing, camera/world space, and freeze behavior. Historical experiments found the broad blend categories but not a reliable final bit layout; use the newer pass evidence where available and keep unverified bits visible in diagnostics.

Particles with soft gradients must not automatically inherit the alpha-test threshold used by every other effect-like draw. Blend, alpha test, depth test, depth write, culling, and sorting are separate fields in the runtime descriptor. Group draws only when all required state and ordering constraints match.

37.11 Child generators and lifecycle

Several commands reference another generator. Controlled experiments indicate that the child inherits position and can be affected by parent scale and rotation. The trigger may occur at creation, parent expiration, particle completion, or another event depending on the command. Represent the relationship as an event edge rather than recursively spawning every reference immediately.

```
enum class EffectEvent { OnStart, OnEmit, OnParticleDeath, OnDrain, OnStop };

struct ChildEdge {
    EffectEvent event;
```

```
    ResourceId child;
    TransformInheritance inheritance;
};

void dispatchEvent(EffectRuntime& rt, EffectInstance& parent, EffectEvent e) {
    for (const ChildEdge& edge : parent.program.childEdges)
        if (edge.event == e) rt.spawnChild(parent, edge);
}
```

37.12 A deterministic particle pool

Use immutable decoded programs and mutable instances. Each instance owns clocks, deterministic random state, live particle handles, and child-event state. A central pool prevents unbounded allocation and makes the maximum live-particle count inspectable. Update in a fixed order: scheduler, emission, particle integration, curve evaluation, death events, then draw-list construction.

37.13 Regression corpus

A large archived usage table maps many spell resources to the command values observed in their generator sections. It is valuable as a test corpus, not as proof of semantics. Choose minimal effects containing one candidate command, related tiers sharing most commands, and rare effects exercising unusual commands. For every fixture, assert zero section overruns, stable unknown-command preservation, deterministic particle counts, and bounded dependencies.

```
TEST(EffectParser, PreservesUnknownCommands) {
    const auto bytes = loadFixture("effect_minimal.dat");
    const EffectProgram p = parseEffect(bytes);
    EXPECT_TRUE(p.sectionsAreInBounds);
    EXPECT_EQ(hashRawUnknowns(p), expectedUnknownHash);
}

TEST(EffectRuntime, IsDeterministic) {
    auto a = runFixture("effect_minimal.dat", 600, 0x12345678);
    auto b = runFixture("effect_minimal.dat", 600, 0x12345678);
    EXPECT_EQ(a.frameHashes, b.frameHashes);
}
```

37.14 What remains open

Open work includes the exact attachment-to-bone mapping, the complete billboard field, all blend/depth flag bits, several spawn shapes, morph-curve semantics, sound and point-light parameters, child trigger distinctions, generator-selected card groups, batched precipitation, and the original update order. Unsupported commands must remain data, not disappear during parsing.

> Implementation checkpoint. Finish this chapter with a viewer that can pause at any simulation frame and display generator source offsets, decoded commands, unknown payloads, live particles, curve samples, child events, and the exact render descriptor for each draw.

```
// Cumulative implementation excerpt: pauseable, inspectable effect simulation.
class EffectInspector {
    EffectRuntime runtime_;
    bool paused_ = false;
public:
    void setPaused(bool value) { paused_ = value; }
    void step() { runtime_.simulateOneFrame(); }
    void update() { if (!paused_) step(); }
    InspectorSnapshot snapshot() const {
        return {runtime_.frame(), runtime_.decodedCommands(),
                runtime_.unknownCommands(), runtime_.particles(),
                runtime_.curveSamples(), runtime_.childEvents(),
                runtime_.drawDescriptors()};
```

```
    }
};
```

Publication rule. Describe a command as confirmed only when its bytes, section, payload, and behavior agree under a controlled test or original-client trace. Use corroborated for repeated structural agreement, and speculation for names inferred only from visual resemblance.

38. Effect Browser and Simulation Workspace

The effect workspace turns sectioned command streams into an inspectable simulation. It combines command ledger, dependency graph, deterministic seed, transport controls, live-particle table, curve plots, render descriptors, and unknown operations.

Build outcome.

Build a deterministic effect workspace with command, simulation, rendering, and evidence views driven by one timeline.

38.1 Responsibilities and ownership

EffectDocument is immutable parsed data; EffectInstance owns simulation state; EffectPlayback owns time and seed; EffectTrace records spawn, update, child, and draw events with source command IDs.

38.2 Implementation sequence

Opening builds the ledger and dependencies before simulation. Seeking resets to a checkpoint and deterministically replays. Pausing freezes emission and updates; stepping advances exactly one configured tick.

38.3 Failure and recovery

Unknown commands remain in order and can block only the behavior they control. Pool exhaustion, invalid curves, recursive child cycles, and missing resources produce trace events rather than silent disappearance.

38.4 Verification gate

Tests assert deterministic particle states, seek/replay equivalence, child-cycle limits, sorting keys, unknown-command preservation, and time-indexed screenshot hashes.

38.5 Cumulative C++ implementation

Implementation checkpoint. Build a deterministic effect workspace with command, simulation, rendering, and evidence views driven by one timeline.

```
// Cumulative implementation listing
void EffectWorkspace::seek(double target) {
    const SimulationCheckpoint& cp = checkpoints_.nearestAtOrBefore(target);
    instance_.restore(cp.state);
    trace_.truncateAfter(cp.time);
    double t = cp.time;
    while (t + tick_ <= target) {
        instance_.step(tick_, trace_);
        t += tick_;
    }
    if (t < target) instance_.step(target - t, trace_);
    playback_.setTime(target);
```

```
    refreshViews(instance_, trace_);
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

39. Transparency, Sorting, and Multi-Pass Effects

Transparency is not only a blend setting. It is an ordering problem across geometry, materials, targets, and passes. A renderer must decide which opaque results form the destination, which transparent draws can be grouped, which must be sorted, and when a later pass should reuse or replace an earlier result. Multi-pass effects make these decisions visible through layering, accumulation, and order-sensitive artifacts.

Scope. This chapter describes composition problems and evidence methods. It does not confirm Final Fantasy XI's transparent-pass ordering, sorting algorithm, or render-target workflow.

Implementation checkpoint. Translate supported pass records into transparent queues, target changes, sorting policies, and multi-pass composition.

```
// Cumulative implementation excerpt: decoded pass records drive targets and queues.
void executePasses(RenderGraph& graph, RenderContext& rc) {
    for (const PassRecord& pass : graph.passes()) {
        rc.setRenderTarget(graph.target(pass.target));
        if (pass.clear) rc.clear(pass.clearColor);
        auto draws = graph.draws(pass.drawRange);
        if (pass.sort == SortPolicy::BackToFront)
            std::stable_sort(draws.begin(), draws.end(), fartherFirst);
        rc.setDepth(pass.depthCompare, pass.depthWrite);
        rc.setBlend(pass.sourceBlend, pass.destinationBlend);
        for (const Draw& draw : draws) rc.draw(draw);
    }
}
```

39.1 Why sorting is difficult

Blended geometry needs a meaningful destination color, but two translucent objects can overlap in different depth relationships across their surfaces. Sorting by object center, by draw group, by particle, or not at all can produce different approximations. Some effects tolerate these approximations; others avoid them through special geometry, depth rules, or multiple passes. A visual artifact can reveal an ordering constraint, but it does not identify the chosen algorithm by itself.

39.2 Opaque foundation and transparent phases

A common conceptual structure is an opaque foundation that writes reliable depth, followed by cutout and blended phases with different depth-write behavior. The actual frame can include several interruptions: sky, water-like surfaces, particles, character components, interface layers, and off-screen targets may have their own ordering needs. Treat phase names as hypotheses until trace evidence shows the transition.

39.3 Multi-pass composition

One visible effect can be assembled through several passes over the same geometry or through different layers. A base material may be followed by a glow, a mask, a distortion-like layer, or a depth-aware overlay. A pass may write to the main target or to an intermediate surface. Confirming a multi-pass effect requires evidence of repeated geometry or linked target flow, not simply a complex-looking texture.

39.4 Diagnosing artifacts

Missing translucent regions, incorrect overlap, bright accumulation, halo edges, and self-intersection can be caused by blend factors, depth writes, culling, target contents, sorting, or capture timing. Diagnose by holding the camera and effect state stable while comparing adjacent draws and their states. Name the visible symptom first; then test the smallest set of mechanisms that can explain it.

39.5 Pass ledger

For each suspected pass, record target, clear state, draw order, geometry range, textures, stages or shader, blend and depth state, culling, and the image contribution that changes after it. Link passes only when resource flow, repeated geometry, or controlled visual dependence supports the relationship. This ledger prevents a descriptive effect name from becoming a fabricated render graph.

> Evidence standard. Ordered trace events and target bindings can confirm a pass sequence. Calling the sequence a transparency sorter, post-process, or named effect pipeline requires additional implementation or behavioral evidence.

40. Shadows, Decals, and Projected Visuals

Shadows, marks on surfaces, and projected patterns often look simple but can arise from very different techniques. A dark region may be a textured quad, a projected texture, a separate mesh, vertex color, a blended overlay, a render-target result, or a lighting response. A decal-like mark may follow geometry, float slightly above it, be baked into a texture, or appear only from a presentation layer.

> Scope. This chapter is a classification and testing guide. It does not confirm Final Fantasy XI's shadow method, decal system, or projection implementation.

> **Implementation checkpoint.** Decode supported projected-visual records behind interchangeable shadow, decal, and projector interfaces.

```
// Cumulative implementation excerpt: interchangeable projected-visual decoders.
class ProjectedVisual {
public:
    virtual ~ProjectedVisual() = default;
    virtual void submit(const Scene&, DrawList&) const = 0;
};
class BlobShadow final : public ProjectedVisual {
    ShadowRecord record_;
public:
    explicit BlobShadow(ShadowRecord r) : record_(r) {}
    void submit(const Scene& scene, DrawList& out) const override {
        out.add(projectBlob(record_, scene.receivers(), DepthWrite::Off));
    }
};

std::unique_ptr<ProjectedVisual> decodeProjected(const RecordView& r) {
    if (r.kind() == RecordKind::Shadow) return std::make_unique<BlobShadow>(readShadow(r));
    if (r.kind() == RecordKind::Decal) return std::make_unique<Decal>(readDecal(r));
    throw UnsupportedRecord(r.kind());
}
```

40.1 Shadow-like results

A shadow-like effect is identified first by behavior: where it appears, whether it moves with an object, whether it conforms to nearby surfaces, whether it changes with light or time, and how it overlaps other content. These observations can distinguish a world-attached mark from a screen-space darkening, but they do not yet identify the geometry or calculation behind it.

40.2 Decals and offset geometry

A decal can be implemented as a texture projected onto a receiver, a small mesh placed near a surface, an additional material pass, or data baked into a base texture. Offset geometry may avoid depth conflict at the cost of visible separation from the receiver at some angles. Observe parallax, clipping, draw order, depth bias-like behavior, and whether the mark crosses material or object boundaries.

40.3 Projected coordinates

Projection maps an image using a transform related to a projector, light-like source, or camera. A changing projected pattern may depend on a source transform and receiver geometry. It can resemble an ordinary texture transform in a capture. To establish projection, record coordinate-generation state or shader behavior, the source transform, receiver draws, and the way the pattern responds to changes in source and receiver.

40.4 Target-based and composited effects

Some apparent marks or shadows can be produced by rendering to an intermediate surface and blending it later. In that case, the relationship between source scene data and final receiver may not be visible in one draw. Look for target changes, linked texture bindings, full-screen or receiver-like passes, and ordering relative to opaque geometry and interface layers.

40.5 Investigation checklist

Capture the effect under controlled changes in object position, receiver geometry, camera angle, time, and nearby occluders. Record all related draws, targets, depth and blend state, culling, textures, coordinate transforms, and whether the mark is present before or after the receiver draw. Describe the behavior first, then use the smallest confirmed chain to classify it.

> Evidence standard. A persistent visible mark is observation. Projected texture, decal mesh, blob shadow, or target-based shadow are implementation claims that require resource, geometry, state, and ordering evidence.

41. Interface Rendering and 2D Composition

An interface is a rendered layer with its own geometry, textures, ordering, coordinate system, and readability requirements. It may be drawn after world rendering, interleaved with presentation passes, or composed through a separate target. Its apparent flatness does not prove a particular implementation, but it makes UI elements valuable markers when reconstructing the end of a frame.

> Scope. This chapter provides a method for analyzing 2D composition. It does not confirm Final Fantasy XI's UI renderer, target layout, coordinate system, or update order.

> **Implementation checkpoint.** Parse supported interface records and build an orthographic renderer that composes evidence-scoped 2D layers.

```
// Cumulative implementation excerpt: decoded rectangles become orthographic draws.
void UiRenderer::draw(const UiDocument& ui, IDirect3DDevice8& device,
                      std::uint32_t width, std::uint32_t height) {
    const Mat4 projection = orthoOffCenterLH(0, float(width),
                                             float(height), 0, 0, 1);
    const D3DMATRIX d3dProjection = projection.d3d();
    device.SetTransform(D3DTS_PROJECTION, &d3dProjection);
    device.SetRenderState(D3DRS_ZENABLE, FALSE);
    for (const UiLayer& layer : ui.layersInOrder()) {
```

```
            setScissor(device, layer.clip);
            for (const UiSprite& s : layer.sprites)
                drawTexturedQuad(device, s.destination, s.uv, textureCache_.get(s.texture));
        }
    }
```

41.1 Screen space and world space

World-space draws are positioned through scene transforms and camera projection. Interface elements are commonly positioned in a screen-oriented coordinate system, where placement can follow pixels, normalized dimensions, anchors, or another layout rule. A marker that appears attached to an object may still be screen-composited after a world-to-screen calculation. Test camera, resolution, and object motion dependencies before assigning a space.

41.2 Quads, atlases, and layers

Many 2D interfaces can be assembled from textured quads, often using regions of a larger texture atlas. Panels, borders, icons, cursors, meters, and simple effects can share textures while using different source rectangles and draw order. Matching artwork is not proof of a shared atlas; resource dimensions, coordinates, and bindings provide the relevant evidence.

41.3 Ordering and depth

UI layers may disable or alter depth testing, use alpha blending, and rely on draw order for stacking. A background panel, text, highlight, cursor, and modal overlay can be several draws with distinct blend and texture state. Interface elements can also include world-aware parts such as nameplates or targeting indicators, which require careful separation from pure screen-space composition.

41.4 Resolution and layout behavior

Interface placement can respond to resolution, aspect ratio, safe areas, window mode, scaling, or localization. A capture at one display configuration does not define the layout rule. Compare the same state across documented configurations and record anchor movement, element dimensions, cropping, and texture sampling changes.

41.5 Investigation checklist

Record the frame position at which UI draws occur, target and depth state, transforms or coordinate setup, vertex layouts, texture bindings, blend factors, draw order, and behavior under controlled resolution and camera changes. Use world content as a reference only after confirming whether the interface element shares world-space dependencies.

> Evidence standard. A late ordered draw with screen-oriented geometry supports a UI-layer hypothesis. A specific layout engine, atlas system, or update order requires resource and implementation evidence.

42. Text, Icons, Fonts, and Window Layers

Text and icons make technical UI questions unusually visible: they reveal texture sampling, clipping, blending, layout, and ordering at a small scale. A window is often a composition of background, border, text, icons, cursor or highlight, and mask-like pieces. The player-facing label of one window therefore should not be confused with one renderable object.

Scope. This chapter is a rendering-oriented vocabulary for interface elements. It does not confirm Final Fantasy XI's font resource format, glyph cache, icon atlas, or window manager.

Implementation checkpoint. Decode supported font, glyph, icon, and window records, then implement text layout, atlases, clipping, and layering.

```
// Cumulative implementation excerpt: glyph metrics, atlas lookup, clipping, layering.
std::vector<GlyphQuad> layoutText(const Font& font, std::u32string_view text,
                                  Vec2 origin, Rect clip) {
    std::vector<GlyphQuad> out;
    Vec2 pen = origin;
    for (char32_t ch : text) {
        if (ch == U'\n') { pen.x = origin.x; pen.y += font.lineHeight; continue; }
        const Glyph& g = font.glyph(ch);
        Rect dst{pen.x + g.bearing.x, pen.y - g.bearing.y,
                 float(g.size.x), float(g.size.y)};
        if (intersects(dst, clip)) out.push_back({dst, g.atlasUv, g.atlasPage});
        pen.x += g.advance;
    }
    return out;
}
```

42.1 Glyphs as geometry and texture

Raster text is commonly rendered by placing textured quads for glyphs or groups of glyphs. A font resource can store individual images, an atlas, metrics, spacing data, and language-dependent mappings. The final appearance also depends on filtering, alpha treatment, color, clipping, and draw order. A visible font does not by itself identify the resource or layout mechanism.

42.2 Icons and atlas regions

Icons may use standalone textures or source rectangles within an atlas. Reusing an atlas can reduce binds and simplify UI batching, while different coordinates select different images. To confirm an atlas relationship, compare dimensions, bindings, texture coordinates, and neighboring icon draws; visual style alone is insufficient.

42.3 Window layering and clipping

A window can require background and border draws before content, then overlays for selection, focus, or modal state. Clipping may be implemented through scissor-like rectangles, geometry masks, texture alpha, or separate targets. A text line disappearing at a panel edge is useful evidence, but the mechanism requires state and geometry context.

42.4 Localization and layout

Different languages can change glyph selection, string length, line breaks, and window dimensions. These changes provide controlled evidence for layout behavior when the same UI state can be compared across editions. They do not automatically reveal whether text is precomposed, dynamically laid out, or cached; inspect resource and draw changes as well.

42.5 Investigation checklist

Record the UI state, language and display configuration, text or icon content, target and depth state, texture resources, coordinate ranges, vertex counts, filtering, blend state, clipping evidence, and draw order. Use short stable strings and repeated icons to make comparisons precise. The goal is to separate glyph data, layout decision, and final composition.

Evidence standard. A textured glyph or icon draw can be confirmed from its resources and geometry. Font format, cache policy, and window hierarchy require additional data or implementation evidence.

43. Text, Dialog, and Message Resource Explorer

A useful resource explorer must handle non-visual context. This chapter builds a bounded text-resource framework for multiple layouts, byte-preserving decoding, control-token display, search, record cross-reference, and round-trip-safe export.

Build outcome.

Build a layout-pluggable text explorer with token-aware decoding, search, provenance, and lossless diagnostic export.

43.1 Responsibilities and ownership

TextRecord stores identity, source range, raw bytes, decoded runs, controls, and decoding issues. Display strings are derived views and never replace the original byte sequence.

43.2 Implementation sequence

Recognition selects a layout decoder; record framing occurs before character decoding; a code-page service converts ordinary spans; game-specific controls become typed tokens; the browser indexes normalized display text and token names.

43.3 Failure and recovery

Invalid byte sequences produce replacement runs with exact offending ranges. Unknown controls remain hex tokens. Export includes identity, raw bytes, decoded text, tokens, and issues so no evidence is lost.

43.4 Verification gate

Fixtures cover empty records, malformed offsets, mixed text and controls, private glyph fallback, duplicate strings, search normalization, and export/reimport identity.

43.5 Cumulative C++ implementation

Implementation checkpoint. Build a layout-pluggable text explorer with token-aware decoding, search, provenance, and lossless diagnostic export.

```
// Cumulative implementation listing
TextRecord decodeTextRecord(ByteView bytes, SourceRef source,
                            const TextCodec& codec) {
    TextRecord out{source};
    std::size_t begin = 0;
    for (std::size_t i = 0; i < bytes.size();) {
        if (!isControlLead(bytes[i])) { ++i; continue; }
        if (begin < i) out.runs.push_back(codec.decode(bytes.subview(begin, i-begin),
                                                       source.subrange(begin, i-begin)));
        ControlParse c = parseControl(bytes.subview(i), source.offset + i);
        out.runs.push_back(c.run);
        i += c.consumed;
        begin = i;
    }
    if (begin < bytes.size())
        out.runs.push_back(codec.decode(bytes.subview(begin), source.subrange(begin)));
```

```
    return out;
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

44. Audio Catalog, Decode, and Playback

Audio resources require the same evidence discipline as graphics. The application catalogs supported containers, parses headers, decodes PCM or supported ADPCM into owned samples, streams playback without blocking the UI, and exposes unsupported codecs without pretending to decode them.

Build outcome.

Build a searchable audio workspace with safe header parsing, asynchronous decode, playback transport, looping, and unsupported-codec reporting.

44.1 Responsibilities and ownership

AudioDocument contains container metadata and provenance; DecodedAudio owns normalized sample frames; PlaybackVoice owns device state; AudioCatalog stores searchable identity without pre-decoding every payload.

44.2 Implementation sequence

The catalog reads headers. Opening schedules decode, fills a bounded ring buffer, and posts progress. Playback consumes immutable blocks; seek cancels pending blocks and restarts from a decoder checkpoint.

44.3 Failure and recovery

Unsupported codecs show metadata with playback disabled. Invalid rates, channel counts, block sizes, or truncated payloads stop before device submission. Temporary decoded data has explicit lifetime and size limits.

44.4 Verification gate

Tests cover header variants, PCM identity, ADPCM reference vectors, seek, loop boundaries, cancellation, device changes, and temporary-cache eviction.

44.5 Cumulative C++ implementation

Implementation checkpoint. Build a searchable audio workspace with safe header parsing, asynchronous decode, playback transport, looping, and unsupported-codec reporting.

```
// Cumulative implementation listing
void AudioWorkspace::play(ResourceId id) {
    stop();
    document_ = audio_.readHeader(id);
    if (!audio_.supports(document_->codec)) {
        status_ = "preview unavailable for this codec";
        return;
    }
    decodeJob_ = jobs_.start([this, doc = *document_](StopToken stop) {
        auto decoder = audio_.createDecoder(doc);
        while (!stop.stopRequested()) {
            SampleBlock block = decoder->next(4096);
            if (block.empty()) break;
            if (!queue_.push(std::move(block), stop)) break;
        }
```

```
        queue_.finish();
    });
    voice_.start(queue_, document_->sampleRate, document_->channels);
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

Part V — Desktop Application, Presentation, and Performance

45. Resolution, Aspect Ratio, and Display Scaling

The image produced by a renderer is eventually mapped to a display surface with a particular size and aspect relationship. Resolution, aspect ratio, viewport, letterboxing or cropping, scaling, filtering, and color conversion can change the final appearance without changing world geometry or materials. Technical comparisons must therefore describe the presentation path as carefully as the scene.

> Scope. This chapter provides display-analysis terminology. It does not confirm Final Fantasy XI's internal resolutions, viewport policy, scaling algorithm, or aspect-ratio behavior.

> **Implementation checkpoint.** Separate decoded presentation settings, internal rendering dimensions, output resolution, viewport, aspect ratio, and UI scaling.

```
// Cumulative implementation excerpt: render size, output size, viewport, and UI scale.
struct PresentationState {
    Extent2D renderSize, outputSize;
    Rect viewport;
    float uiScale = 1.0f;
};

PresentationState makePresentation(Extent2D internal, Extent2D output) {
    const float scale = std::min(float(output.w) / internal.w,
                                 float(output.h) / internal.h);
    const Extent2D fitted{std::uint32_t(internal.w * scale),
                          std::uint32_t(internal.h * scale)};
    Rect vp{(output.w - fitted.w) / 2.0f, (output.h - fitted.h) / 2.0f,
            float(fitted.w), float(fitted.h)};
    return {internal, output, vp, scale};
}
```

45.1 Render size and presentation size

An internal render target can differ from the window or display size to which it is presented. Scaling can enlarge, reduce, stretch, or preserve the source image with borders. The final image may also be cropped or placed in a viewport. A captured pixel dimension alone does not identify the internal render size; target descriptors and presentation parameters are stronger evidence.

45.2 Aspect ratio and projection

Aspect ratio affects both the shape of a presentation surface and, potentially, the projection used for world rendering. A wider display can show more horizontal view, stretch an existing view, crop it, or use another policy. UI anchoring can behave independently. Compare stable landmarks and interface elements across documented configurations before attributing a difference to camera projection or layout.

45.3 Scaling and filtering

Scaling methods affect sharpness, texture detail, thin lines, text, and aliasing. Point-like scaling preserves discrete source pixels; filtered scaling mixes neighboring samples; additional display processing can add further change. These effects can imitate texture-resolution differences or alter the apparent threshold of alpha-tested edges. Preserve unscaled captures where possible and record any conversion path.

45.4 Safe areas, borders, and cropping

Some presentations reserve margins or use borders so that critical UI remains visible on an expected display. Window decorations, capture crops, and overlay software can add their own boundaries. A missing UI element or shifted edge should first be tested against viewport and crop conditions before it becomes evidence of a different interface layout.

45.5 Comparison protocol

For every image comparison, record target size when known, output size, aspect ratio, window/full-screen state, viewport or border behavior, scaling mode, capture method, and any subsequent resampling. Use identical camera and UI state where possible. This protocol turns display differences into explicit variables rather than hidden noise in renderer research.

> Evidence standard. A documented target or presentation parameter is confirmed. The internal reason for a visible stretch, crop, or sharpness change remains an inference until the relevant target and presentation path are observed.

46. Desktop Shell, Docking, and Workspace Lifecycle

The desktop shell turns independent viewers into one coherent tool. It owns the main window, menu and command surfaces, docked browser and diagnostics panes, document tabs, focus routing, persistence boundaries, and orderly workspace closure.

> Build outcome.

Build the multi-document shell and workspace factory/lifecycle contract used by every explorer and viewer.

46.1 Responsibilities and ownership

IWorkspace exposes identity, title, dirty state, command targets, render/update hooks, and close negotiation. The shell owns workspace objects; panes subscribe through weak or tokenized connections.

46.2 Implementation sequence

Opening a resource asks the factory registry for the best workspace, reuses an existing compatible document when policy permits, activates its tab, and binds shared inspector panes to its selection context.

46.3 Failure and recovery

A workspace may veto closure to finish or discard an export. Exceptions in one workspace are converted to a failure panel. Destroyed workspaces revoke subscriptions before releasing documents and GPU views.

46.4 Verification gate

Tests cover duplicate-open policy, tab activation, command focus, pane rebinding, close cancellation, exception isolation, layout restoration, and shutdown order.

46.5 Cumulative C++ implementation

> Implementation checkpoint. Build the multi-document shell and workspace factory/lifecycle contract used by every explorer and viewer.

```
// Cumulative implementation listing
WorkspaceId WorkspaceManager::open(const OpenRequest& request) {
    if (auto existing = findReusable(request)) {
        activate(*existing);
        return *existing;
    }
    const WorkspaceFactory& factory = factories_.bestMatch(request);
    std::unique_ptr<IWorkspace> workspace = factory.create(services_, request);
    const WorkspaceId id = ids_.next();
    workspaces_.emplace(id, std::move(workspace));
    activate(id);
    return id;
}

bool WorkspaceManager::close(WorkspaceId id) {
    IWorkspace& w = *workspaces_.at(id);
    if (!w.prepareToClose()) return false;
    subscriptions_.revokeOwner(id);
    workspaces_.erase(id);
    activate(nextWorkspace());
    return true;
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

47. Commands, Menus, Settings, and Tool Windows

Commands should describe intent independently of menu IDs, shortcuts, or buttons. This chapter builds typed command routing, enable/check state, settings validation, tool-window ownership, and delayed application of changes requiring resource or device recreation.

Build outcome.

Implement typed commands and versioned settings that drive menus, shortcuts, dialogs, and tool windows through one policy.

47.1 Responsibilities and ownership

CommandId is stable application vocabulary; CommandContext describes the active workspace and selection; CommandRouter queries and invokes targets; Settings uses typed sections and versioned serialization.

47.2 Implementation sequence

UI surfaces query command state immediately before display. Invocation flows from focused control to workspace to shell. Settings edits enter a draft, validate as a whole, then apply immediate, reload-required, and restart-required differences separately.

47.3 Failure and recovery

Unknown persisted settings survive load/save when practical. Invalid values fall back with diagnostics. Numeric menu identifiers never cross into application logic.

47.4 Verification gate

Tests verify command precedence, disabled invocation, check state, shortcut equivalence, settings migration, invalid-value fallback, tool-window singleton policy, and device-reset batching.

47.5 Cumulative C++ implementation

Implementation checkpoint. Implement typed commands and versioned settings that drive menus, shortcuts, dialogs, and tool windows through one policy.

```
// Cumulative implementation listing
bool CommandRouter::invoke(CommandId id, const CommandContext& c) {
    for (ICommandTarget* target : targetsFor(c)) {
        if (target->query(id, c).enabled) {
            target->execute(id, c);
            return true;
        }
    }
    return false;
}

ApplyReport SettingsController::commit(const SettingsDraft& draft) {
    ValidationReport valid = validate(draft.value);
    if (!valid.ok()) return {false, valid.issues, {}};
    SettingsDiff diff = compare(current_, draft.value);
    applyImmediate(diff);
    scheduleReloads(diff);
    current_ = draft.value;
    store_.saveVersioned(current_);
    return {true, {}, diff.restartRequired};
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

48. Input, Cameras, Picking, and Navigation Modes

Interactive inspection needs predictable input. This chapter separates raw events from actions, focus and capture from navigation, and orbit, free-flight, grounded, and object-focus camera modes from the scene data they observe.

Build outcome.

Build configurable input, four navigation modes, reliable pointer capture, and selection rays consistent with the rendered viewport.

48.1 Responsibilities and ownership

InputState records physical state; ActionMap resolves bindings; NavigationController consumes actions and elapsed time; CameraRig owns mode-specific state; Picker converts viewport coordinates into rays.

48.2 Implementation sequence

Window events update input first. Focus rules suppress navigation while text or numeric controls edit. Captured pointer deltas become camera actions. Selection clicks use the same camera matrices rendered for that frame.

48.3 Failure and recovery

Focus loss releases capture and clears transient state. Missing collision disables grounded mode explicitly. Extreme elapsed time is clamped and reported rather than causing a camera jump.

48.4 Verification gate

Tests verify bindings, modifier chords, focus suppression, capture release, frame-rate independence, camera-mode transitions, pick rays, and grounding fallback.

48.5 Cumulative C++ implementation

Implementation checkpoint. Build configurable input, four navigation modes, reliable pointer capture, and selection rays consistent with the rendered viewport.

```
// Cumulative implementation listing
void NavigationController::update(const InputState& input, double elapsed) {
    const double dt = std::clamp(elapsed, 0.0, 0.1);
    ActionState a = bindings_.resolve(input);
    if (!focus_.allowsNavigation()) a.clearMotion();
    switch (camera_.mode()) {
    case CameraMode::Orbit:    updateOrbit(camera_, a, dt); break;
    case CameraMode::Fly:      updateFly(camera_, a, dt); break;
    case CameraMode::Grounded: updateGrounded(camera_, a, dt, collision_); break;
    case CameraMode::Focus:    updateFocus(camera_, a, dt, selection_); break;
    }
}

Ray Picker::ray(Vec2 pixel, Viewport vp, const CameraMatrices& m) const {
    const Vec2 ndc{2.0f * (pixel.x - vp.x) / vp.width - 1.0f,
                   1.0f - 2.0f * (pixel.y - vp.y) / vp.height};
    return unprojectRay(ndc, inverse(m.viewProjection));
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

49. State Changes, Batching, and Resource Reuse

A renderer balances correct composition against the cost of changing state and submitting work. Textures, buffers, transforms, shaders, texture stages, blend rules, targets, and depth settings can all change between draws. Batching groups compatible work to reduce some of those changes, while resource reuse avoids recreating or reloading data unnecessarily. Neither concept is visible directly in a screenshot.

Scope. This chapter is a performance-analysis vocabulary. It does not confirm Final Fantasy XI’s batching policy, cache design, or exact cost model.

Implementation checkpoint. Add state caching, draw sorting, batching, and resource reuse without erasing source identity or required order.

```
// Cumulative implementation excerpt: cache state, but preserve required source order.
void Renderer::submit(std::span<const Draw> sourceOrdered) {
    std::vector<Draw> reorderable, fixed;
    for (const Draw& d : sourceOrdered)
        (d.requiresSourceOrder ? fixed : reorderable).push_back(d);
    std::stable_sort(reorderable.begin(), reorderable.end(),
        [](const Draw& a, const Draw& b) { return a.stateKey() < b.stateKey(); });
    for (const Draw& d : mergeWithoutCrossingBarriers(reorderable, fixed)) {
        stateCache_.applyOnlyChanges(d.state);
        resourceCache_.bind(d.resources);
        device_->DrawIndexedPrimitive(d.primitive, d.minIndex, d.vertexCount,
                                      d.startIndex, d.primitiveCount);
    }
}
```

49.1 State-change boundaries

A state change occurs when a draw requires a different active setting or binding from the previous draw. Some changes are inexpensive relative to others; some are necessary for correctness; some can be avoided by reordering compatible opaque work. In a trace, the important question is not simply how many calls occur, but which active values actually change and why a neighboring draw could or could not share them.

49.2 Batching

Batching can mean submitting several compatible objects together, combining geometry into fewer draws, grouping by material, or collecting work before a rendering phase. These strategies have different tradeoffs for visibility, transforms, transparency, animation, and memory. Repeated draws with the same state suggest an opportunity or an existing grouping; they do not by themselves reveal whether geometry was combined or which batching algorithm selected it.

49.3 Resource reuse

A texture, vertex buffer, index buffer, shader, or target can live across many frames and be bound repeatedly. Reuse is often observable as recurring identifiers and stable descriptors, while updates reveal changing content or parameters. Distinguish resource lifetime from draw frequency: a resource can be long-lived but rarely used, or updated frequently but reused across many draws.

49.4 Correctness constraints

Transparency, depth, effects, and interface layering can limit reordering even when a different order would reduce state changes. A renderer may deliberately accept extra binds or draws to preserve a visible result. Performance analysis should therefore pair every proposed optimization with its composition constraint rather than implying that fewer calls always means a better or more authentic path.

49.5 Trace analysis

Group a trace by target and scene boundary, then record changes in textures, streams, declarations, shaders, render state, and draw type. Compare each group with the visible scene and resource lifetime. The strongest conclusion identifies a concrete pattern—such as repeated binding of one texture across a run of compatible draws—without inventing a cache or batch system that has not been observed.

> Evidence standard. Repeated bindings and ordered state changes are confirmed trace facts. 'Batch,' 'cache,' and 'sort key' are implementation labels that require geometry, scheduling, or code evidence.

50. CPU–GPU Work Distribution

A frame is produced by cooperating processors with different responsibilities. CPU-side work can include scene updates, animation evaluation, visibility selection, resource management, state setup, and draw submission. GPU-side work can include vertex processing, primitive rasterization, texture sampling, blending, and target writes. Drivers and presentation add another layer of scheduling. The visible frame does not reveal this division by itself.

> Scope. This chapter explains performance-analysis categories. It does not measure Final Fantasy XI’s CPU/GPU balance or identify its threading and driver behavior.

Implementation checkpoint. Measure the full asset-to-frame path with parse/decode/upload timings and CPU-GPU frame telemetry.

```
// Cumulative implementation excerpt: scoped CPU timing plus GPU completion markers.
class ScopedTimer {
    Telemetry& t_; Stage stage_; Clock::time_point begin_ = Clock::now();
public:
    ScopedTimer(Telemetry& t, Stage s) : t_(t), stage_(s) {}
    ~ScopedTimer() { t_.record(stage_, Clock::now() - begin_); }
};

FrameMetrics renderMeasured(ClientApp& app) {
    { ScopedTimer t(app.telemetry, Stage::Parse);  app.assets.parseReady(); }
    { ScopedTimer t(app.telemetry, Stage::Upload); app.assets.uploadReady(); }
    app.gpuMarkers.beginFrame();
    { ScopedTimer t(app.telemetry, Stage::Submit); app.renderer.draw(app.scene); }
    app.gpuMarkers.endFrame();
    app.renderer.present();
    return app.telemetry.finishFrame(app.gpuMarkers.completedDuration());
}
```

50.1 Preparation and submission

Before geometry reaches the graphics pipeline, an application may update world state, resolve animation, choose visible work, construct transforms, lock or update resources, and set device state. Draw calls describe work but also consume CPU and driver time. A scene with modest pixel cost can still be limited by many small submissions or expensive preparation.

50.2 GPU-side cost categories

Vertex workload depends on submitted geometry and its processing path. Pixel workload depends on screen coverage, texture sampling, blending, depth rejection, targets, and resolution. A large translucent effect can be expensive because it shades many overlapping pixels even with little geometry. A distant complex mesh can be cheap if it covers few pixels or is culled early. Separate these categories before explaining a performance change.

50.3 Queues and synchronization

CPU submission and GPU execution need not occur at the same instant. Work can be queued, and synchronization points can force one side to wait for the other. Presentation behavior, resource locks, readbacks, and certain queries can affect this relationship. A frame-time observation is therefore not automatically attributable to the last visible draw or effect.

50.4 Measuring without overclaiming

Use repeatable scenes and vary one workload class at a time where possible: camera direction for visibility, resolution for pixel cost, effect density for blending, character count for animation and submission, or texture state diversity for bindings. Record frame-time measurements with configuration, capture method, and uncertainty. A correlation can identify a likely bottleneck class before it identifies an exact CPU or GPU function.

50.5 Evidence record

For a performance claim, record scene conditions, display configuration, draw and primitive counts, state-change counts, resource updates, target usage, timing method, repeated samples, and the variable changed. Distinguish measured timing from an explanation of cause. The latter needs profile, trace, or controlled evidence that rules out competing workload classes.

Evidence standard. A measured frame-time change under documented conditions is confirmed. Calling the change CPU-bound, GPU-bound, driver-bound, or synchronization-bound is an inference unless measurement evidence isolates the cause.

50.6 A renderer-independent parser probe

Structural regressions should be detected before pixels are involved. Listing 50.1 hashes the fixture, walks every bounded chunk, records type counts and failures, and compares the result with a checked-in expectation. The probe uses the same registry as the visual client but creates no window or graphics device.

Listing 50.1. Stable structural output for parser regression tests.

```
struct ProbeSummary {
    std::string fileHash;
    std::size_t chunkCount = 0;
    std::map<std::uint8_t, std::size_t> typeCounts;
    std::vector<FailureRecord> failures;
};

ProbeSummary probeResource(const std::filesystem::path& path,
                           const ChunkRegistry& registry) {
    const auto bytes = readWholeFileBounded(path, 256_MiB);
    ProbeSummary out{sha256(bytes)};
    EvidenceLog evidence;
    for (const ChunkView& chunk : walkChunkViews(bytes)) {
        ++out.chunkCount;
        ++out.typeCounts[chunk.info.type];
        try {
            const Recognition r = registry.inspect(chunk, evidence);
            if (r == Recognition::Invalid)
                out.failures.push_back({chunk.offset, "invalid"});
        } catch (const std::exception& e) {
            out.failures.push_back({chunk.offset, e.what()});
        }
    }
    return out;
}

void requireFixture(const ProbeSummary& actual, const Fixture& expected) {
    require(actual.fileHash == expected.fileHash);
    require(actual.chunkCount == expected.chunkCount);
    require(actual.typeCounts == expected.typeCounts);
    require(actual.failures == expected.failures);
}
```

A fixture expectation should contain only stable facts: input hash, selected range, chunk offsets and types, supported-record counts, dimensions, and named failures. Do not encode pointer values, unordered iteration order, timing, or an image-derived guess as structural truth.

51. Background Jobs, Cancellation, and UI Handoffs

Catalog scans, decoding, hashing, and export must not freeze the interface. This chapter implements bounded worker execution, cooperative cancellation, progress, generation checks, exception capture, and safe publication back to the UI thread.

Build outcome.

Build the reusable job system used by cataloguing, decode, streaming, audio, screenshots, and export.

51.1 Responsibilities and ownership

Job owns stop source, progress, result channel, and generation. JobQueue owns workers and bounds. UI dispatcher owns callbacks. Documents publish immutable results tagged with the request generation.

51.2 Implementation sequence

Submitting replaces or queues according to a typed key. Workers catch every exception, periodically check cancellation, and post completion. The UI accepts a result only if the document and generation still match.

51.3 Failure and recovery

Cancellation is not thread termination. Long decoders must expose checkpoints. Queue saturation applies backpressure. Shutdown stops admission, requests cancellation, drains UI completions, then joins workers.

51.4 Verification gate

Tests cover replacement races, cancellation latency, queue bounds, exceptions, stale results, progress monotonicity, shutdown under load, and deterministic single-worker mode.

51.5 Cumulative C++ implementation

Implementation checkpoint. Build the reusable job system used by cataloguing, decode, streaming, audio, screenshots, and export.

```
// Cumulative implementation listing
template<class Work, class Publish>
JobHandle JobQueue::replace(JobKey key, Work work, Publish publish) {
    cancel(key);
    const std::uint64_t generation = ++generations_[key];
    return enqueue(key, [=, work = std::move(work),
                        publish = std::move(publish)](StopToken stop) mutable {
        try {
            auto result = work(stop);
            ui_.post([=, result = std::move(result),
                      publish = std::move(publish)]() mutable {
                if (generations_[key] == generation) publish(std::move(result));
            });
        } catch (...) {
            ui_.post([=, error = std::current_exception()] {
                if (generations_[key] == generation) report(error);
            });
        }
    });
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

52. Memory Budgets, Streaming, and Loading Behavior

A persistent world must make resources available when they are needed while respecting finite memory and transfer bandwidth. Textures, geometry, animations, sounds, UI resources, targets, and temporary buffers compete for space. Loading behavior is what a player observes; streaming, caching, eviction, prefetching, and package boundaries are possible explanations that require resource and timing evidence.

Scope. This chapter is a resource-lifetime framework. It does not confirm Final Fantasy XI's memory budgets, streaming system, package format, or cache policy.

Implementation checkpoint. Build dependency-aware streaming, caches, eviction, and recoverable handling for missing, corrupt, or unsupported records.

```
// Cumulative implementation excerpt: dependency-aware cache with recoverable failure.
std::shared_future<AssetResult> Streamer::request(ResourceId id) {
    if (auto cached = cache_.find(id)) return makeReadyFuture(*cached);
    return jobs_.submit([this, id] -> AssetResult {
        try {
            auto bytes = readResource(id);
            auto asset = decodeAsset(bytes);
            for (ResourceId dependency : asset.dependencies) request(dependency);
            cache_.insert(id, asset, asset.memoryBytes());
            cache_.evictLeastRecentlyUsedUntilWithin(budgetBytes_);
            return asset;
        } catch (const UnsupportedRecord& e) { return Unsupported{e.what()}; }
          catch (const std::exception& e)     { return Failed{e.what()}; }
    });
}
```

52.1 Resource lifetime

A resource can be created, populated, bound, used, updated, released, or retained for reuse. These events can occur at different times and on different threads or subsystems. A resource appearing in a trace after a transition may have been loaded then, created from already available data, or merely bound for the first time in the captured scene. Record the event type rather than calling every first sighting a load.

52.2 Loading and streaming observations

A loading screen, pause, object pop-in, changed texture detail, or delayed animation is an observation of availability or presentation. It may correspond to bulk loading, incremental streaming, visibility selection, decompression, target creation, network state, or another process. Controlled routes and repeated timing samples can narrow the possibilities, but resource creation and update records are needed to establish a pipeline.

52.3 Budgets and tradeoffs

Memory limits encourage choices about texture resolution, mip levels, geometry detail, animation data, temporary surfaces, and reuse. Transfer bandwidth and CPU time influence when those resources can become ready. A renderer may prefer a coarse representation, delay a detail layer, or keep a frequently used resource resident. These are general tradeoffs, not evidence of any one policy in the game.

52.4 Transition studies

Choose a repeatable transition: entering an area, changing a camera route, changing equipment, or triggering a visible effect. Record time, draw presence, resource descriptors, creation/update events, binds, and the visible sequence before and after. Repeat from a cold and a warmed state where possible, while documenting what 'cold' and 'warmed' mean for the test. This separates one-time initialization from recurring availability behavior.

52.5 Evidence record

For each resource claim, record identifier, type, descriptor, first observed creation, updates, bindings, last observed use, scene context, timing samples, and any release or reset evidence. Link the record to a visual change

only when the event order supports it. A missing texture or delayed model is not enough to name an eviction or streaming algorithm.

> Evidence standard. Recorded resource events and measured transitions are confirmed. Terms such as streaming, cache hit, eviction, and prefetch are implementation claims that require a demonstrated policy or code path.

53. Resource Caches and Explicit Lifetime

A long-running explorer repeatedly opens related assets and must reuse them without hiding ownership. This chapter separates source bytes, decoded CPU documents, and device-dependent GPU objects into caches with explicit keys, budgets, pinning, generations, and eviction.

> Build outcome.

Implement layered CPU/GPU caches with stable handles, coalesced loads, budgets, diagnostics, and device-loss recovery.

53.1 Responsibilities and ownership

CacheKey includes installation identity, resource ID, source hash, decoder version, and decode options. CPU and GPU cache entries have independent sizes and lifetimes. A document holds handles, not cache-internal pointers.

53.2 Implementation sequence

Lookup returns a ready handle, joins an in-flight decode, or starts one job. Budget enforcement evicts least-recently-used unpinned entries. Device reset drops only GPU generations; CPU documents remain reusable.

53.3 Failure and recovery

Failed results have short negative-cache lifetimes to prevent retry storms. A changed source hash creates a new key. Pinned workspaces may exceed soft budget but report the excess.

53.4 Verification gate

Tests verify coalesced loads, key separation, deterministic eviction, pinning, source changes, negative expiry, device generation loss, and zero live GPU objects after shutdown.

53.5 Cumulative C++ implementation

> Implementation checkpoint. Implement layered CPU/GPU caches with stable handles, coalesced loads, budgets, diagnostics, and device-loss recovery.

```
// Cumulative implementation listing
void ResourceCache::trim(std::size_t budgetBytes) {
    while (residentBytes_ > budgetBytes) {
        auto victim = std::min_element(entries_.begin(), entries_.end(),
            [](const auto& a, const auto& b) {
                if (a.second.pins != b.second.pins) return a.second.pins < b.second.pins;
                return a.second.lastUse < b.second.lastUse;
            });
        if (victim == entries_.end() || victim->second.pins != 0) break;
        residentBytes_ -= victim->second.bytes;
        entries_.erase(victim);
    }
}
```

```
void GpuCache::onDeviceLost() {
    ++generation_;
    entries_.clear();
    residentBytes_ = 0;
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

54. Performance Tradeoffs and Legacy Hardware Design

A renderer is a collection of tradeoffs, not a list of isolated features. More geometry can compete with more animation work; richer textures can compete with memory and bandwidth; transparency can compete with fill rate and ordering; higher resolution can compete with every pixel-cost operation. Legacy-hardware design is best understood as prioritization under a particular platform and content goal, not as a simple absence of modern techniques.

Scope. This chapter provides a framework for interpreting tradeoffs. It does not establish the original design decisions, hardware budgets, or performance targets of Final Fantasy XI without primary evidence.

Implementation checkpoint. Enforce configurable budgets and benchmark file decoding, resource preparation, rendering, and presentation independently.

```
// Cumulative implementation excerpt: independent budgets and repeatable benchmarks.
struct Budgets { double decodeMs, prepareMs, renderMs, presentMs; std::size_t memory; };

BenchmarkResult benchmarkScene(ClientApp& app, const RegressionScene& scene,
                               const Budgets& b) {
    BenchmarkResult r;
    r.decode = measure([&] { app.assets.decode(scene.resources); });
    r.prepare = measure([&] { app.scene.prepare(scene); });
    r.render = measure([&] { app.renderer.draw(app.scene); });
    r.present = measure([&] { app.renderer.present(); });
    r.memory = app.assets.residentBytes();
    r.withinBudget = r.decode.ms <= b.decodeMs && r.prepare.ms <= b.prepareMs
                  && r.render.ms <= b.renderMs && r.present.ms <= b.presentMs
                  && r.memory <= b.memory;
    return r;
}
```

54.1 The frame budget is shared

CPU preparation, submission, vertex processing, rasterization, texture sampling, blending, targets, presentation, and memory movement all consume parts of a frame budget. Improving one category can worsen another. Combining geometry may reduce draw submissions while making visibility less selective; extra texture stages may improve material detail while increasing sampling and state complexity.

54.2 Content choices and renderer choices

A low-detail distant object, repeated texture, compact effect, or fogged horizon may reflect an art-direction choice, a resource-format constraint, a performance tradeoff, or several of these together. Do not label a visible simplification an optimization unless measurements, design records, or implementation evidence connect it to cost. The same visual technique can serve both aesthetics and performance.

54.3 Platform-aware interpretation

Different platforms can expose different APIs, memory structures, display targets, and driver behavior. A portability decision may preserve shared content while requiring different paths or limits. When comparing versions, identify the observed difference first and then ask whether configuration, platform capability, asset variation, or implementation divergence is supported by evidence.

54.4 Authenticity and modernization

A technical account should distinguish historical behavior from a modern reconstruction or enhancement. Replacing a limitation with a more capable technique can improve an image while no longer describing the original behavior. Conversely, preserving an artifact may reproduce a presentation result without reproducing the original cause. State the goal of a comparison explicitly: observed behavior, inferred mechanism, or visual approximation.

54.5 A tradeoff argument

A strong tradeoff claim names the observed choice, the measured or documented constraint, the plausible alternatives, and the evidence that connects them. For example, it can say that a documented state change reduces a class of work under a measured condition; it should not claim that developers chose it for a particular budget without sources. This standard respects both engineering reality and historical uncertainty.

> Evidence standard. Measured costs and documented platform limits can support a tradeoff analysis. Intent, priority, and original design rationale require primary documentation or direct development evidence.

55. Screenshots, Reports, and Data Export

Export is an evidence boundary, not just a Save button. This chapter writes screenshots, decoded tables, geometry interchange data, textures, and investigation reports with source identity, coordinate conventions, confidence, and omitted-feature warnings.

> Build outcome.

Build transactional screenshot, table, texture, geometry, and report exports with manifests and explicit conversion policy.

55.1 Responsibilities and ownership

ExportRequest freezes a document snapshot, selection, options, destination, and provenance. Exporter implementations write to temporary siblings, validate output, then replace the destination atomically.

55.2 Implementation sequence

The UI validates intent, obtains overwrite approval, starts a cancellable job, and records a manifest. Geometry export declares axes, handedness, units, winding, vertex attributes, materials, and unsupported content.

55.3 Failure and recovery

Cancellation removes temporary output. Existing destinations remain unchanged until commit. Export never copies source game files wholesale or implies ownership of their content.

55.4 Verification gate

Tests cover overwrite policy, cancellation, disk-full simulation, stable manifests, coordinate conversion, image row orientation, selected-only export, and reopening exported diagnostics.

55.5 Cumulative C++ implementation

Implementation checkpoint. Build transactional screenshot, table, texture, geometry, and report exports with manifests and explicit conversion policy.

```
// Cumulative implementation listing
ExportResult ExportService::run(const ExportRequest& r, StopToken stop) {
    const auto temporary = siblingTemporaryPath(r.destination);
    removeIfExists(temporary);
    try {
        ExportManifest manifest = exporters_.at(r.format).write(r, temporary, stop);
        if (stop.stopRequested()) { removeIfExists(temporary); return {false, "cancelled"}; }
        validateExport(temporary, manifest);
        atomicReplace(temporary, r.destination, r.allowOverwrite);
        writeManifest(r.destination, manifest);
        return {true, {}};
    } catch (...) {
        removeIfExists(temporary);
        throw;
    }
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

56. Sessions, Presets, and Reproducible Workspaces

A research tool should reopen an investigation, not merely remember window coordinates. Sessions store installation identity, open resources, workspace types, selections, cameras, time, weather, render controls, and evidence links while treating missing resources as recoverable.

Build outcome.

Implement versioned sessions and focused presets that reproduce an inspection without serializing runtime-only state.

56.1 Responsibilities and ownership

SessionDocument is versioned and contains stable logical identities, never process pointers or GPU handles. Presets are smaller named subsets for cameras, characters, environments, exports, or comparison conditions.

56.2 Implementation sequence

Saving snapshots all workspaces on the UI thread, serializes off-thread, writes transactionally, and records application/schema versions. Restore validates the installation first, opens workspaces asynchronously, then applies state whose dependencies succeeded.

56.3 Failure and recovery

Unknown fields survive migration. Missing resources create placeholder tabs with relink actions. Invalid camera or timing values fall back and enter the restore report.

56.4 Verification gate

Tests cover round-trip, schema migration, unknown preservation, partial restore, relocated installation, conflicting open requests, and deterministic comparison presets.

56.5 Cumulative C++ implementation

Implementation checkpoint. Implement versioned sessions and focused presets that reproduce an inspection without serializing runtime-only state.

```
// Cumulative implementation listing
RestoreReport SessionService::restore(const SessionDocument& saved) {
    SessionDocument current = migrations_.upgrade(saved);
    RestoreReport report;
    for (const WorkspaceSnapshot& ws : current.workspaces) {
        try {
            WorkspaceId id = workspaces_.open({ws.resource, ws.kind});
            workspaces_.whenReady(id, [state = ws.state, &report](IWorkspace& w) {
                report.merge(w.restore(state));
            });
        } catch (const std::exception& ex) {
            report.missing.push_back({ws.resource, ex.what()});
            workspaces_.openPlaceholder(ws, ex.what());
        }
    }
    return report;
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

Part VI — Case Studies, Verification, and Delivery

57. Reading an Opaque Environment Draw

An opaque environment draw is a useful case-study unit because it connects scene selection, geometry, material state, depth behavior, and a visible image contribution. The purpose is not to label a draw from its appearance. It is to build a chain from an observed render event to the smallest supported description of what it does.

> Scope. This is a case-study procedure, not a claim about a specific Final Fantasy XI environment draw.

> **Implementation checkpoint.** Build an opaque environment case study that verifies source bytes, parsed records, resources, state, depth behavior, and draw order.

```
// Cumulative implementation excerpt: opaque draw dossier from bytes to submission.
EnvironmentDrawReport inspectEnvironmentDraw(ResourceId id, const Camera& camera) {
    EvidenceLog evidence;
    const DatFile dat = openDat(id, evidence);
    const Scene scene = buildZone(dat, evidence);
    const auto visible = gatherVisible(scene, camera);
    const Draw draw = firstOpaqueEnvironmentDraw(visible);
    return {id, dat.hash(), draw.sourceChunkOffset, draw.geometry.header,
            draw.material, draw.state, draw.depthKey, draw.sourceOrder,
            evidence.entries()};
}

void verify(const EnvironmentDrawReport& r) {
    require(r.state.depthWrite && !r.state.blendEnabled);
    require(r.geometry.indexCount % 3 == 0 || r.geometry.isStrip);
    require(r.sourceChunkOffset < r.sourceFileSize);
}
```

57.1 Establish the capture context

Begin with platform, client build, zone and landmark, camera position and direction, time and weather, display configuration, and capture method. Identify a stable frame where the target geometry is visible and record nearby occluders, effects, and interface elements. Without this context, a later trace cannot be reliably matched to the image.

57.2 Locate the candidate draw

Use ordered draw evidence and controlled visual changes to associate a candidate draw with a screen contribution. Compare frames that differ by a small camera movement, visibility change, or target condition. Record the draw's target, primitive type and count, buffers and ranges, transform, textures, stages or shader, and depth/culling/fog state. Do not name the object until the correlation is strong.

57.3 Describe the confirmed state

Write a factual description: for example, a triangle draw using this range of a vertex and index buffer, these texture bindings, depth testing and writing, back-face culling, and specified texture-stage operations. This description may be technical but narrow. It is more valuable than a broad material label that cannot be traced to evidence.

57.4 Test alternatives

Ask whether the same draw could be a different surface, a second pass, a shared material group, or a partially hidden object. Change camera, distance, or neighboring conditions one at a time. If the draw persists while the suspected screen region changes, the correlation may be incomplete. If it disappears with the region under documented conditions, confidence increases.

57.5 Produce the case-study result

The final entry should include capture context, trace excerpt or record, confirmed resource/state facts, observed image contribution, alternatives considered, confidence, and next test. A good conclusion might say that the draw is an opaque, depth-writing environment contribution using identified resources; it need not claim the engine's internal object name or batching policy.

> Evidence standard. Case studies should lead with confirmed draw facts, then state only the inference needed to connect those facts to the visible scene.

58. Reading a Character Draw

Character draws are difficult case-study targets because a player-recognized character can be assembled from many components and can change through pose, equipment, distance, effects, and interface markers. A disciplined analysis isolates one render event or small component group, then connects it to the visible character through controlled state changes.

> Scope. This is a character-draw case-study method, not a classification of a specific Final Fantasy XI character path.

> **Implementation checkpoint.** Build a character case study exposing each decoded component, binding, animation, material, equipment, and draw decision.

```
// Cumulative implementation excerpt: character component and decision dossier.
CharacterReport inspectCharacter(const CharacterSelection& choice, float time) {
    EvidenceLog evidence;
    Character c = assembleCharacter(choice, repository());
    Pose pose = evaluatePose(c.skeleton, c.animation, time, evidence);
    std::vector<PartReport> parts;
    for (const CharacterPart& p : c.parts) {
        parts.push_back({p.source, p.equipmentSlot, p.bonePalette,
                         p.material, p.alphaClass,
                         buildCharacterDraw(p, pose).state});
    }
    return {choice, time, pose.hash(), std::move(parts), evidence.entries()};
}
```

58.1 Freeze the character state

Record character identity only as a test label, then document race/body variation if relevant, equipment state, action or pose, animation time, distance, camera, zone, lighting-like conditions, effects, and UI overlays. Choose a stable pose before changing one component. The goal is to avoid confusing animation, occlusion, and equipment variation with a draw association.

58.2 Find component boundaries

Compare captures or traces with one equipment, pose, or visibility condition changed. Look for draws that remain stable, draws that appear or disappear, and draws whose resources or transforms change. Record buffers, ranges,

vertex layout, textures, material states, transforms, blend/depth behavior, and neighboring draws. A component boundary is confirmed by evidence, not by a menu slot or visual silhouette alone.

58.3 Separate rigid and deforming work

A moving draw may be a rigid attachment following a transform, a skinned surface, a separately animated object, or a camera-related effect. Compare vertex attributes, transform changes, constants, shader/fixed-function path, and pose response. State the smallest supported result: for example, that a draw changes with a pose and uses a given layout, rather than naming a skinning method without proof.

58.4 Account for layered materials

Hair, equipment trim, cutouts, transparent ornaments, weapon effects, and shadows can add draws around an otherwise opaque character. Identify target, order, blend and depth state, culling, textures, and overlap behavior. A missing visual piece may be a later transparent draw, a visibility rule, or a resource condition—not evidence that the base mesh changed.

58.5 Character case-study result

Publish the character case as a component ledger: capture context; controlled variation; candidate draws; confirmed resource/state/transform facts; observed response; alternatives; confidence; and next test. Keep player-facing labels separate from technical labels. This makes the result extensible when later evidence reveals additional components or a different assembly relationship.

> Evidence standard. A draw's response to a controlled equipment or pose change can strongly support component association. Internal slot names, skeleton relations, and animation systems require direct data or implementation evidence.

59. Reading a Transparent Effect

A transparent effect is now a complete data-to-pixel case study. The reader should be able to start at a generator's source bytes, follow section-qualified commands into resources and simulation state, and account for each ordered transparent contribution.

> Scope. This procedure applies the decoded generator model without claiming that every opcode or scheduling rule is complete.

59.1 Freeze the evidence context

Record the client build or DAT set, resource identity, file hash, chunk offset, generator header, section boundaries, dependency closure, trigger, camera, actor pose, random seed, and time range. A deterministic seed lets the C++ runtime reproduce the same particle arrangement.

59.2 Build the command ledger

List every {section, opcode}, payload size, raw bytes, decoded meaning, confidence, and referenced resource. Preserve unknown commands in their original order. This ledger distinguishes an unsupported feature from a parser that silently skipped data.

59.3 Reconstruct time

Track scheduler window, generator cadence, emission count, each particle's birth frame, age, lifetime, curve time, and death frame. Include child-generator events. Compare predicted live-particle counts and visible transitions against captures before tuning appearance.

59.4 Reconstruct transforms

For each particle, record attachment transform, local spawn position, velocity integration, rotation, scale, billboard constraint, and final world matrix. A misplaced effect can originate from attachment, coordinate conversion, transform order, or billboarding; the ledger keeps these causes separate.

59.5 Reconstruct rendering

For every draw, record geometry family, card or group selection, texture, vertex color, UV offset, alpha test, blend equation, depth test, depth write, bias, culling, and sort key. Then link the draw to the visible layer it contributes.

59.6 Publish the result

Publish a synchronized table of source command, runtime event, particle state, draw state, and observed pixels at selected frames. Mark omitted operations and alternative interpretations. The goal is a revisionable explanation, not a screenshot accompanied by a confident name.

```
struct EffectFrameRecord {
    std::uint32_t frame;
    std::vector<EmissionRecord> emissions;
    std::vector<ParticleSnapshot> particles;
    std::vector<DrawRecord> draws;
    std::vector<UnknownCommandRecord> unresolved;
};
```

Implementation checkpoint. Export the effect-frame record as machine-readable JSON and a human-readable report so parser changes can be diffed across revisions.

```
// Cumulative implementation excerpt: export one effect frame in two reviewable forms.
void exportEffectFrame(const EffectFrameRecord& frame,
                       const std::filesystem::path& base) {
    JsonWriter json(base.string() + ".json");
    json.object("frame", frame.frame)
        .array("emissions", frame.emissions)
        .array("particles", frame.particles)
        .array("draws", frame.draws)
        .array("unresolved", frame.unresolved);

    std::ofstream text(base.string() + ".txt");
    text << "Frame " << frame.frame << '\n';
    for (const auto& e : frame.emissions) text << describe(e) << '\n';
    for (const auto& d : frame.draws) text << describe(d) << '\n';
    for (const auto& u : frame.unresolved) text << "OPEN: " << describe(u) << '\n';
}
```

Evidence standard. A successful reconstruction explains timing, transforms, resources, and state together. A visual match alone cannot validate the internal model when several mechanisms can produce similar pixels.

60. Reconstructing a Frame from Evidence

A frame reconstruction is an evidence-backed model of how a final image was assembled. It is not a decorative diagram of what seems plausible. The reconstruction should preserve ordered facts, distinguish observed

contributions from inferred pass roles, identify missing links, and remain revisable when a new trace, capture, or resource record changes the picture.

> Scope. This chapter defines a reconstruction workflow. It does not present a confirmed Final Fantasy XI frame graph.

> **Implementation checkpoint.** Create a frame-inspection mode that links file evidence to parsed records, resources, state transitions, and intermediate targets.

```
// Cumulative implementation excerpt: select a draw and walk back to source evidence.
FrameInspection inspectFrame(const CapturedFrame& frame, DrawId selected) {
    const DrawRecord& draw = frame.draw(selected);
    FrameInspection out;
    out.draw = draw;
    out.stateChanges = frame.stateHistory.leadingTo(selected);
    out.resources = frame.resources.boundAt(selected);
    out.targets = frame.renderGraph.ancestry(draw.target);
    for (ResourceId id : out.resources)
        out.sourceEvidence.push_back(frame.evidence.forResource(id));
    return out;
}
```

60.1 Start with a frame dossier

Collect one stable frame context: platform and client build, scene location, camera, time and weather, display path, visible effects, UI state, capture method, and trace range. Store original evidence with stable identifiers. The dossier is the boundary that prevents unrelated observations from being combined into one imagined frame.

60.2 Build the ordered event list

List clear operations, target changes, scene boundaries, resource updates, state transitions, draw calls, and presentation in observed order. Attach descriptors rather than interpretations: target identifier, primitive count, texture bindings, transform, depth and blend state, and visible correlation. If an event’s order is unknown, mark it unknown rather than placing it where a conventional pipeline would expect it.

60.3 Group contributions conservatively

Group draws only where state, resources, geometry, timing, and visual behavior support a relation. A group may be described as opaque depth-writing environment work, a character component set, a blended layer, a UI layer, or an unresolved contribution. Avoid implying a named engine subsystem merely because several draws are adjacent.

60.4 Map evidence to the final image

For each proposed group, state what region or behavior it explains and what it does not explain. Use controlled frame differences to test the map: when a candidate draw changes, does the expected image region change? This process can establish local contributions without requiring every pixel of the frame to be solved at once.

60.5 Report uncertainty and alternatives

A useful reconstruction includes confirmed nodes, strong-inference links, speculative links, and open questions. For each inference, name the competing explanation and the next observation that would distinguish it. A blank or unresolved region is more valuable than a falsely certain pass label because it directs future research.

60.6 Publish a revisionable frame graph

Present the result as a dated evidence graph or ledger with links to captures and records. Keep API events, inferred roles, and player-facing labels in separate fields. When later evidence changes one link, revise only the affected claim and confidence rather than rewriting history. This is how a textbook can remain technically honest while its reconstruction becomes more complete.

> Evidence standard. Event order and state/resource records can confirm parts of a frame. A complete graph remains a model until every claimed link has direct support.

61. Confirmed Architecture, Open Questions, and Competing Models

A technical history should not present every gap as a mystery to be concealed or every plausible explanation as a discovery. Its architecture map must show what is directly established, what is strongly inferred, what remains speculative, and where two or more models explain the same evidence. This chapter defines that publication discipline.

> Scope. This chapter sets the reporting framework. It does not declare a complete confirmed architecture for Final Fantasy XI.

> **Implementation checkpoint.** Maintain architecture, header, and flag registries with provenance, confidence, contradictions, and reproducible next tests.

```
// Cumulative implementation excerpt: versioned claims with contradictions and next tests.
struct Finding {
    std::string key, statement;
    Confidence confidence;
    std::vector<EvidenceRef> evidence;
    std::vector<std::string> contradicts;
    std::string nextTest;
};

class FindingsRegistry {
    std::map<std::string, std::vector<Finding>> history_;
public:
    void publish(Finding f) { history_[f.key].push_back(std::move(f)); }
    const Finding& current(std::string_view key) const {
        const auto& versions = history_.at(std::string(key));
        return versions.back();
    }
    bool hasUnresolvedConflict(std::string_view key) const {
        return !current(key).contradicts.empty();
    }
};
```

61.1 Confirmed architecture

Confirmed architecture consists of claims tied to direct evidence: observed API events, resource descriptors, reproducible state changes, documented platform behavior, recovered structures with validated meaning, or implementation paths whose execution is established for the case. Each claim should have a stable identifier, evidence links, scope, and version or client context. Confirmation is local unless evidence demonstrates broader applicability.

61.2 Open questions

An open question is not a weakly written answer. It is a precisely stated missing link: for example, an unresolved source of a coordinate transform, a draw group whose scene role is unknown, or a target whose content flow is incomplete. State what is known around the gap, what evidence is absent, and what observation would answer it. This turns uncertainty into a research plan.

61.3 Competing models

Competing models are useful when different mechanisms remain compatible with the evidence. A moving surface might involve coordinate scrolling, animated images, or geometry deformation; a distant fade might involve fog, LOD, or blending. Present each model with its predictions, supporting facts, contradictory facts, and discriminating test. Do not choose a winner because one sounds more familiar.

61.4 Confidence can change

New evidence can strengthen, weaken, split, or retire a claim. Preserve earlier records with their original date and scope, then add revisions that explain why confidence changed. This avoids rewriting a speculative history as if it had always been confirmed, and it lets readers judge the quality of the evidence trail.

61.5 The architecture register

Maintain an architecture register with claim ID, subsystem or question, statement, confidence, evidence types and links, scope, alternatives, contradictions, next test, and revision history. Keep descriptive labels separate from internal names unless the mapping is confirmed. The register can support the narrative chapters while remaining precise enough for technical review.

> Publication standard. The book should be comfortable saying 'unknown,' 'strong inference,' and 'competing models.' Credibility comes from the evidence boundary, not from an illusion of completeness.

62. Lessons from Vana’diel’s Visual Technology

A long-lived virtual world is not explained by one renderer feature. Its visual identity emerges from the interaction of geometry, materials, animation, atmosphere, effects, interface composition, platform constraints, and a pipeline that arranges them in time. The central lesson of this book is therefore methodological as much as technical: a convincing image is evidence of a result, not automatically of its cause.

> Scope. This conclusion summarizes the book’s approach. It does not convert unresolved models into confirmed facts about Final Fantasy XI.

Implementation checkpoint. Integrate the verified DAT-reading and rendering systems into a testable visual client with regression scenes and documented extension points.

```
// Cumulative implementation excerpt: integrated visual client and regression scenes.
int VisualClient::run(const ClientConfig& config) {
    graphics_.create(config);
    for (const RegressionScene& fixture : regressionCatalog()) {
        Scene scene = assets_.loadScene(fixture.resource, evidence_);
        renderer_.draw(scene, fixture.camera);
        const Image image = graphics_.captureBackBuffer();
        const ImageDiff diff = compareImages(image, fixture.reference,
                                             fixture.tolerance);
        regression_.record(fixture.name, diff, evidence_.entries());
    }
```

```
    if (!regression_.allPassed()) return EXIT_FAILURE;
    return interactiveLoop();
}
```

62.1 Read the frame as a system

A frame can be understood as scene state, selected work, geometry, material and state rules, ordered composition, and presentation. Each category affects the others. Fog can shape a horizon while changing the interpretation of distance; alpha state can make an effect visible while making its order important; a texture can suggest a material role while its stage operation determines the actual contribution. System thinking prevents isolated visual clues from carrying more meaning than they can support.

62.2 Constraints create style

Hardware and API constraints do not merely remove options. They encourage particular uses of texture, color, fog, geometry reuse, draw ordering, and interface layering. An analysis should respect these pressures without treating every visible choice as a direct consequence of cost. Visual style may be an aesthetic decision, a technical adaptation, or both. The evidence boundary matters even when the historical explanation feels obvious.

62.3 Preserve the distinction between observation and explanation

The most reusable research record begins with what can be observed: a state value, resource binding, draw order, transform, timing change, or controlled pixel result. Explanation comes next and is labeled by confidence. This practice keeps the text useful when new information arrives, because a corrected model does not erase the original observation.

62.4 Make uncertainty productive

Unknowns should become questions with discriminating tests, not empty caveats. Competing models should state their predictions. Negative results should be recorded. A provisional frame graph should mark its unresolved links. This makes future investigation cumulative and allows readers to participate without inheriting unsupported claims.

62.5 A living technical history

The completed textbook should remain revisionable: chapters can gain direct evidence, case studies can be expanded, appendices can collect API references and terminology, and the architecture register can document changes in confidence. English and Japanese editions should preserve the same claim identifiers, evidence labels, and technical referents even when the prose is adapted for clarity. In that form, the work becomes both a guide to graphics technology and a durable record of how the conclusions were reached.

> Final standard. Precision, reproducibility, and candid uncertainty are not limits on the book’s ambition. They are what make its detailed account worth trusting.

62.6 Completion test for the first resource-to-triangle slice

The first vertical slice is complete when one resource identifier resolves through the installation tables; its file hash and chunk walk are recorded; one supported palette texture and one supported unskinned geometry record decode without unchecked reads; the resources upload; an explicit material packet draws indexed triangles; selecting the draw reveals its source fields; and the headless probe reproduces the expected structural summary.

Broader compatibility is not part of this completion claim. Weighted geometry, additional texture types, transformed map geometry, animation, transparent effects, and environment reconstruction advance only through their own fixtures and evidence gates. A small verified client is a stronger foundation than a broad parser that silently guesses.

63. Testing, Fuzzing, and Regression Hardening

A parser and renderer that only works on demonstration files is not a dependable application. This chapter unifies unit fixtures, malformed-input corpora, property tests, decoder fuzzing, structural probes, image comparisons, lifetime checks, and end-to-end workspace tests.

> Build outcome.

Build a layered test runner and fuzz entry point that exercise every decoder and application boundary without requiring interactive operation.

63.1 Responsibilities and ownership

Test assets are identified by hashes and expected legal use. Structural expectations are separate from image expectations. Every failure reports seed, resource identity, decoder version, options, and the narrowest reproducible input.

63.2 Implementation sequence

Unit tests run pure readers and math; probes exercise files without graphics; headless rendering captures fixtures; UI harnesses open workspaces and invoke commands; fuzzers mutate bounded chunks and verify termination and invariants.

63.3 Failure and recovery

A crash, timeout, unchecked allocation, out-of-range read, stale callback, leaked object, or silent unsupported-data acceptance is a failure. Image tolerances are documented per fixture and cannot replace structural assertions.

63.4 Verification gate

The release gate runs deterministic tests in multiple worker configurations, the malformed corpus, device recreation, cancellation storms, session migration, exports, and full shutdown accounting.

63.5 Cumulative C++ implementation

> Implementation checkpoint. Build a layered test runner and fuzz entry point that exercise every decoder and application boundary without requiring interactive operation.

```
// Cumulative implementation listing
int fuzzOneInput(std::span<const std::byte> bytes) noexcept {
    try {
        Budget budget{.maxBytes = 64u << 20, .maxNodes = 100000,
                      .maxDepth = 64, .deadline = Clock::now() + 100ms};
        ByteView view(bytes, budget);
        Recognition r = recognizers().classify(view.prefix(4096));
        for (const Candidate& c : r.candidates) {
            DecoderResult result = decoders().at(c.type).decode(view, budget);
            require(result.bytesConsumed <= bytes.size());
            require(result.nodeCount <= budget.maxNodes);
```

```
        }
    } catch (const BoundedParseError&) {
        return 0;
    } catch (...) {
        recordUnexpectedFailure(std::current_exception(), bytes);
        return 1;
    }
    return 0;
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

64. Packaging, Documentation, and Safe Extension

The final chapter converts a working development build into a maintainable application. It defines runtime prerequisites without prescribing an IDE, first-run behavior, portable settings, diagnostic bundles, decoder extension points, compatibility policy, licensing boundaries, and a release checklist.

Build outcome.

Produce a toolchain-agnostic release layout, manifest, first-run flow, diagnostic bundle, extension registry, and final acceptance checklist.

64.1 Responsibilities and ownership

A release manifest records application version, schema versions, decoder versions, supported capabilities, platform prerequisites, and file hashes. Extensions register recognizers, decoders, workspace factories, exporters, and tests through narrow interfaces.

64.2 Implementation sequence

First run opens without data, explains how to select a lawful local installation, validates it, and offers a catalog scan. Diagnostic bundles include logs and configuration but exclude source asset bytes by default.

64.3 Failure and recovery

An incompatible extension is refused before registration. Unknown settings and session fields are preserved. Upgrades never silently rewrite or redistribute source data.

64.4 Verification gate

The completion gate installs or copies the packaged output into a clean user directory, runs first-run setup, catalogs a fixture installation, opens every workspace family, exports diagnostics, restores a session, and uninstalls or removes application-owned data cleanly.

64.5 Cumulative C++ implementation

Implementation checkpoint. Produce a toolchain-agnostic release layout, manifest, first-run flow, diagnostic bundle, extension registry, and final acceptance checklist.

```
// Cumulative implementation listing
bool ExtensionRegistry::registerExtension(const Extension& e,
                                          const ReleaseManifest& host) {
    if (!e.requiredHost.contains(host.applicationVersion)) return false;
    if (e.abiVersion != host.extensionAbi) return false;
```

```
    for (auto& recognizer : e.recognizers) recognizers_.add(recognizer);
    for (auto& decoder : e.decoders) decoders_.add(decoder);
    for (auto& workspace : e.workspaces) workspaces_.add(workspace);
    for (auto& exporter : e.exporters) exporters_.add(exporter);
    return true;
}

DiagnosticBundle makeDiagnosticBundle(const Application& app) {
    DiagnosticBundle b;
    b.addText("manifest.json", app.releaseManifest().toJson());
    b.addText("settings-redacted.json", app.settings().redactedJson());
    b.addText("diagnostics.json", app.diagnostics().toJson());
    return b; // Source resource bytes are excluded by default.
}
```

Evidence boundary. The code is an independent implementation of the book's contracts. File-specific meaning is limited to the scope confirmed by the corresponding evidence notes.

Appendix A. Graphics API Reference

This appendix is a compact reference to the Direct3D 8 vocabulary used in the book. It describes API capabilities and common interface roles. It must not be read as a list of calls confirmed for Final Fantasy XI; a game-specific claim still requires the evidence standard defined in Chapters 4 and 61.

Reference boundary. API terminology is confirmed by platform documentation. Client usage is a separate question.

A.1 Core interfaces

IDirect3D8 represents the Direct3D object used to enumerate adapters and create a rendering device. IDirect3DDevice8 is the primary rendering interface: it creates resources, sets state, binds streams and textures, issues draw calls, manages scenes, and presents images. Resource interfaces include IDirect3DTexture8, IDirect3DVertexBuffer8, IDirect3DIndexBuffer8, and surface-oriented resources.

A.2 Device creation and presentation

IDirect3D8::CreateDevice creates an IDirect3DDevice8 under selected presentation and behavior parameters. A conventional scene sequence uses IDirect3DDevice8::BeginScene, draw and state methods, IDirect3DDevice8::EndScene, and IDirect3DDevice8::Present. Present transfers the next back buffer through the presentation path. Target and depth surfaces can be queried or changed through device methods.

A.3 Resource creation

Representative creation methods include CreateTexture, CreateVertexBuffer, CreateIndexBuffer, CreateRenderTarget, CreateDepthStencilSurface, and CreateImageSurface. Resource descriptors express dimensions, formats, usage, and pool. Lock and Unlock operations are relevant when application code reads or updates eligible resources.

A.4 Transforms, streams, and drawing

SetTransform establishes standard transform categories. SetStreamSource binds vertex data and stride; SetIndices binds index data. SetVertexShader selects an FVF or programmable vertex-shader path; SetVertexShaderConstant supplies constants for programmable processing. DrawPrimitive and DrawIndexedPrimitive submit geometry interpreted according to primitive type and active state.

A.5 State and texture-stage methods

SetRenderState controls device-level rendering conditions such as depth, culling, fog, alpha testing, and blending. SetTexture binds a texture to a stage. SetTextureStageState controls fixed-function color, alpha, coordinate, and other stage operations. SetRenderTarget changes the rendering destination. State-block methods can capture and apply groups of state.

A.6 How to cite API activity

When a trace contains one of these methods, record the exact interface/method, parameters or resulting descriptor, order relative to nearby calls, client/build context, and visual correlation. Do not translate a method directly into an artistic label. For example, SetTexture records a binding; it does not by itself establish a diffuse map, light map, or effect pass.

> Research rule. Preserve the original API identifier in both language editions; translate the explanation, not the identifier.

A.7 Toolchain-agnostic compilation checklist

Language mode: C++20 or later with exceptions enabled. Platform declarations: Windows and Direct3D 8. Linkage: the operating-system libraries required by the window functions and the import library that provides the Direct3D 8 entry points. Character entry: Listing 7.4 uses an ordinary main function and obtains the process instance at runtime, so it does not require a particular IDE project template.

Treat each complete checkpoint listing as a separate executable. In the cumulative client, move its declarations and definitions into the responsibility modules from Section 1.7 and retain one entry point. Never compile several checkpoint entry points into the same executable.

Appendix B. Render-State Reference

This reference groups commonly discussed Direct3D-era render-state concepts by their visual role. It is a reading aid for traces and chapters, not a list of values confirmed for a particular client or material.

> Reference boundary. A state category is API vocabulary; an observed value is evidence only for the recorded draw and context.

B.1 Depth and rasterization

Depth testing compares a candidate fragment with stored depth. Depth writing updates stored depth after acceptance. Depth function determines the comparison rule. Depth bias-like settings can alter coplanar behavior. Fill mode selects point, wireframe, or filled primitive presentation. Clipping and related rasterization choices influence which geometry can reach later stages.

B.2 Culling and orientation

Cull mode determines whether front-facing, back-facing, or neither class of triangle is rejected. Its interpretation depends on vertex winding and transform conventions. Culling is distinct from object-level visibility selection and from depth rejection after rasterization.

B.3 Fog

Fog enablement, mode, color, and start/end or density-like parameters govern a fog contribution where supported. Fog can be vertex- or pixel-related in conceptual terms depending on the path, and can be enabled for some draws but not others. Confirm the active state and associated conditions before calling a visible fade fog.

B.4 Alpha test

Alpha testing enables a comparison between an alpha value and an alpha reference. The alpha function specifies the comparison. A failed fragment is discarded rather than partially blended, producing cutout behavior when other state permits.

B.5 Blending

Alpha blending enables combination of source and destination colors. Source and destination blend factors determine weighting; separate alpha behavior may have its own rules depending on the API path. Blend state must be read with target, ordering, and depth state because the destination is the image accumulated so far.

B.6 Shading and color behavior

Shading mode, lighting enablement, color-vertex behavior, material source choices, ambient-like values, and related settings affect how vertex and material inputs become current color. Texture-stage operations can then extend or replace that result. Record these state categories together when analyzing a material.

B.7 Trace notation

For each state record, use the symbolic state name, observed value, draw identifier, target, order, client context, and confidence. Preserve unknown values explicitly. A compact state diff is often more informative than a full unannotated state dump because it shows what changed between related draws.

> Research rule. Never infer a render state solely from a visual effect when the state can be recorded directly.

Appendix C. Texture-Stage and Combiner Reference

Fixed-function texture stages assemble a material by combining texture samples, diffuse or vertex color, current results, and other configured arguments. This appendix supplies compact notation for recording those operations. It is an API reference, not evidence that a particular material or effect used a particular sequence.

> Reference boundary. Record exact stage state first; assign material roles only after scene and resource correlation.

C.1 Stage inputs

Common conceptual inputs include TEXTURE, DIFFUSE, CURRENT, TEMP where supported, and complements or alpha replicas of these values. TEXTURE is sampled from the texture bound to the stage. DIFFUSE commonly refers to a vertex-derived diffuse color. CURRENT is the result accumulated from prior stage processing. The exact valid argument set depends on the operation and API path.

C.2 Color and alpha operations

Color operations and alpha operations are configured separately. Useful conceptual operations include SELECTARG1, SELECTARG2, MODULATE, ADD, SUBTRACT-like variants, and interpolation-like operations. A stage can modulate color while choosing alpha from one source, or can disable a direction of processing. Preserve operation and every argument in a trace record.

C.3 Coordinate selection

A stage may use a selected texture-coordinate set and a coordinate transform. Coordinates can be passed through, transformed, generated from another source, or otherwise configured according to stage state. Record the coordinate source, transform flags, and the mesh attributes available to the stage before calling a pattern tiled, projected, or scrolling.

C.4 Sampler-related state

Addressing, minification and magnification filtering, mip filtering, and related settings affect the sample supplied to a stage. These settings can create repetition, seams, blur, sharpness, or distance behavior. They are distinct from the combiner operation that consumes the sample.

C.5 Symbolic trace notation

For each active stage, write an expression such as C1 = MODULATE(TEXTURE0, DIFFUSE) and A1 = SELECTARG1(TEXTURE0), then feed C1/A1 into the next stage as CURRENT where appropriate. Mark any unknown input or unsupported operation explicitly. The expression is a readable restatement of recorded state, not a substitute for it.

C.6 Evidence checklist

Record stage index, bound texture, coordinate source, coordinate transform, color operation and arguments, alpha operation and arguments, sampler state, active vertex attributes, draw identifier, and resulting visual correlation. A later reader should be able to reproduce the combiner expression without relying on a prose material label.

> Research rule. Do not call a stage a light map, detail map, reflection map, or mask solely from its position in the chain.

Appendix D. Matrix and Coordinate-Space Primer

This appendix provides compact mathematical vocabulary for transforms used in real-time graphics. Conventions vary: row versus column vectors, multiplication order, handedness, and depth ranges must be verified for a particular API path or client. Use the symbols here as explanatory notation, not as an assumed implementation convention.

> Convention boundary. A matrix equation is a model until the relevant convention and values are observed.

D.1 Points, vectors, and homogeneous coordinates

A point represents a location; a vector represents a direction or displacement. Homogeneous coordinates add a fourth component so that translation, rotation, scale, and projection-like operations can be represented with

matrices. In common notation, a position uses w = 1 and a direction uses w = 0, but the storage and multiplication convention still matter.

D.2 Local-to-world transform

A local position p_local is placed in world space by a world transform W. In one notation: p_world = p_local × W. Another convention writes W × p_local. Do not mix these forms. The essential idea is unchanged: W places reusable local geometry at a scene position, orientation, and scale.

D.3 View and projection transforms

A view transform V expresses world positions relative to the camera. A projection transform P maps camera-relative coordinates toward clip space. A conceptual sequence is p_clip = p_local × W × V × P under one multiplication convention. Perspective division then yields normalized screen-related coordinates before viewport mapping.

D.4 Hierarchies

For an attachment or bone hierarchy, a child's final transform combines its local transform with a parent's accumulated transform. The multiplication order depends on convention, but the dependency is the key fact: changing a parent can move every descendant. Record local and parent relationships separately when analyzing animation.

D.5 Texture coordinates

Texture coordinates are usually two-dimensional values (u, v), often extended for transforms. A texture transform can scale, rotate, translate, or project those coordinates independently of object position. When an image scrolls on a static mesh, a coordinate transform is one candidate explanation—not a conclusion without stage and transform evidence.

D.6 Practical notation

For each recorded transform, identify source space, destination space, matrix or constant identifier, multiplication convention when known, and affected draw. Prefer labels such as local → world and world → view over ambiguous names. Mark unknown convention explicitly rather than silently transposing a recovered matrix to fit a visual expectation.

> Research rule. A matrix value without source/destination spaces is incomplete evidence.

Appendix E. Blend, Depth, and Alpha Formula Reference

This appendix uses compact formulas to describe common depth, alpha-test, and blending behavior. The formulas are conceptual reading tools: API details, color representation, target format, and active state determine the actual result of a draw.

> Reference boundary. Do not apply a formula until the relevant state, target, and ordering have been recorded.

E.1 Depth comparison

Let z_s be source depth and z_d be stored destination depth. A depth function decides whether a fragment proceeds, for example by a relation such as z_s < z_d or z_s ≤ z_d under a particular convention. If depth writing is enabled and the fragment is accepted, z_d may be replaced by z_s. The direction of the comparison and meaning of near/far must be verified for the path under study.

E.2 Alpha testing

Let α_s be the source alpha and α_ref the configured reference. An alpha test evaluates a comparison function f(α_s, α_ref). If false, the fragment is discarded; if true, it continues to later operations. This is binary acceptance at the tested fragment, even though filtering can make the visible edge appear soft at a larger scale.

E.3 General color blending

A common conceptual form is C_out = C_s × F_s + C_d × F_d, where C_s is source color, C_d is destination color, and F_s/F_d are the configured source and destination factors. Factors can depend on source alpha, destination alpha, colors, or constants according to active blend state. The equation is applied only after earlier tests and operations allow the fragment to reach blending.

E.4 Familiar special cases

Source-alpha-like translucency can be represented conceptually as C_out = C_s × α_s + C_d × (1 − α_s). Additive-like accumulation can be represented as C_out = C_s × F_s + C_d × 1 for a chosen source factor. These names are convenient shorthand, but exact factors, clamping, and alpha-channel behavior must be read from the trace.

E.5 Ordering and destination

The destination C_d is the target color at the moment the draw is blended. Therefore an identical source draw can produce a different result after a different opaque draw, transparent draw, clear color, or target pass. Blend equations are not complete without target and order. This is why a transparent effect requires a pass ledger rather than a texture image alone.

E.6 Formula record

For every formula used in an analysis, record source resource and stage result, alpha-test enablement/function/reference, blend enablement/factors, depth test/function/write, target, order, and color/format assumptions. State which terms are observed and which are inferred. This makes an equation auditable rather than decorative.

> Research rule. A familiar visual result does not license a familiar blend equation; verify the factors and destination chain.

Appendix F. Evidence Register Template

Use one register entry for one bounded technical claim. The entry should make a result independently reviewable: another reader should be able to understand what was observed, under what conditions, what it supports, what remains possible, and how to test it further.

> Template rule. Do not use this form to make an uncertain claim appear formal. Missing evidence belongs in the entry.

F.1 Identity

Claim ID: a stable identifier, for example RV-YYYY-NNN. Title: a short neutral description. Status: confirmed, strong inference, or speculation. Revision: date, author/editor, and link to any superseded entry.

F.2 Claim and scope

Claim: one sentence limited to what the evidence supports. Scope: platform, client/build, scene or asset context, and conditions to which the claim applies. Do not silently generalize a local observation to all zones, models, platforms, or versions.

F.3 Observation record

Observation: what directly occurred or appeared. Context: location/landmark, camera, time/weather, display configuration, UI/effect state, and capture method. Evidence type: direct capture, controlled comparison, API trace, resource inspection, implementation analysis, documentation, or another stated source. Attach stable file, image, trace, or citation identifiers.

F.4 Technical record

Record only fields relevant to the claim: target, draw order, geometry/buffer ranges, transforms, textures, stages or shader, render states, blend/depth behavior, resource updates, timings, and output changes. Preserve original identifiers and values. Mark unavailable fields as unknown rather than replacing them with an assumed label.

F.5 Alternatives and confidence

Alternatives: list explanations still compatible with the evidence. Contradictions: list observations that do not fit cleanly. Confidence rationale: explain why the status is confirmed, strong inference, or speculation. A confirmed narrow fact can coexist with speculative explanation of its purpose.

F.6 Next test and publication text

Next test: the smallest observation or experiment that would discriminate among alternatives. Publication text: a reader-facing sentence using the correct confidence label. Source links: primary documentation, captures, trace records, and related claim IDs. Revision note: record what changed and why.

> Example neutral form. 'RV-2026-014: Under the recorded Windows client/build and camera conditions, draw D-184 uses these observed blend and depth states. Its contribution to the visible glow is a strong inference; the target sequence remains unresolved.'

Appendix G. Glossary

This glossary fixes the working meanings of recurrent terms. API identifiers remain in their standard English form. A glossary definition is explanatory, not evidence that the named feature occurs in a particular client.

> Terminology rule. Preserve claim confidence separately from vocabulary: a precise term can still describe a speculative model.

G.1 Core terms

Alpha test — A binary fragment comparison against an alpha reference. Alpha blending — Combining source and destination colors through blend factors. Back buffer — A device-owned surface prepared for presentation. Blend factor — A value that weights source or destination contribution. Culling — Rejecting geometry by orientation or broader visibility rules; specify which kind.

Draw call — A command that submits primitives under active resources and state. Fragment — A candidate pixel-level result before final target write. Geometry — Vertex and index data interpreted as primitives. Material — The active resource and state recipe that determines a draw's surface contribution. Pass — An ordered rendering contribution; use only with evidence of order and target context.

G.2 Coordinate and scene terms

Local space — Coordinates relative to an object. World space — Coordinates placing objects in a scene. View space — Coordinates relative to a camera. Projection — Mapping from camera-related coordinates toward screen space. Frustum — The camera-defined volume considered for viewing. Transform — A mapping between spaces or positions, commonly represented by a matrix.

Zone — A player-facing area label; do not assume it equals a render partition. Chunk — An analytical label for a spatial subset; confirm internal meaning before using it as a fact. Draw group — A set of submissions treated together by observed state/resource/order evidence.

G.3 Resource and pipeline terms

Texture — Sampled image data bound to a stage or material. Surface — Two-dimensional device storage, possibly a texture level, target, or depth surface. Mipmap — A sequence of reduced-resolution texture levels. Vertex buffer — A resource containing vertex records. Index buffer — A resource containing vertex references. Texture stage — A fixed-function unit that combines configured inputs. Render state — Device setting affecting a draw's interpretation.

G.4 Evidence terms

Confirmed — Directly supported by reproducible observation, primary documentation, implementation evidence, or equivalent decisive evidence. Strong inference — Best-supported explanation with a remaining decisive gap. Speculation — Plausible but unproven model. Controlled comparison — Test that changes one relevant variable while recording conditions. Evidence register — Structured record connecting a claim to its context, sources, alternatives, and confidence.

> Editorial rule. If a term's meaning is uncertain in a source, quote or preserve the source term and mark the interpretation as unresolved.

Appendix H. Chronology of Technical Findings

This appendix is a dated research ledger, not a chronology of guesses. Each entry records when a claim entered the textbook, what evidence supported it, its confidence at that time, and what revision later changed it. It should contain no client-specific finding until the corresponding evidence-register entry exists.

> Chronology rule. Preserve the date and original confidence of a claim; add revisions rather than rewriting its history.

H.1 Entry format

Date — Claim ID — Short finding — Scope — Evidence types — Confidence — Source/record links — Revision status. Use ISO date format (YYYY-MM-DD) for sorting. If the event date of an observation differs from the date the claim is entered, record both.

H.2 Initial editorial milestones

2026-08-09 — RV-ED-001 — English and Japanese editions established with aligned chapter, appendix, evidence-label, and terminology structure. Scope: textbook editorial framework. Evidence: document revision record. Confidence: confirmed.

2026-08-09 — RV-ED-002 — The main text establishes a separation between confirmed findings, strong inferences, and speculation. Scope: editorial policy. Evidence: Chapters 1–5 and Appendix F. Confidence: confirmed.

2026-08-09 — RV-ED-003 — Direct3D 8 API vocabulary is documented as reference context, while client-specific usage remains unentered pending evidence. Scope: API reference policy. Evidence: Appendix A and primary API documentation. Confidence: confirmed.

H.3 Client-specific finding policy

Before adding a client-specific entry, create the evidence-register record first. The chronology entry should then summarize that record rather than duplicate it. Include the platform, client/build, scene or asset scope, and confidence. If a later test contradicts the finding, add a new dated revision entry that links to both records.

H.4 Example future entries

YYYY-MM-DD — RV-YYYY-NNN — Observed draw/state/resource fact. Scope: platform, client/build, scene. Evidence: trace and controlled capture. Confidence: confirmed. Revision: none.

YYYY-MM-DD — RV-YYYY-NNN — Best-supported explanation of observed behavior. Scope: stated conditions. Evidence: controlled comparison plus trace. Confidence: strong inference. Next test: stated discriminating observation.

> Current status. No game-client-specific technical findings are entered in this chronology yet; this preserves the book’s stated standard of evidence.

Appendix I. Bibliography and Primary Sources

This bibliography records primary and official technical sources used for platform context and API vocabulary. It is intentionally narrow at this stage. New client-specific claims should add the exact evidence record, capture, trace, document, or primary source that supports them.

> Citation rule. Cite the source that supports the claim, not merely a page that discusses a related topic.

I.1 Official Final Fantasy XI sources

Square Enix. FINAL FANTASY XI — WE ARE VANA'DIEL 20th Anniversary Commemorative Website. Adventurers' History. https://we-are-vanadiel.finalfantasyxi.com/history/?lang=en Accessed 2026-08-09. Used for high-level platform and service-history context.

Square Enix. Switching from the PlayStation 2 or Xbox 360 Version to the Windows Version. PlayOnline. https://www.playonline.com/ff11us/envi/win/winshift.html Accessed 2026-08-09. Used for the official 2016 platform-service transition notice.

I.2 Official Direct3D 8 references

Microsoft. IDirect3DDevice8. Microsoft Learn, archived documentation. https://learn.microsoft.com/en-us/previous-versions/windows/embedded/ms889277%28v%3Dmsdn.10%29 Accessed 2026-08-09. Used for device-interface and method-group reference.

Microsoft. IDirect3DDevice8::BeginScene. Microsoft Learn, archived documentation. https://learn.microsoft.com/sr-latn-rs/previous-versions/ms889279%28v%3Dmsdn.10%29 Accessed 2026-08-09. Used for scene-boundary reference.

Microsoft. IDirect3DDevice8::Present. Microsoft Learn, archived documentation. https://learn.microsoft.com/en-us/previous-versions/ms889707%28v%3Dmsdn.10%29 Accessed 2026-08-09. Used for back-buffer presentation reference.

Microsoft. IDirect3DDevice8::CreateTexture. Microsoft Learn, archived documentation. https://learn.microsoft.com/en-us/previous-versions/ms889290%28v%3Dmsdn.10%29 Accessed 2026-08-09. Used for texture-resource reference.

Microsoft. IDirect3DDevice8::CreateVertexBuffer. Microsoft Learn, archived documentation. https://learn.microsoft.com/en-us/previous-versions/ms889291%28v%3Dmsdn.10%29 Accessed 2026-08-09. Used for vertex-buffer reference.

I.3 Evidence-register sources

For future technical claims, cite the evidence register first, then the underlying primary materials: original capture, trace, resource descriptor, controlled comparison, executable-analysis note, official documentation, or an explicitly identified archival source. Do not cite an explanatory chapter as the only support for a claim that the chapter itself is evaluating.

> Current status. The bibliography supports the book’s platform/API reference context. It is not yet a source list for client-specific rendering findings.

Appendix J. Index

This subject index uses chapter and appendix references instead of page numbers so it remains valid while the working manuscript changes. Entries identify discussion locations, not confirmation status; consult the referenced section’s evidence label before treating a term as a game-specific fact.

> Index rule. Add an entry when a term is substantively explained, not merely mentioned.

J.1 A–D

J.2 E–M

J.3 N–S

J.4 T–Z

> Maintenance note. When a future chapter adds a confirmed client-specific finding, add its claim ID alongside the chapter reference so readers can locate the evidence record quickly.